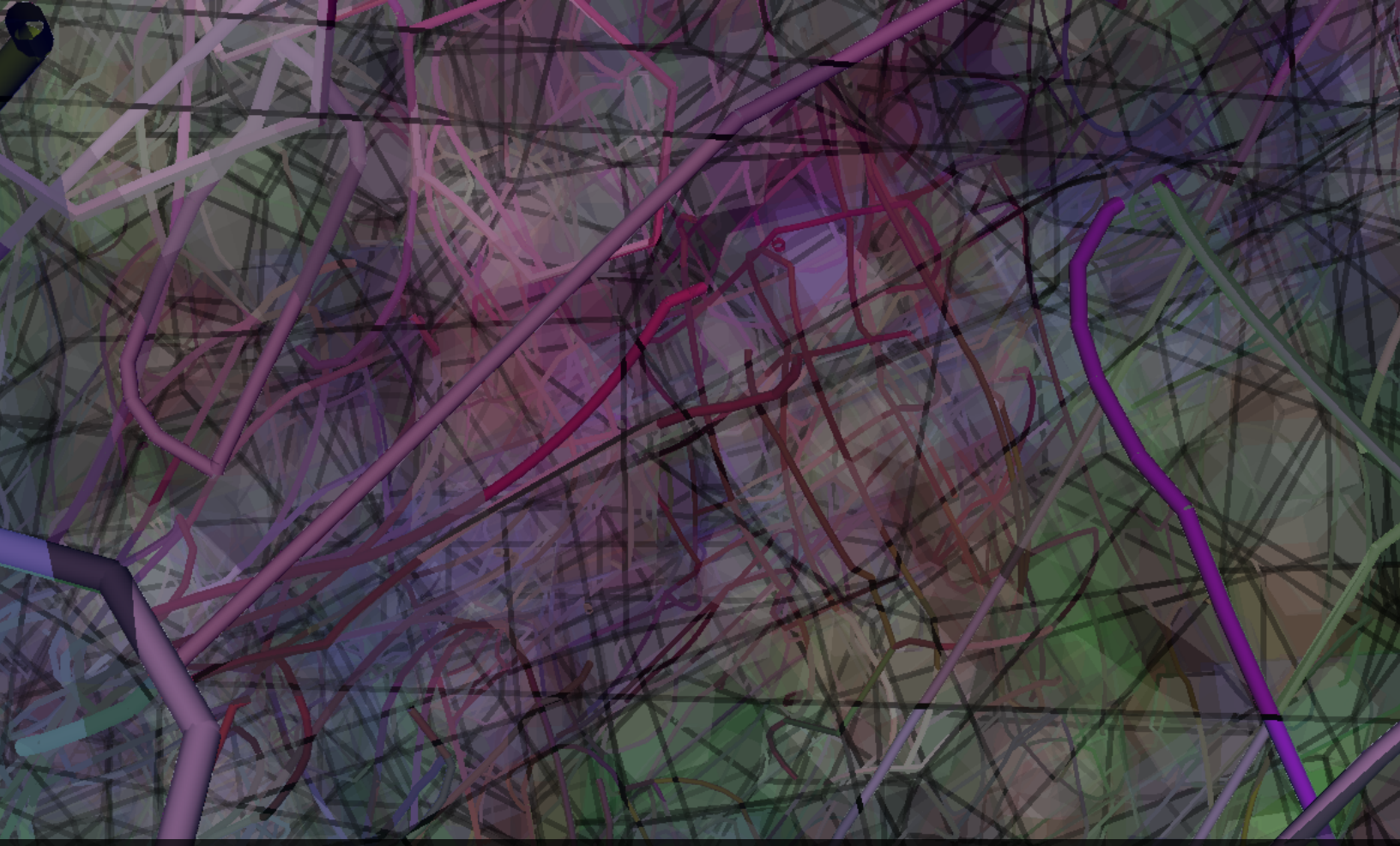


MoDELib

Mechanics of Defects Evolution Library

Architecture, Theory and User Guide

Code version 2.0.0 — commit `eeaa727` + local changes

21 October 2026



Preface

MoDELib—the Mechanics of Defects Evolution Library—provides a framework for modeling defects in crystalline materials and their interactions with mechanical and other physical fields. Understanding such simulations requires both a physical description of the problem and a clear account of how that description is represented in the code. Our aim with this manual is to bring these two perspectives together.

We have organized the manual around the architecture of the library. It begins with the geometric foundations: simplicial and polyhedral meshes, their connectivity, and periodic identification. Materials and polycrystals then give this geometry its crystallographic meaning. The finite-element infrastructure supplies interpolation, weak forms, and field constraints, while the multiphysics architecture connects the individual physical models through shared data and a common simulation lifecycle.

This order is intended to help readers build an understanding of the software before entering the details of a particular model. Theory is introduced where it explains an implementation choice or a physical assumption. Figures and short code examples connect the mathematical objects to their computational counterparts. The appendices collect longer dislocation-theory derivations and supporting identities, so that these remain available without interrupting the architectural discussion.

The manual is also a working reference. Some interfaces continue to evolve, and the cluster-dynamics formulation is being revised. The code version identified on the cover therefore provides the context for the descriptions and examples presented here. As the library develops, we intend this document to preserve the connection between the governing equations, the design of the software, and the practical choices needed to construct and interpret a simulation.

Giacomo Po

21 October 2026

How to use this manual

The manual follows the construction of a simulation: geometry and periodicity first, then materials and grains, finite elements, and finally the physics that share those services. Each architectural chapter explains responsibilities and data flow, introduces only the theory needed to understand them, and identifies the input files or source classes involved.

Part I describes the shared infrastructure. Part II introduces the multiphysics contract and the elastic-deformation and inclusion modules. Part III treats the architecture of discrete dislocation dynamics (DDD). Part IV collects the longer DDD derivations and mathematical notes as appendices. Cluster dynamics (CD) is deliberately placed last, in Appendix F; its current interfaces are described briefly while the model is being revised.

The architecture chapters were checked against the repository working tree on 21 October 2026, including the in-progress `PolyhedralMesh` implementation. This is a snapshot, not a claim that those changes have been released. Source paths are relative to the repository root. Example input fragments modify copies of templates and are not complete input decks unless stated otherwise.

The retained DDD derivations are a research reference. Editorial reorganization does not constitute a mathematical validation of every inherited equation. Explicit status notes identify incomplete derivations; the original source files are retained in `manual_beta/archive/`. Schematics and analytical plots explain relationships, and are distinguished from numerical simulation output in their captions.

Contents

Preface i

How to use this manual ii

I Geometry, materials and finite elements 1

1 Simulation Domains and Meshes 2
- 1.1 Generating meshes . . . 2
- 1.2 MoDELib mesh architecture . . . 4
- 1.3 Simplices and shared topology . . . 4
- 1.4 Polyhedral cells and oriented incidence . . . 6
- 1.5 Mesh access from Python . . . 6
- 1.6 Plane sections and lifetime . . . 7
- 1.7 Preparing and checking a mesh . . . 8
- 1.8 Geometric periodicity . . . 8

2 Materials and polycrystals 10
- 2.1 Shared material, grain-specific crystallography . . . 10
- 2.2 Units and temperature . . . 10
- 2.3 Grains, rotations and boundaries . . . 11
- 2.4 Input workflow . . . 11

3 Finite-element infrastructure 13
- 3.1 Geometry, interpolation and fields . . . 13
- 3.2 The weak-form contract . . . 13
- 3.3 Periodic equivalence classes and field constraints . . . 14
- 3.4 Physical boundaries and rigid modes . . . 14
- 3.5 Enabling and diagnosing FEM . . . 15

II Multiphysics and shared field physics 17

4 The multiphysics architecture 18
- 4.1 Shared context and evolving state . . . 18
- 4.2 Lifecycle and common field queries . . . 18
- 4.3 One simulation step . . . 18

4.4 Selecting and extending physics . . . 19

5 Running a simulation 20
5.1 Build the tools . . . 20
5.2 The simulation folder . . . 20
5.3 A first DDD run . . . 20
5.4 Outputs and restart . . . 21
5.5 Periodic mesh input . . . 21

6 Elastic-deformation physics 22
6.1 Uniform and finite-element formulations . . . 22
6.2 Loading controls . . . 22
6.3 FEM field state . . . 22

7 Inclusion physics 24
7.1 Objects and field contributions . . . 24
7.2 Initialization and coupling . . . 24
7.3 User workflow . . . 24

III Discrete dislocation dynamics 25

8 Discrete dislocation dynamics: architecture 26
8.1 State and responsibilities . . . 26
8.2 Forces, velocities and updates . . . 26
8.3 Periodic dislocation continuity and image fields . . . 27
8.4 DDD input and resolution . . . 27
8.5 Guide to the retained theory . . . 27

IV Appendices 28

A DDD: elastic theory of discrete dislocations 29
A.1 Eigendistortion theory in elastic media . . . 29
A.2 Discrete dislocation loops . . . 34

B DDD: regularization of singular fields 42
B.1 Stress kernel . . . 42

C DDD: fields of piecewise-straight loops 44
C.1 Straight segments . . . 44
C.2 Stress field of a straight segment . . . 48
C.3 Straight Displacement . . . 51

D DDD: mathematical identities 54
D.1 Mathematical preliminaries . . . 54
D.2 Elastic energy of a dislocation loop: working derivation . . . 56
D.3 Convolution integrals . . . 57

D.4 The nabla operator . . . 57
D.5 The Green's tensor and the F-tensor in classical anisotropic elasticity . . . 57

E DDD: supplementary derivations **60**
E.1 Kunin identity . . . 60
E.2 Distributional identities for loop fields . . . 60
E.3 Uniform loading: retained derivation . . . 61

F Cluster dynamics: current interface and revision boundary **62**
F.1 Connection to the common architecture . . . 62
F.2 Current user entry points . . . 62
F.3 What to document after the revision . . . 62

Part I

Geometry, materials and finite elements

Chapter 1

Simulation Domains and Meshes

1.1 Generating meshes

A simulation begins with a mesh file describing the specimen geometry, volume elements and region IDs. Simple specimens can be generated with Gmsh; more elaborate polycrystalline specimens can be generated with Neper. After generation, MoDELib reads the mesh and constructs the connectivity and grain-boundary topology used by the simulation.

1.1.1 Simple geometries with Gmsh

Gmsh provides geometry definition and finite-element mesh generation. For cubes, cylinders and simple partitions, start from the geometry files `Library/Meshes/unitCube.geo`, `cylinder.geo` or `bicrystal.geo`. For example, from the repository root:

```
gmsh Library/Meshes/unitCube.geo -3 -order 1 -format msh2 -o mesh.msh
```

This writes a tetrahedral mesh to `mesh.msh`; changing the geometry's size parameters changes the discretization. The current reader uses ASCII Gmsh version 2 records and takes the second element tag as the region ID. Preserve distinct volume entities for different grains. See the Gmsh reference manual for geometry and meshing options.

1.1.2 Polycrystals and periodicity with Neper

Neper generates grain tessellations and meshes them, including periodic and partially periodic polycrystals. Its tessellation and meshing stages are separate:

```
neper -T -n 10 -periodicity all -o poly10
neper -M poly10.tess -order 1 -format msh:ascii
```

These commands illustrate a ten-grain periodic tessellation and its mesh; they are not recipes for reproducing a particular stored library file. Neper's meshing stage uses Gmsh for tetrahedral meshing. The tessellation documentation describes grain morphology and periodicity, and the meshing documentation describes element size and output formats.

Neper's periodicity selector accepts all directions or a subset of its generation axes, such as `x,y`. Periodic directions in the simulation need not align with the global Cartesian axes: rotate a periodic mesh into the desired frame, or apply the nonsingular import transformation $\boldsymbol{F}$ introduced below. A source translation $\boldsymbol{a}_k$ then becomes $\boldsymbol{F}\boldsymbol{a}_k$, allowing oblique directions as well. The same transformation must act on the whole mesh so that paired nodes and triangulations still match. MoDELib activates the desired geometric pairing when the mesh is constructed; generating a periodic file alone does not select a field boundary condition.

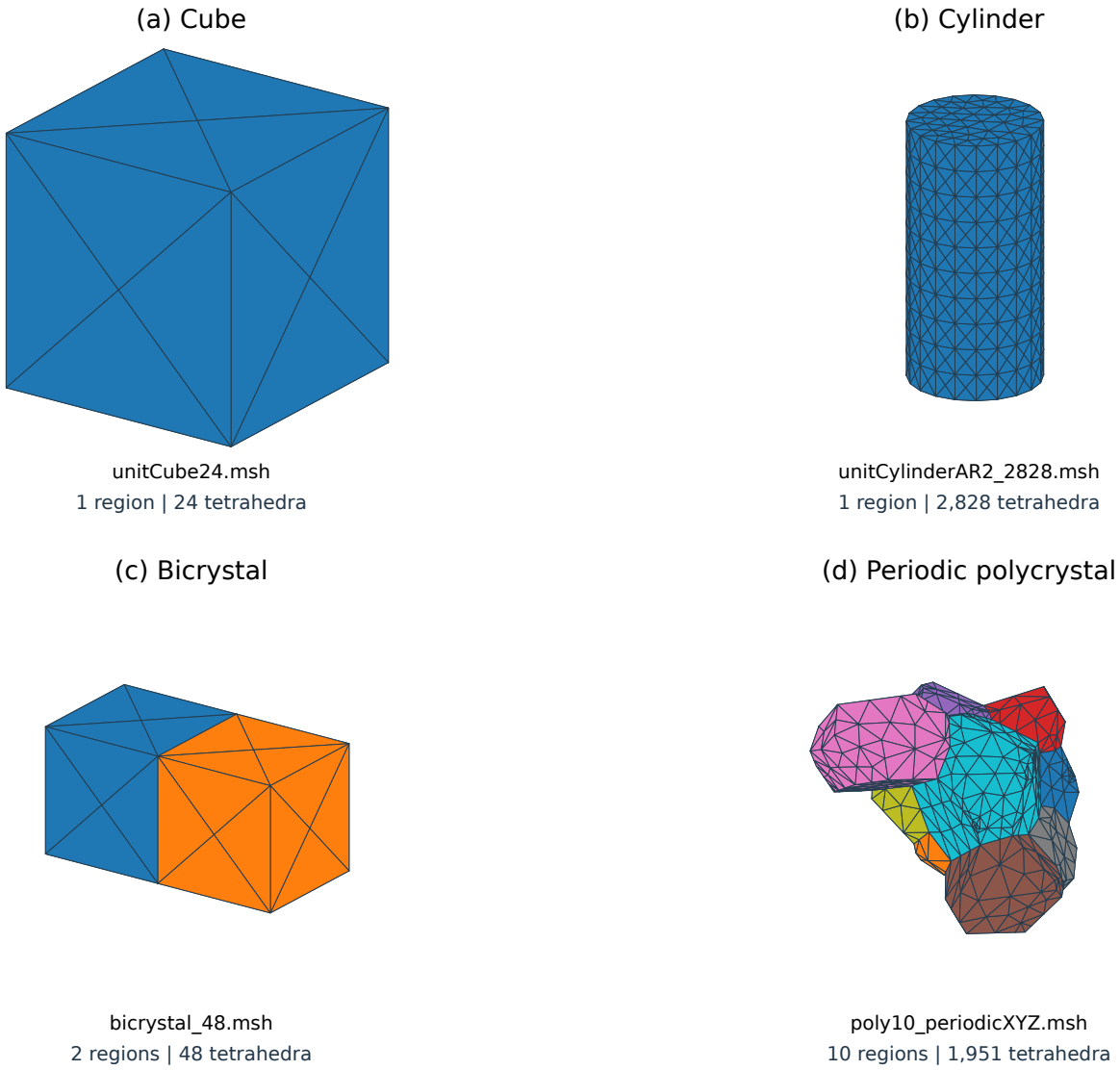


Figure 1.1: Existing volume meshes in `Library/Meshes`, displayed through their exterior triangulations. Colors identify region IDs. Each specimen is scaled independently for display; counts refer to volume tetrahedra. Quadratic edge nodes, when present in the source file, are omitted from the sketches.

1.1.3 Examples in the mesh library

Figure 1.1 shows four existing files from `Library/Meshes`. The cube and cylinder provide single-region examples; the bicrystal introduces a grain boundary. The ten-grain periodic unit has a jagged exterior following whole grains, and translated copies fill space. Its exterior is not a cut through grains at the boundary of a rectangular box. Further examples include `polycrystalCube_10_periodicX.msh`, `polycrystalCube_10_periodicXY.msh`, `poly30_periodicXYZ.msh` and `polycrystalCyl_50.msh`. Select a file with the geometry, grain structure and matching periodic boundaries required by the problem, then check its region IDs against the material input.

Implementation: `src/Mesh/SimplexReader.cpp`.

1.2 MoDELib mesh architecture

The `.msh` mesh files are the input to the MoDELib mesh representation. The mesh constructor reads coordinates and connectivity, applies the prescribed transformation, and inserts the volume elements.

1.2.1 Constructing a mesh from a file

Both examples load `mesh.msh` from the working directory with the identity transformation, zero origin shift and no periodic face pairing. The coordinates are mapped as $\boldsymbol{x} = \boldsymbol{F}(\boldsymbol{X} - \boldsymbol{x}_0)$. Replace the filename with your mesh path. C++ requires MoDELib's include and link settings; Python requires NumPy and an importable `pyMoDELib` module.

C++

```cpp
#include <MeshModule.h>

int main()
{
    Eigen::Matrix3d F = // deformation gradient
        Eigen::Matrix3d::Identity();
    Eigen::Vector3d x0 = // origin shift
        Eigen::Vector3d::Zero();

    model::SimplicialMesh<3> mesh(
        "mesh.msh", F, x0, {});
}
```

```python
import numpy as np
import pyMoDELib

F = np.eye(3)     # deformation gradient
x0 = np.zeros(3)  # origin shift

mesh = pyMoDELib.SimplicialMesh3(
    "mesh.msh", A=F, x0=x0,
    periodicFaceIDs=set()
)
```

Listing 1.1: Creating a mesh.

Implementation: `tools/pyMoDELib/MeshBindingsImplementation.h`.

1.2.2 Simplicial and polyhedral representations

MoDELib implements both a discretization of the material volume and a description of its boundary topology:

- `SimplicialMesh<dim>` stores volume simplices and their lower-dimensional children. In three dimensions, these are tetrahedra, triangles, edges and vertices. It provides global point search functions and supports the finite-element architecture described in Section 3.1.
- `PolyhedralMesh<dim>` stores a cell complex of the grains: regions, planar faces, perimeter edges and vertices. It supports oriented boundary traversal and plane sections.

  A polyhedral region is a grain-shaped volume bounded by faces. Similarly, a polyhedral face may contain many triangular mesh faces. Internal triangulation edges are absent from its perimeter, while subdivisions of the perimeter, including collinear mesh vertices, are retained.

The two representations therefore share geometry without assigning the same meaning to every edge or face.

Implementation: `include/Mesh/SimplicialMesh.h`.

Implementation: `include/Mesh/PolyhedralMesh.h`.

1.3 Simplices and shared topology

A `Simplex<dim,order>` represents a simplex of topological dimension `order`. Vertex IDs define its identity; adjacent higher-order simplices share lower-order entities. The mesh indexes entities through one `SimplexFactory<dim,order>` per order, accessed with `simplexFactory<order>()`. The factory canonicalizes the vertex-ID key and reuses an existing simplex through `getOrCreate`; the same volume simplex cannot be assigned to two regions.

The mesh's `simplices()` map owns the volume simplices. Each simplex owns its children through shared pointers, while child-to-parent links and factory entries are weak pointers. `liveSimplices()` returns a map of shared pointers to the currently live entities; `parents()` similarly returns a temporary map of live parents. Use these views for

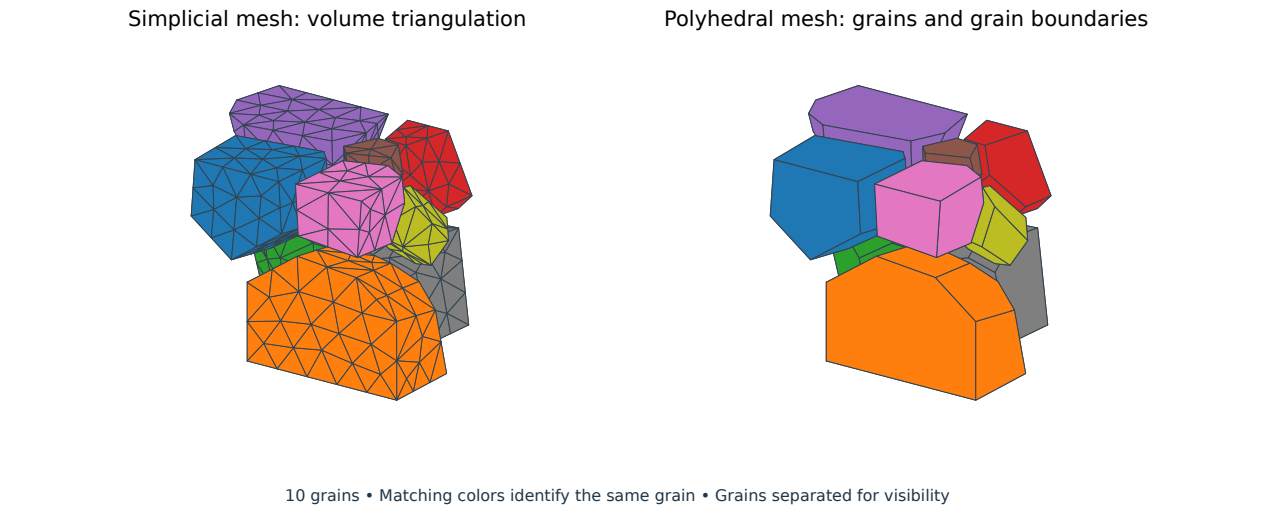


Figure 1.2: Two views of the ten-grain mesh `Library/Meshes/polycrystalCube_10_periodicX.msh`, containing 1081 tetrahedra. Left: the boundary triangulation of each grain, shown using tetrahedron corner vertices; quadratic edge nodes are omitted. Right: the same grain faces with internal coplanar triangle edges suppressed. Colors identify corresponding grains. Grains are separated only for visualization, exposing internal interfaces.

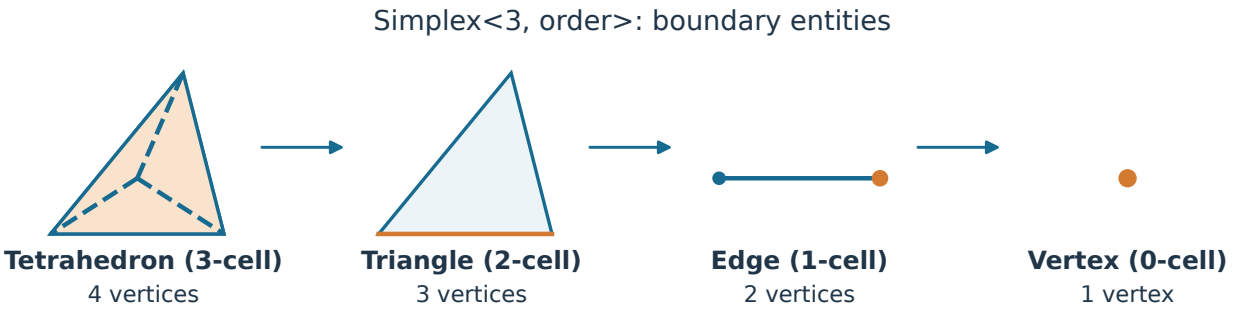


Figure 1.3: The simplex hierarchy in three dimensions. Arrows lead to lower-dimensional boundary entities; orange highlights a selected child in each sketch. Dashed lines show the tetrahedron's hidden edges.

traversal rather than treating raw factory entries as owning pointers. `pruneExpired()` removes expired factory entries. Keep the mesh alive while using its entities, since they also refer to mesh-owned geometry and factories.

Implementation: `include/Mesh/SimplexFactory.h`.

Implementation: `include/Mesh/SimplexChild.h`.

Implementation: `include/Mesh/SimplexParent.h`.

For a point $\boldsymbol{x}$, a tetrahedron's barycentric coordinates satisfy

$$\boldsymbol{x} = \sum_{a=0}^{3} \lambda_a \boldsymbol{X}_a, \qquad \sum_{a=0}^{3} \lambda_a = 1. \tag{1.1}$$

Nonnegative coordinates characterize containment, up to numerical tolerance. The C++ search routines return a success flag and a simplex pointer. Check the flag before using the pointer as a containing element. `searchWithGuess` permits reuse of a nearby element, while `searchRegion` and `searchRegionWithGuess` restrict the search to a grain. This distinction is important near an interface, where geometry alone does not select a unique crystal orientation. In Python, `mesh.search(point)` returns the containing simplex on success and `None` on failure; it does not return the C++ pair.

Implementation: `src/Mesh/SimplicialMesh.cpp`.

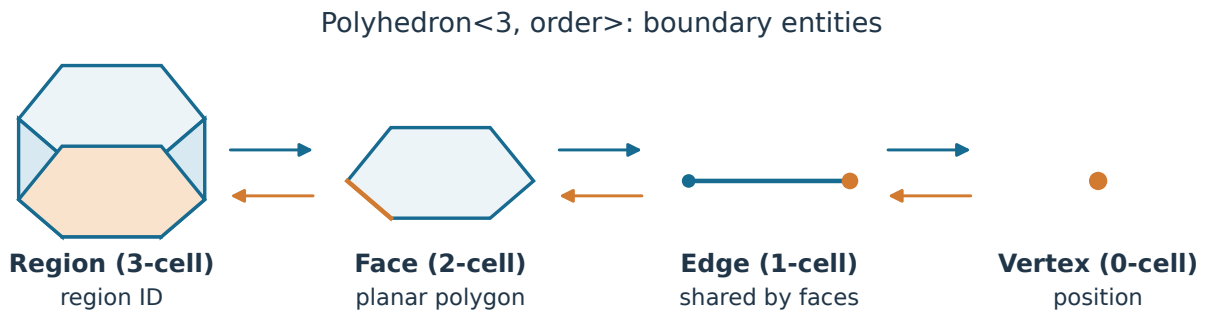


Figure 1.4: Geometric entities of the polyhedral complex in three dimensions. Orange highlights a selected child in each sketch. Blue arrows point to children (shared ownership); orange arrows point to parents (weak references). A face can belong to two regions. The polygonal prism is illustrative; grains may have more general shapes.

1.4 Polyhedral cells and oriented incidence

`Polyhedron<dim,order>` is the common cell type. In three dimensions, a region is order 3, a face order 2, an edge order 1 and a vertex order 0. One factory per order provides shared entities keyed by IDs. Regions and faces use scalar region and face IDs as keys; lower-dimensional boundary cells are keyed by their vertex IDs.

Region cells, `Polyhedron<dim,dim>`, group volume simplices by region ID. A boundary between two regions is represented by `MeshRegionBoundary`; its planar pieces are codimension-one `Polyhedron<dim,dim-1>` objects.

The mesh constructor then constructs region boundaries, updates regions and their planar faces in the shared polyhedral factories, completes their boundary topology, and finally identifies active periodic face pairs. This order matters: periodic matching operates on completed external faces, and downstream consumers must see a consistent topology.

Children are held by shared pointers and parents by weak pointers. This allows a face to belong to two regions, or an edge to several faces, without creating parent–child ownership cycles. Each cell also stores its associated simplices. Codimension-one cells carry face normals, centers, region IDs and periodic partners directly. This unified representation replaces the earlier separate `MeshRegion` and `PlanarMeshFace` classes in the current mesh refactor. In 3D, a face with one incident region is external; a face with two incident regions separates grains.

An edge itself has no single orientation valid for every face. `PolyhedralFaceIncidence` supplies the orientation of that edge within one face perimeter through its `reversed` flag and perimeter index. `source()`, `sink()` and `tangent()` use that orientation. `next()` and `prev()` traverse the same perimeter; in 3D, `otherFace(regionID)` crosses the edge to the adjacent face bounding the specified region. Specifying the region is essential at a grain junction, where an edge can have more than two parent faces.

The constructor derives boundaries by counting simplex incidences. A boundary sub-simplex occurs once; an internal one occurs twice. Validation rejects duplicate or non-manifold input, branched or disconnected edges, and open, repeated or multiple polygon perimeters. The current polygon representation requires one closed perimeter per 2-cell; do not assume support for a face with holes. Normal orientation and grain-shell checks have a specialized 3D implementation.

Implementation: `include/Mesh/PolyhedralMeshImplementation.h`.

Implementation: `src/Mesh/PolyhedralMesh.cpp`.

1.5 Mesh access from Python

The bindings expose `SimplicialMesh1`, `SimplicialMesh2` and `SimplicialMesh3`, with corresponding `SimplexReader1/2/3` and `PolyhedralMesh1/2/3` types. `SimplicialMesh` and `SimplexReader` remain aliases for the three-dimensional types. Entity names encode dimension and order: for example, `Simplex32` is a triangular simplex in 3D and `Polyhedron32` is a polyhedral face in 3D. The older `Polyhedron0/1/2/3` names alias the corresponding 3D entities.

C++

```cpp
auto triangles =
    mesh.simplexFactory<2>()
        .liveSimplices();
const auto& poly =
    mesh.polyhedralMesh();
const auto& faces = poly.factory<2>();
poly.validate();

auto tet = mesh.simplices()
    .begin()->second;
const auto x = tet->center().eval();
auto [found, hit] = mesh.search(x);
if (found) {
    auto bary = hit->pos2bary(x);
}
```

```python
triangles = mesh.simplices(2)
poly = mesh.polyhedralMesh()
faces = poly.cells(2)
poly.validate()

tet = next(iter(
    mesh.simplices().values()))
x = tet.center()
hit = mesh.search(x)
if hit is not None:
    bary = hit.pos2bary(x)
```

Listing 1.2: Inspecting mesh topology and locating a point.

The Python mesh constructor accepts `filename`, `A`, `x0` and `periodicFaceIDs`. The last three arguments default to the identity matrix, zero vector and empty set, so `SimplicialMesh3("mesh.msh")` is sufficient for an untransformed, nonperiodic mesh. A `SimplexReader3` can instead read coordinates and connectivity without constructing mesh topology: `reader.read(filename)`, `reader.nodes()` and `reader.elements()` expose node positions and element records, where an element record is a pair of vertex IDs and region ID.

Python access	Result or use
`mesh.simplices(order)`	Dictionary keyed by vertex-ID tuples; default order is the spatial dimension.
`mesh.polyhedralMesh()`	Derived polyhedral complex; use `cells(order)`, `region(id)`, `face(id)` and `validate()`.
`mesh.regions()`, `mesh.region(id)`	Region dictionary or one grain-shaped cell.
`simplex.children()`, `simplex.parents()`	Child list and live-parent dictionary, where the respective orders exist.
`cell.children`, `cell.parents`	Polyhedral child and live-parent dictionaries exposed as properties.
`cell.simplices()`, `cell.simplexIDs()`	Associated simplex objects or their vertex-ID tuples.
`simplex.vertices()`	Vertex objects with coordinates in `P0`; polyhedral vertices use `position`.

The dictionaries are snapshots of the current entity maps, not mutable topology containers. Both `simplices(order)` and `cells(order)` reject orders outside $0, \ldots, \mathrm{dim}$. A polyhedral region provides `faces()`; a face provides `regionIDs`, `center()`, `isExternal()` and `outNormal(regionID)`. Positive-order simplices provide `bary2pos()` and `vertexPositionMatrix()`; volume simplices also provide `pos2bary()` and their `region`.

The following examples continue from the nonempty 3D mesh constructed above. They retrieve triangular simplices and polyhedral faces, then locate the center of a known volume simplex.

Implementation: `tools/pyMoDELib/MeshBindingsImplementation.h`.

1.6 Plane sections and lifetime

`MeshPlane` combines a geometric plane with its intersections with the mesh. `BoundingMeshSegments` and `MeshBoundarySegment` carry the resulting boundary segments and their supporting faces. These are the geometric foundation for a glide plane clipped to a grain. A plane section must retain connectivity and face incidence; sorting all intersection points around a center is insufficient for general nonconvex domains.

`SimplicialMesh` owns its polyhedral complex and exposes it through `polyhedralMesh()`. Re-reading the mesh rebuilds this derived structure and clears periodic simplex pairs. References to old cells, faces, or finite-element data must not be reused after a rebuild. Python objects returned by mesh and connectivity queries retain the owning mesh; retaining a child therefore keeps its mesh alive without making reciprocal parent–child ownership cycles. This lifetime support does not make old entities valid after a C++ mesh rebuild. The Python interface currently exposes neither `readMesh` nor `identifyParallelFaces`; construct a new mesh to change its input or active pairing. In

normal simulations, `DefectiveCrystalBase` supplies shared ownership of the mesh to consumers and treats the geometry as fixed.

Implementation: `include/Mesh/MeshPlane.h`.

Implementation: `src/Mesh/MeshBoundarySegment.cpp`.

1.7 Preparing and checking a mesh

The simulation's `inputFiles/polycrystal.txt` selects `meshFile`. Paths are resolved relative to `inputFiles/`. The matrix `F` and vector `X0` transform the input coordinates as $\boldsymbol{x} = \boldsymbol{F}(\boldsymbol{X} - \boldsymbol{X}_0)$; `X0` is subtracted before applying `F`. This is a mesh placement/scaling operation, not the evolving elastic deformation. Coordinates used internally are in Burgers-vector units. Region IDs must agree with the grain rotations described in Chapter 2.

Before a run, check that the mesh is nonempty, the intended region IDs exist, and periodic partners have matching translated triangulations. The repository tests `testPolyhedralMesh.cpp`, `testPolyhedralMeshDimensions.cpp`, `testPlaneSectionMesh.cpp` and `testNonconvexMeshSearch.cpp` exercise different parts of these contracts. `testPythonMesh.py` additionally checks the bindings across dimensions 1–3, including connectivity, periodic pairing and ownership. The C++ tests identify verification entry points; this documentation update does not report a new execution of those tests.

Implementation: `include/Mesh/Polyhedron.h`.

Implementation: `src/Mesh/GmshReader.cpp`.

1.8 Geometric periodicity

1.8.1 The translation lattice

Periodicity identifies points separated by integer combinations of active translation vectors. Let $\boldsymbol{A} = [\boldsymbol{a}_1 \ \cdots \ \boldsymbol{a}_p]$ contain a basis of the periodic directions, with $0 \le p \le 3$. Equivalent points obey

$$\boldsymbol{x}' = \boldsymbol{x} + \boldsymbol{A}\boldsymbol{n}, \qquad \boldsymbol{n} \in \mathbb{Z}^p. \tag{1.2}$$

Partial periodicity means $p < 3$: unpaired external faces remain physical boundaries. The basis need not coincide with the Cartesian axes. `DefectiveCrystalBase` stores the basis and its dual

$$\boldsymbol{A}^* = \boldsymbol{A}(\boldsymbol{A}^T\boldsymbol{A})^{-1}, \qquad \boldsymbol{\xi} = (\boldsymbol{A}^*)^T(\boldsymbol{x} - \boldsymbol{x}_c), \tag{1.3}$$

where $\boldsymbol{x}_c$ is the mesh bounding-box center. There is no 2π factor in this coordinate dual. For partial periodicity, $\boldsymbol{\xi}$ records only the coordinates along the periodic span.

1.8.2 Selecting and matching faces

The mesh constructor accepts a set of `periodicFaceIDs`. An empty set disables pairing; `-1` enables all valid discovered periodic directions. Explicit values select external planar face IDs. These are MoDELib face IDs, not automatically Gmsh physical surface tags. Face IDs are allocated during topology construction and must not be assumed portable between separately constructed meshes. In C++, inspect the current mesh and pass its selected IDs to `identifyParallelFaces`.

`identifyParallelFaces` first discovers a valid full lattice and then selects the active lattice. A face pair must match under a translation, including its boundary simplex connectivity. Parallel normals alone are insufficient. Only pairs whose shifts are integer combinations of the selected lattice are activated. The mesh records both the planar-face relation and a `PeriodicSimplexPair` for each paired boundary simplex.

A simplex pair stores the partner pointer, the shift with convention

$$\boldsymbol{x}_{\text{partner}} = \boldsymbol{x}_{\text{source}} + \boldsymbol{s}, \tag{1.4}$$

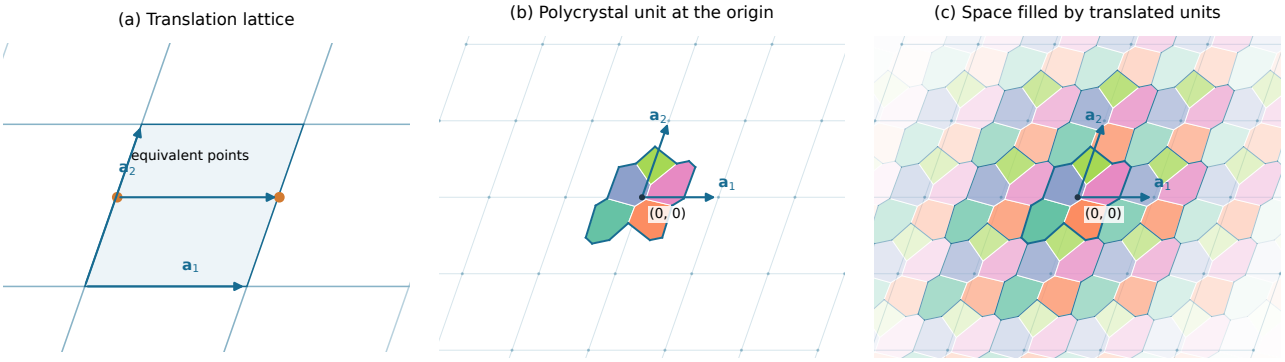


Figure 1.5: (a) An oblique translation lattice and equivalent points. (b) A two-dimensional polycrystal unit at the origin, composed of whole grains with a jagged outer boundary along grain boundaries. (c) Translations of this same jagged unit tile the plane without cutting grains at a lattice parallelogram. Opacity decreases with distance from the origin only for visual clarity; the polycrystal is schematic.

and the permutation matching source vertices to partner vertices. The reverse relation must have the opposite shift and inverse permutation. Pairing is committed only after these checks succeed. This makes all subsequent users consume the same geometric correspondence. If pairing fails, check translated boundary nodes, triangulation and selected face IDs. Field constraints are described in Section 3.3.

Implementation: `include/Mesh/PeriodicMeshFacePairing.h`.

Implementation: `include/Mesh/PeriodicSimplexPair.h`.

Implementation: `src/Mesh/SimplicialMesh.cpp`.

1.8.3 Inspecting periodicity in Python

`mesh.periodicBasis()` exposes the basis with one column per active translation. Eigen-to-NumPy conversion can flatten the one-dimensional case; use `np.asarray(mesh.periodicBasis()).reshape(mesh.dim,-1)` when a uniform `(dim,p)` array is needed. `mesh.periodicSimplexPairs()` returns a dictionary keyed by the source boundary simplex's vertex-ID tuple. Each value is `(partner,shift,permutation)`: source vertex i translated by `shift` matches partner vertex `permutation[i]`. On a polyhedral face, `periodicPartner` returns its paired face or `None`, and `periodicShift` gives the translation. These queries expose geometric identification; they do not impose field constraints. Select pairing with the constructor argument `periodicFaceIDs`, for example `periodicFaceIDs={-1}` for all valid directions.

Implementation: `tools/pyMoDELib/MeshBindingsImplementation.h`.

Chapter 2

Materials and polycrystals

2.1 Shared material, grain-specific crystallography

`PolycrystallineMaterialBase` reads the material identity, crystal structure, temperature-dependent scales, elastic constants, enabled slip systems and second phases. `Polycrystal<3>` combines this shared material with the mesh's collection of grains. Each mesh region produces a `Grain`; each grain derives from `SingleCrystalBase`, which provides its oriented lattice, plane normals, slip systems and second phases.

In this design, changing a grain's orientation changes its crystallography while the material scale is shared. A collection of differently oriented grains is therefore not, by itself, a collection of different material files. The current base reads one material file for the polycrystal and checks that grains have a common atomic volume.

Implementation: `src/PolycrystallineMaterials/PolycrystallineMaterialBase.cpp`.

Implementation: `src/PolycrystallineMaterials/Polycrystal.cpp`.

2.2 Units and temperature

The fundamental scales are the Burgers-vector magnitude b_{SI}, the shear modulus μ_{SI}, and the shear-wave speed $c_{s,\mathrm{SI}}$. The current material constructor uses

$$\mu_{\mathrm{SI}}(T) = \mu_{0,\mathrm{SI}} + \mu_{1,\mathrm{SI}} T, \quad c_{s,\mathrm{SI}} = \sqrt{\mu_{\mathrm{SI}}/\rho_{\mathrm{SI}}}, \quad E_{\mathrm{SI}} = 2\mu_{\mathrm{SI}}(1+\nu). \tag{2.1}$$

Internal values of b, μ and c_s are one. To interpret outputs, use

Quantity	Internal value	SI conversion
Position	$\hat{x}$	$b_{\mathrm{SI}}\hat{x}$
Time	$\hat{t}$	$(b_{\mathrm{SI}}/c_{s,\mathrm{SI}})\hat{t}$
Velocity	$\hat{v}$	$c_{s,\mathrm{SI}}\hat{v}$
Stress	$\hat{\sigma}$	$\mu_{\mathrm{SI}}\hat{\sigma}$
Energy	$\hat{E}$	$\mu_{\mathrm{SI}} b_{\mathrm{SI}}^3 \hat{E}$

A physical length used to scale a unit mesh must be divided by b_{SI}. Input names ending in `_SI`, or naming units such as `_eV`, identify dimensional values; other controls must be checked in their template. Temperature is in kelvin.

The presence of crystallographic anisotropy in slip systems does not imply that every elastic solver uses an anisotropic stiffness. The current elastic-deformation module constructs isotropic stiffness from μ and ν. The anisotropic dislocation derivations retained later in the manual are general theory and should not be read as a statement that every runtime kernel implements that generality.

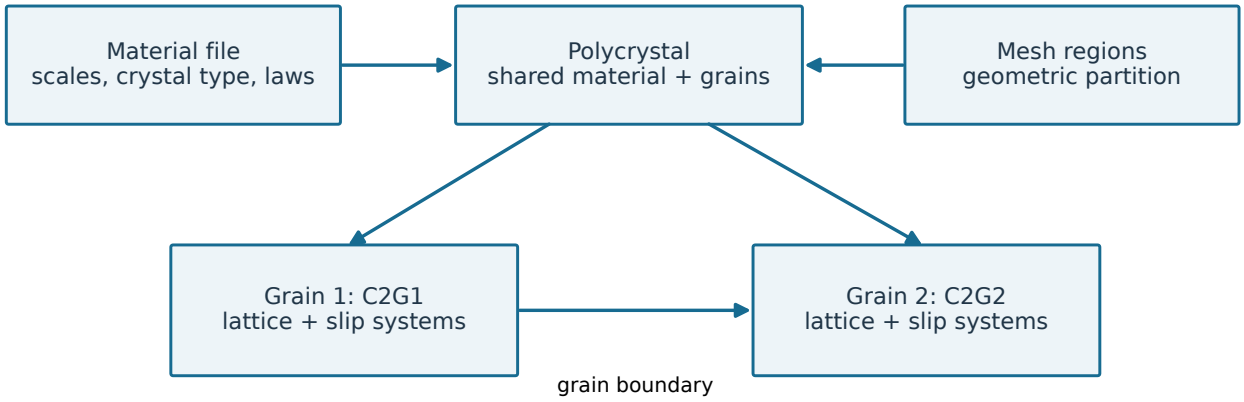


Figure 2.1: Construction of the polycrystal. Geometry provides the regions, the shared material provides constitutive data, and a rotation supplies each grain's crystallographic orientation.

2.3 Grains, rotations and boundaries

For each region ID r, `Grain` reads the crystal to global (`C2G`) transformation matrix from the entry named `C2G` followed by that ID, for example `C2G1`. This matrix maps crystal coordinates to global coordinates. For a proper rotation $\boldsymbol{Q}$, a crystal vector transforms as $\boldsymbol{v}_g = \boldsymbol{Q}\boldsymbol{v}_c$, and a second-order tensor as $\boldsymbol{T}_g = \boldsymbol{Q}\boldsymbol{T}_c\boldsymbol{Q}^T$. Use orthogonal matrices with determinant +1; a mesh scaling belongs in `F`, not in the crystal rotation.

`SingleCrystalSelector` chooses the lattice and available crystallographic objects from the material's crystal structure. `enabledSlipSystems` selects which systems are constructed. A slip system combines a slip direction, plane and mobility behavior; material mobility data must be compatible with the selected systems. Grain boundaries associate neighboring grains with their geometric region boundary and planar face. Their existence as objects does not automatically supply a constitutive rule for transmission of dislocations.

Implementation: `src/PolycrystallineMaterials/Grain.cpp`.

Implementation: `include/PolycrystallineMaterials/SingleCrystalBase.h`.

Implementation: `include/PolycrystallineMaterials/GrainBoundary.h`.

2.4 Input workflow

Copy a material file from `Library/Materials/` to the simulation's `inputFiles/`. Select it in `polycrystal.txt` alongside the mesh and temperature. Supply a rotation for every mesh region, using the actual region IDs rather than assuming consecutive numbering.

```
# Illustrative polycrystal fragment: filenames must exist locally.
materialFile=W.txt;
meshFile=unitCube24.msh;
absoluteTemperature=300;
F=1000 0 0
  0 1000 0
  0 0 1000;
X0=0 0 0;
C2G1=1 0 0
     0 1 0
     0 0 1;
periodicFaceIDs=-1;
```

This example scales a unit cube to side length $1000b$. It is a geometry/material fragment, not a complete simulation. For a multigrain mesh, add the corresponding `C2G` entries. The tutorial helper `PolyCrystalFile` is useful for generating single-grain input; inspect its output before extending it to several grains.

A source-level material investigation should follow the reading class: basic elastic scales in `PolycrystallineMaterialBase`, slip-system construction in the crystal-specific classes, mobility in `DislocationMobilities`, and diffusion/reaction parameters in the CD context. This avoids treating one material file as a single constitutive law.

Chapter 3

Finite-element infrastructure

3.1 Geometry, interpolation and fields

The volume mesh is geometry. `FiniteElement<ElementType>` adds interpolation nodes and elements. `TrialFunction` carries a field's unknowns and conditions; differential operators and integration domains build weak forms; `WeakProblem` assembles and solves the constrained system. Keeping these roles separate allows multiple physics to reuse the same geometric discretization while defining different fields and equations.

The current `DefectiveCrystalBase::ElementType` is `LagrangeElement<3,2>`: quadratic tetrahedra. A tetrahedron therefore has ten interpolation nodes, including its edge midpoints, even if the source Gmsh mesh supplies only the four vertices of each volume element. A vector displacement has three components per interpolation node; concentrations have their own component counts.

Implementation: `include/FEM/FiniteElement.h`.

Implementation: `include/FEM/LagrangeElement.h`.

Implementation: `include/FEM/TrialFunction.h`.

3.2 The weak-form contract

For illustration, small-strain elasticity uses $\boldsymbol{\varepsilon}(\boldsymbol{u}) = \operatorname{sym} \nabla \boldsymbol{u}$. Its bilinear form is

$$a(\boldsymbol{v}, \boldsymbol{u}) = \int_\Omega \boldsymbol{\varepsilon}(\boldsymbol{v}) : \mathbb{C} : \boldsymbol{\varepsilon}(\boldsymbol{u}) \, \mathrm{d}V. \tag{3.1}$$

The elastic module builds the corresponding expression with `def(u)`, a stiffness multiplication, a test function and a volume domain. This expression describes a mathematical operator; boundary conditions, affine modes and rigid-motion control are supplied separately. The right-hand side depends on the physics, external loading and coupling fields.

Solve a clamped elasticity problem. The following C++ function accepts a shared, nonperiodic mesh of a connected block with a planar $x = x_{\min}$ face. It clamps all displacement components on that face, applies a constant body force, and leaves the other faces traction-free. The example uses $\mu = 1$ and $\nu = 0.3$ in consistent units. The returned vector contains the solved nodal displacements. The weak-form API used here is not currently exposed by `pyMoDELib`.

For a discrete linear system $\boldsymbol{K}\boldsymbol{u} = \boldsymbol{f}$, constraint elimination can be understood through $\boldsymbol{u} = \boldsymbol{P}\boldsymbol{q} + \boldsymbol{c}$:

$$\boldsymbol{P}^T \boldsymbol{K} \boldsymbol{P} \boldsymbol{q} = \boldsymbol{P}^T (\boldsymbol{f} - \boldsymbol{K}\boldsymbol{c}). \tag{3.2}$$

This is a conceptual reduction, not an assertion that every solve uses only this equation: `WeakProblem` also supports global linear constraints and additional affine modes. Its checks detect incompatible prescribed values and invalid constraint systems, and verify residuals after solving.

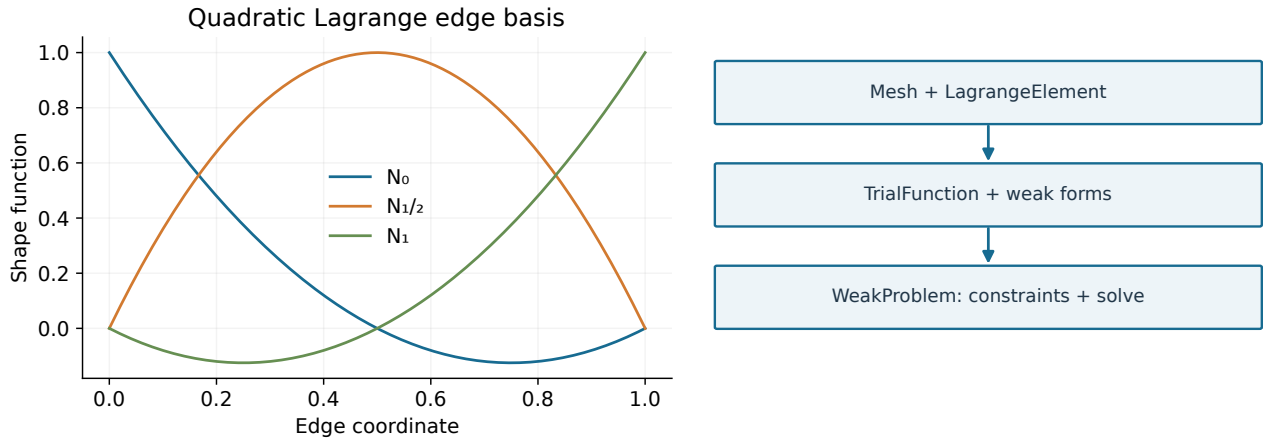


Figure 3.1: Left: the three analytical quadratic Lagrange functions along an element edge. Their midpoint degree of freedom explains why vertex-only periodic matching is insufficient. Right: the responsibility chain from geometry to a constrained solve.

Implementation: `include/FEM/WeakProblem.h`.

Implementation: `src/ElasticDeformation/ElasticDeformationFEM.cpp`.

3.3 Periodic equivalence classes and field constraints

`PeriodicFEMNodePairing` extends boundary-simplex correspondence to finite-element nodes using their barycentric integer indices. This includes higher-order edge and face nodes, not only mesh vertices. A graph traversal constructs a representative node and translation for every equivalence class. Cycles at edges and corners must give consistent total shifts.

The pairing object records geometry; the weak-problem solver uses it to constrain field degrees of freedom. For an affine-periodic vector field,

$$\boldsymbol{u}_i = \boldsymbol{u}_{r(i)} + \boldsymbol{H}\boldsymbol{s}_i + \boldsymbol{q}_i - \boldsymbol{q}_{r(i)}, \tag{3.3}$$

where $r(i)$ is the representative, $\boldsymbol{s}_i$ the geometric shift, $\boldsymbol{H}$ the prescribed field gradient, and $\boldsymbol{q}$ an optional nodal offset. Zero gradient and zero offset give a strictly periodic field. Global affine modes may also be solved as unknowns by the loading controller. Incompatible nodal Dirichlet values and affine/periodic conditions are rejected rather than silently overwritten.

Continuous finite-element spaces acquire periodic node relations. Discontinuous spaces retain independent nodes. If active mesh pairing is changed after finite-element construction, explicitly rebuild the periodic node pairing before constructing or solving dependent fields.

Implementation: `include/FEM/PeriodicFEMNodePairing.h`.

Implementation: `include/FEM/FiniteElement.h`.

Implementation: `include/FEM/WeakProblem.h`.

If a solve rejects constraints, inspect prescribed values on every copy of an equivalence class. Relevant checks include `testPeriodicFEMNodes.cpp`.

3.4 Physical boundaries and rigid modes

Equation (3.3) expresses the relationship between equivalent field nodes. The finite-element pairing supplies representatives and shifts, while the field supplies its gradient and optional offset. This distinction allows the same periodic mesh to host different fields.

```cpp
#include <cfloat>
#include <FiniteElement.h>
#include <WeakProblem.h>
#include <cmath>
using namespace model;

Eigen::VectorXd solveElasticity(std::shared_ptr<SimplicialMesh<3>> mesh)
{
    using FE = FiniteElement<LagrangeElement<3,2>>;
    auto fe = std::make_shared<FE>(mesh);
    auto u = fe->trial<3>();                 // displacement
    const double mu = 1.0, nu = 0.3;
    const double lambda = 2*mu*nu/(1-2*nu);
    Eigen::Matrix<double,6,6> C = Eigen::Matrix<double,6,6>::Zero();
    C.topLeftCorner<3,3>().setConstant(lambda);
    C.diagonal().head<3>().array() += 2*mu;
    C.diagonal().tail<3>().setConstant(mu); // engineering shear strain

    for (size_t n=0; n<fe->nodeSize(); ++n)
        if (std::abs(fe->node(n).P0(0)-mesh->xMin()(0)) < 1.e-10)
            for (int c=0; c<3; ++c)
                u->dirichletConditions()[3*n+c] = 0.0; // clamp x-min face

    auto dV = fe->domain<EntireDomain,4,GaussLegendre>();
    auto e = def(u);
    auto stiffness = (test(e), C*e)*dV;
    const Eigen::Vector3d force(0.0, 0.0, -1.e-3); // body force per volume
    auto load = (test(u), make_constant(force))*dV;
    auto problem = weakProblem(stiffness, load);
    problem->solveWithMasterSlaveConstraints(); // assemble, constrain, solve
    return u->dofVector();                      // three entries per FE node
}
```

Listing 3.1: Solving linear elasticity with a clamped face and a body force.

A paired external face is not a free surface. `NonPeriodicExternalBoundary` excludes paired boundary simplices from the physical-boundary integration domain. For elastic loading, periodic directions can carry free affine jumps determined by the controller, while physical faces receive the associated traction. Applying a free-surface load to both members of a periodic pair would count a boundary that has already been identified internally.

Elasticity also has rigid-motion null modes. The elastic module uses `GlobalRigidMotionPenalty` to control admissible mean displacement and mean infinitesimal rotation globally. Its weight is `inertiaReliefPenaltyFactor`. This is a global reference condition; it is not an arbitrary clamp on a chosen mesh vertex.

Implementation: `include/FEM/NonPeriodicExternalBoundary.h`.

Implementation: `src/ElasticDeformation/ElasticDeformation.cpp`.

3.5 Enabling and diagnosing FEM

`useFEM` in `DefectiveCrystal.txt` creates the shared finite-element infrastructure. `useElasticDeformationFEM` in `ElasticDeformation.txt` selects the elastic module's FEM formulation. They have different roles: enabling the shared FEM does not itself choose elastic-deformation physics. Conversely, a physics needing the FEM cannot work with that infrastructure disabled.

For an elastic FEM run, select `ElasticDeformation` in `physics`, set `useFEM=1`, and set `useElasticDeformationFEM=1`. Keep the material, boundary pairing, and loading choices consistent. On a failed solve, inspect constraints and null modes before changing numerical tolerances. On a periodic mismatch, verify higher-order nodes as well as mesh vertices. For accuracy, compare an observable under mesh refinement; the polynomial order alone is not an accuracy certificate.

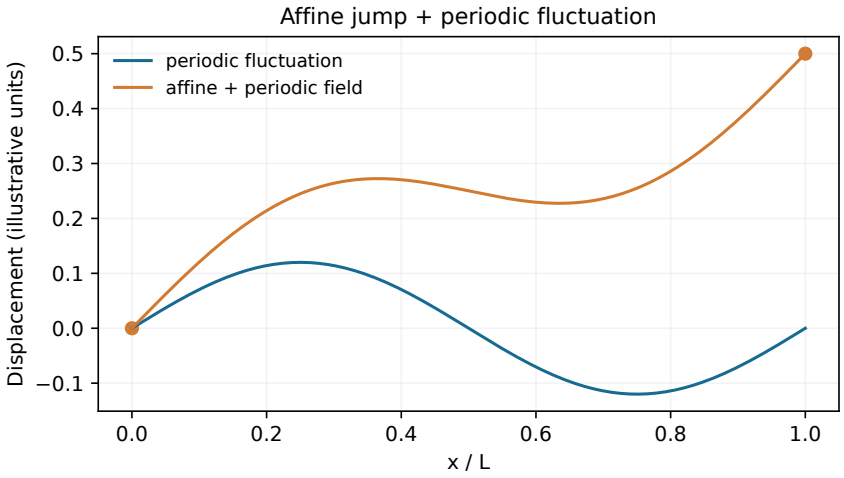


Figure 3.2: An affine-periodic field $u(x) = 0.5x + 0.12\sin(2\pi x)$ combines an affine jump with a periodic fluctuation. These analytical curves illustrate field constraints; they are not solver output.

Part II

Multiphysics and shared field physics

Chapter 4

The multiphysics architecture

4.1 Shared context and evolving state

`DefectiveCrystalBase` constructs the simulation context: parameters and paths, shared mesh ownership, periodic translations, shared polycrystal ownership, optional finite elements, and the base contexts of enabled physics. It also serves microstructure generation. `DefectiveCrystal` creates the selected evolving microstructures and advances the simulation.

`MicrostructureBase` is the interface each physics implements. `MicrostructureContainer` is both a collection of these objects and an implementation of the same interface. Thus the container can forward lifecycle operations and expose aggregate fields without requiring callers to know the concrete type of every physics.

Implementation: `include/DefectiveCrystal/DefectiveCrystalBase.h`.

Implementation: `include/DefectiveCrystal/MicrostructureBase.h`.

Implementation: `src/DefectiveCrystal/MicrostructureContainer.cpp`.

4.2 Lifecycle and common field queries

The interface includes initialization from a configuration, `solve()`, `reSolve()`, `getDt()`, `output()` and `updateConfiguration()`. It also exposes stress, displacement, inelastic displacement rate, mobile concentrations, immobile clusters, and volume-average quantities. A physics contributes the quantities it represents and returns the appropriate zero contribution for others.

The container sums returned fields. For example,

$$\boldsymbol{\sigma}(\boldsymbol{x}) = \sum_{\alpha} \boldsymbol{\sigma}^{(\alpha)}(\boldsymbol{x}), \qquad \langle \boldsymbol{\beta}^{P} \rangle = \sum_{\alpha} \langle \boldsymbol{\beta}^{P,(\alpha)} \rangle. \tag{4.1}$$

This additive interface makes coupling accessible, but the module must still define which part of the field it owns. In particular, an elastic correction must not double-count a dislocation's analytical stress. The optional node, element and simplex arguments of field queries provide geometric context and can avoid redundant searches.

4.3 One simulation step

The actual ordering in `DefectiveCrystal::runSingleStep` is:

1. Call `solve()` on every microstructure in configured order.
2. Perform `maxResolveSteps` additional `reSolve()` passes.
3. Choose $\Delta t = \min(\Delta t_{\max}, \Delta t_1, \ldots, \Delta t_n)$.

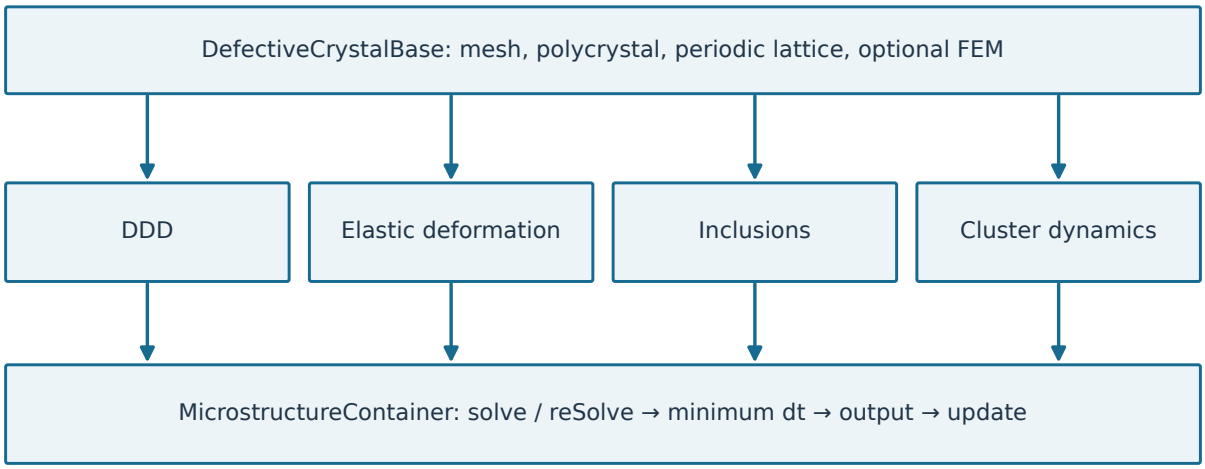


Figure 4.1: Shared context and physics lifecycle. Boxes identify responsibilities rather than inheritance alone. Physics execute in configured order; the diagram does not imply a simultaneous coupled solve.

4. If this step is an output step, write the current configuration, auxiliary data and scalar history.
5. Call `updateConfiguration()` on every microstructure.
6. Advance simulation time and increment `runID`.

Output therefore records the configuration before that step's update. `Nsteps` is the terminal step index in the run loop; after a restart it is not automatically a count of additional steps. Extra re-solve passes are a prescribed iteration count, not a general convergence test for the coupled problem.

Implementation: `src/DefectiveCrystal/DefectiveCrystal.cpp`.

4.4 Selecting and extending physics

The `physics` line in `DefectiveCrystal.txt` accepts space-separated names:

```
physics=DislocationDynamics ElasticDeformation;
```

Available names are `DislocationDynamics`, `ElasticDeformation`, `InclusionMicrostructure` and `ClusterDynamics`. Parsing is case-insensitive and removes duplicate names while preserving order. `none` is a template placeholder; a run with no actual physics is rejected. Because lifecycle calls are sequential, the list order is part of the numerical setup.

A new physics needs an implementation of `MicrostructureBase`, configuration and output support, and registration in the physics parsing and construction paths. Its field contribution, time-step restriction and re-solve behavior must be defined explicitly. Mesh and material data should come from the shared context rather than from a second independently constructed crystal.

Chapter 5

Running a simulation

5.1 Build the tools

The repository uses C++20 and CMake 3.20 or later. Eigen and FFTW are required by the current build. OpenMP is used by the simulation tools. Qt/VTK are needed for the optional viewer, and pybind11 for optional Python bindings. The root `CMakeLists.txt` is the authoritative list of build options for this snapshot.

From the repository root, a command-line configuration is:

```
cmake -S . -B build -DUSE_PYBIND11=OFF -DBUILD_DDQT=OFF
cmake --build build -j
```

This builds the simulation tools without requiring the viewer or Python bindings. Tutorial input generators still need Python and NumPy. Plotting scripts generally need Matplotlib. CD species counts are compile-time options; match them to the chosen CD material when that physics is enabled.

5.2 The simulation folder

A simulation has an `inputFiles/` directory, a `config/` directory, and an `F/` directory for history output. The generator writes the initial microstructure; the driver evolves it. Copy templates into the simulation before editing them.

File	Responsibility
`polycrystal.txt`	Mesh, transformations, temperature, material, grain orientations and periodic faces.
`DefectiveCrystal.txt`	Physics selection, shared FEM switch, time stepping, image counts and output schedule.
Material file	Units, elasticity, crystal structure and constitutive parameters.
`DD.txt`	DDD solver, remeshing and dislocation-specific controls.
`ElasticDeformation.txt`	Uniform/FEM elasticity, loading and rigid-motion control.
`initialMicrostructure.txt`	List of microstructure specifications consumed by the generator.
`ClusterDynamics.txt`	Current CD-specific controls, when enabled.

Inputs generally use `name=value;` and `#` comments. Arrays are whitespace-separated; matrices have rows and one terminating semicolon. Use the complete template for each enabled physics rather than assembling a deck from isolated examples in this manual.

5.3 A first DDD run

The `dipoles` tutorial demonstrates the complete path. Run in a fresh copy if that tutorial folder contains results you wish to keep. From the repository root:

```
cd tutorials/dipoles
python3 generateInputFiles.py
../../build/tools/MicrostructureGenerator/microstructureGenerator .
../../build/tools/DDomp/DDomp .
```

The supplied generator selects DDD and elastic deformation, copies the material and mesh, generates `polycrystal.txt`, and writes the microstructure specifications. Its default is a long run. For an initial check, edit the generated `DefectiveCrystal.txt` to set `Nsteps=2;` and `outputFrequency=1;` before invoking `DDomp`. The commands here document the workflow; no simulation was run as part of this manual revision.

5.4 Outputs and restart

`config/evl_*.txt` (or the binary equivalent) contains configurations. Auxiliary output carries additional fields selected by the physics. `F/F_0.txt` stores scalar history, and `F/F_labels.txt` identifies its columns. Read labels instead of assuming a fixed column layout across physics combinations.

`outputFrequency` controls step-based output and must be positive; `outputBinary` selects configuration encoding. `startAtTimeStep=-1` requests the latest available history step. For a fresh run, ensure that old configuration and history files are not accidentally used as a restart. For continuation, retain the configuration and matching history and set `Nsteps` beyond the restored step index.

Check the selected physics, unit scales, periodic basis and initial configuration before interpreting results. Compare time-step, spatial and image-sum resolution using the physical observable of interest. A run that completes successfully can still be under-resolved.

Implementation: `tutorials/dipoles/generateInputFiles.py`.

Implementation: `src/DefectiveCrystal/DefectiveCrystalParameters.cpp`.

Implementation: `src/DefectiveCrystal/DefectiveCrystalTraitsIO.cpp`.

5.5 Periodic mesh input

Set `periodicFaceIDs` in `polycrystal.txt`: an empty set disables pairing; `periodicFaceIDs=-1;` enables all valid discovered directions. Explicit IDs select external planar MoDELib faces, not Gmsh physical surface tags. Inspect the mesh face numbering and the discovered translation basis before selecting a subset. Geometric matching is described in Section 1.8; FEM constraints and DDD image controls are described in Sections 3.3 and 8.3, respectively.

Chapter 6

Elastic-deformation physics

6.1 Uniform and finite-element formulations

`ElasticDeformation` supplies the shared elastic response through the microstructure interface. With `useElasticDeformationFEM=0`, a uniform controller relates average inelastic strain, applied stress and applied strain. With the FEM formulation enabled, spatial fields are solved with the shared finite elements and a controller for volume-average loading.

The uniform formulation is appropriate when a homogeneous external response is intended. A finite-domain correction requires a spatial solution and consistent physical boundary data. DDD retains its analytical defect fields; elastic deformation supplies its own contribution through the aggregate field interface. The superposition derivation in the DDD theory chapter explains why this division is useful near singular line fields.

Implementation: `src/ElasticDeformation/ElasticDeformation.cpp`.

Implementation: `src/ElasticDeformation/ElasticDeformationFEM.cpp`.

6.2 Loading controls

`ExternalStress0`, `ExternalStressRate`, `ExternalStrain0` and `ExternalStrainRate` define ramps using absolute simulation time. `stiffnessRatio` provides six machine-stiffness ratios. The Voigt ordering is $11, 22, 33, 12, 23, 13$; strain inputs use engineering shear components. Stress is in internal modulus units and strain rate in inverse internal time units.

A zero stiffness ratio gives stress control for the corresponding component; a large value approaches strain control. The FEM controller treats values at least 10^{15} as exact average-strain constraints. The uniform algebra and its limiting cases are retained with the original DDD loading notes in Section E.3, because they were part of that derivation collection. Their current software owner is this physics.

6.3 FEM field state

`ElasticDeformationFEM` creates trial fields `u` and `z`, differential expressions and the elastic bilinear form. `z` participates in the inelastic-displacement bookkeeping; its rate is obtained through the aggregate inelastic-displacement-rate query. The FEM controller adds global loading modes and constraints, while the global rigid-motion penalty removes admissible drift.

For a FEM run, `useFEM=1` must accompany `useElasticDeformationFEM=1`. Use the full `Library/ElasticDeformation/ElasticDeformation.txt` template. When switching from uniform to FEM loading, retain the intended six-component load path and recheck periodic directions, physical surfaces and rigid modes. A compatible mesh does not by itself determine the desired macroscopic loading.

Implementation: `Library/ElasticDeformation/ElasticDeformation.txt`.

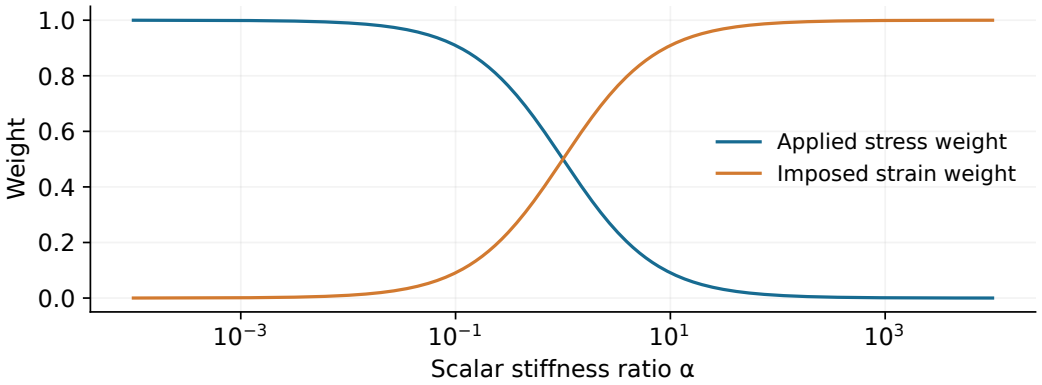


Figure 6.1: Analytical scalar weights $1/(1+\alpha)$ and $\alpha/(1+\alpha)$ illustrate the transition from stress to strain control. The full tensor problem also couples components through stiffness; these curves are not a simulation response.

Chapter 7

Inclusion physics

7.1 Objects and field contributions

`InclusionMicrostructure` is a microstructure whose state is a collection of Eshelby inclusion objects. The current implementation restores spherical, ellipsoidal and polyhedral inclusions from configuration records. The current inclusion state is static: `solve()` and `reSolve()` do not evolve it, and its time-step proposal is the shared maximum. A common `EshelbyInclusionBase` interface provides the elastic-field behavior needed by the surrounding simulation.

An inclusion carries shape data, a transformation eigenstrain, a phase identity and mobility-reduction information. Its stress is a contribution to the aggregate stress seen by other physics. The geometry used by a polyhedral inclusion belongs to that inclusion; it should not be confused with the `PolyhedralMesh` representation of grain boundaries.

Implementation: `src/InclusionMicrostructure/InclusionMicrostructure.cpp`.

Implementation: `include/InclusionMicrostructure/EshelbyInclusionBase.h`.

7.2 Initialization and coupling

Inclusion IDs share one namespace across shapes. Initialization checks for collisions, clears existing inclusion state and restores saved IDs. Spherical and ellipsoidal centers are located in the mesh to obtain a grain and its second-phase definition. An invalid phase ID is an input error. Polyhedral records supply node and face connectivity as well as the inclusion's constitutive data.

The eigenstrain is the stress-free transformation associated with the inclusion. It generates an elastic field when constrained by the surrounding solid. This is the same eigendistortion concept developed in the DDD theory chapter, but the source occupies a volume rather than a dislocation's slip surface. Phase data and shape-specific kernels determine the actual field evaluation.

7.3 User workflow

Select `InclusionMicrostructure` in `physics`. Generate inclusion specifications using the templates under `Library/Microstructures/`, ensure that the material enables the intended second phase, and inspect the generated configuration before coupling it to DDD. Use distinct IDs for all inclusion shapes and check that their locations belong to the intended grain. Inclusion generation and inclusion field evaluation are separate stages; the generator does not select the evolving physics for the driver.

For verification examples, inspect `tests/ellipsoidalInclusion/` and `tutorials/inclusions/`. Comparisons should include a shape limit or symmetry check and the behavior of fields outside the inclusion, not only successful parsing of a configuration.

Part III

Discrete dislocation dynamics

Chapter 8

Discrete dislocation dynamics: architecture

8.1 State and responsibilities

`DislocationNetwork` implements the DDD microstructure. The generic loop-network layer represents oriented loops and their shared network nodes and links. A dislocation loop carries crystallographic and slip-surface information; a network segment represents a geometric line portion that may participate in several loops. Keeping these identities separate makes junctions and topology changes representable without losing the individual loop bookkeeping.

The DDD base context supplies shared dislocation-specific services and parameters. Geometry and grains remain owned by the common crystal context. Glide planes connect lattice planes to grain geometry, and periodic glide planes extend that representation to translated patches. The nodal geometry, quadrature data, velocities and event operations then form the evolving network state.

Implementation: `include/DislocationDynamics/DislocationNetwork.h`.

Implementation: `include/DislocationDynamics/DislocationDynamicsBase.h`.

Implementation: `include/LoopNetwork/LoopNetwork.h`.

8.2 Forces, velocities and updates

At a conceptual level the method evaluates stress, computes line forces, applies mobility laws, determines nodal velocities and advances the network. The Peach–Koehler force per unit length is

$$\boldsymbol{f}_{\mathrm{PK}} = (\boldsymbol{\sigma}\boldsymbol{b}) \times \boldsymbol{\xi}, \tag{8.1}$$

where $\boldsymbol{b}$ is the Burgers vector and $\boldsymbol{\xi}$ the unit tangent. Local mobility converts force and character information into a velocity response. The selected glide or climb solver converts the local response into a consistent set of nodal velocities.

`solve()` prepares and solves the active motion problem. `getDt()` supplies a DDD restriction through the time stepper. `updateConfiguration()` moves nodes and performs the network's discrete-event processing. Remeshing controls segment resolution; junctions and cross slip change topology or slip-plane assignment. These operations have different physical meanings and should not be conflated with the continuous motion step. Periodic boundary-node reconstruction must preserve the loop contour across translated patches.

Climb couples to cluster-dynamics concentrations and has its own solver and re-solve behavior. The common `maxResolveSteps` mechanism allows additional coupling passes, but its iteration count is not proof of coupled convergence. Because CD is being revised, its detailed constitutive coupling is reserved for the final part of this manual.

Implementation: `src/DislocationDynamics/DislocationNetwork.cpp`.

8.3 Periodic dislocation continuity and image fields

DDD uses the same translation lattice for a different task: representing a loop as connected pieces on translated glide-plane patches. `PeriodicGlidePlane` manages those patches. Each `PeriodicPlaneEdge` has perimeter neighbors, a possible twin on an adjacent patch, and a translation change. A physical free surface is not a periodic patch crossing. Remeshing and node motion must preserve this distinction and the continuity of the underlying loop.

Geometric identification and elastic-image summation are separate controls. `periodicImageSize` in `DefectiveCrystal.txt` supplies one image extent m_k per active basis vector. The shared list contains shifts

$$\boldsymbol{s} = \sum_{k=1}^{p} n_k \boldsymbol{a}_k, \quad -|m_k| \le n_k \le |m_k|, \qquad N_{\text{shifts}} = \prod_{k=1}^{p} (2|m_k| + 1). \tag{8.2}$$

Zero extents retain only the central image without disabling geometric periodicity. Nonzero extents increase the number of field evaluations; convergence of the chosen field observable must be checked separately. `EwaldLengthFactor` belongs to the DDD field formulation and does not select periodic faces.

Implementation: `src/DefectiveCrystal/DefectiveCrystalBase.cpp`.

Implementation: `include/GlidePlanes/PeriodicGlidePlane.h`.

Implementation: `include/DislocationDynamics/PeriodicVelocityConstraints.h`.

For a fully periodic cube, use an image-size vector with three entries. For a partially periodic domain, supply exactly one image extent per active basis column; a length mismatch raises an error. If a loop behaves incorrectly at a boundary, distinguish a physical endpoint from an artificial periodic subdivision. Relevant checks include `testPartialPeriodicRemesh.cpp` and `testPeriodicVelocityConstraints.cpp`.

8.4 DDD input and resolution

Copy `Library/DislocationDynamics/DD.txt` to `inputFiles/`. `glideSolverType` and `climbSolverType` choose motion solvers. `Lmin` and `Lmax` control remeshing scales; `quadPerLength` controls line quadrature density; `dxMax` is relevant to adaptive motion stepping. These are distinct discretization choices. Refining the volume FEM mesh does not automatically refine the dislocation network, nor does increasing periodic image counts resolve a long line segment.

Use the material's mobility parameters with the intended crystal and enabled slip systems. To diagnose an unexpected response, first isolate the imposed load and mobility law, then inspect line geometry and topology, and only then assess multi-physics coupling. The tutorial in Chapter 5 provides a small starting workflow.

8.5 Guide to the retained theory

The following appendices keep the previous manual's equations with the physics that uses them:

- Elastic eigendistortion and loop fields: the continuum basis of stress and displacement evaluation, finite-domain superposition, solid angles and multipole expressions.
- Core regularization and straight segments: finite-core kernels and analytical line integration.
- Mathematical identities: surface transport, convolution and anisotropic Green tensors used by the DDD derivations.
- Supplementary notes: earlier distributional identities and the uniform loading derivation.

The purpose is to connect implementation choices to their theory without requiring a reader to begin with a long Green-function derivation. Some inherited passages remain incomplete or unverified and are explicitly marked. In particular, general anisotropic formulas do not establish that the active numerical kernel is anisotropic.

Part IV

Appendices

Appendix A

DDD: elastic theory of discrete dislocations

The key aspect of the DDD method is that dislocation interactions are computed without resorting to expensive atomistic calculations. This is made possible by the elastic theory of dislocations, which provides semi-analytical expressions of the elastic fields (displacement, stress, strain...) generated by arbitrary dislocation loops within an infinite elastic medium. We now briefly summarize this theory, as a special case of Mura's linearized eigendistortion theory [Mura, 1987].

A.1 Eigendistortion theory in elastic media

A.1.1 Eigendistortion theory in infinite anisotropic media

Let us consider an elastic body occupying the infinite three-dimensional space $\mathbb{R}^3$, and let $\boldsymbol{u}(\boldsymbol{x})$ be its displacement field. We adopt here the linearized eigen-distortion framework of Mura [1987], in which the main kinematic assumption is that the displacement gradient is split additively into an elastic distortion $\boldsymbol{\beta}$, and an inelastic distortion $\boldsymbol{\beta}^\star$:

$$u_{i,j} = \beta_{ij} + \beta^\star_{ij}\,. \tag{A.1}$$

The elastic distortion describes the local stretching of the atomic bonds and is related to stress via Hookes law ($\sigma_{ij} = \mathbb{C}_{ijkl}\beta_{kl}$). On the other hand, the inelastic distortion is the *eigendistortion* of the material, and it may contain several (additive) contributions, including plastic deformation, thermal deformation, deformation due to point defects and diffusive species in the lattice, and more. The inelastic distortion is also referred to as an *eigendistortion*, because it is imagined to take place within a fictitious "intermediate" configuration where the body is allowed to deform with incompatibilities (separations and/or interpenetrations) in order to remain stress free. It is the subsequent elastic distortion which is responsible to develop the internal stress field necessary to remove the incompatibilities of the intermediate configuration. It will be shown that an eigendistortion field results in a state of self-stress if it is incompatible, that is if its (negative) curl

$$\alpha_{ij} = -\epsilon_{jkm}\beta^\star_{im,k} \tag{A.2}$$

is non vanishing. If the eigendistortion is due to dislocations, then $\boldsymbol{\alpha}$ can be interpreted as a tensorial density of dislocations, which is known as the Kröner-Nye dislocation density tensor.

In the presence of a body forces density $\boldsymbol{b}$, the static Lagrangian density of the classical linear elastic medium reads

$$\mathcal{L} = -\mathcal{W} - \mathcal{V} = -\frac{1}{2}\mathbb{C}_{ijkl}\beta_{ij}\beta_{kl} + u_i f_i\,,$$

where

$$\mathcal{V} = -u_i b_i \tag{A.3}$$

is the potential of the body force density $\boldsymbol{f}$, while

$$\mathcal{W} = \frac{1}{2}\mathbb{C}_{ijkl}\varepsilon_{ij}\varepsilon_{kl} \tag{A.4}$$

is the strain energy density, which is assumed to be quadratic in the elastic strain

$$\varepsilon_{ij} = \frac{1}{2}\left(\beta_{ij} + \beta_{ji}\right) \tag{A.5}$$

Here $\mathbb{C}_{ijkl}$ is the rank-4 tensor of elastic moduli. By virtue of the symmetries

$$\mathbb{C}_{ijkl} = \mathbb{C}_{jikl} = \mathbb{C}_{ijlk} = \mathbb{C}_{klij}\,, \tag{A.6}$$

it possesses up to 21 independent constants.

The condition of static equilibrium for displacement is expressed by the Euler-Lagrange equation

$$\frac{\delta\mathcal{L}}{\delta u_i} = \frac{\partial\mathcal{L}}{\partial u_i} - \partial_j\frac{\partial\mathcal{L}}{\partial(\partial_j u_i)} = -\sigma_{ij,j} - f_i = . \tag{A.7}$$

where

$$\sigma_{ij} = \frac{\partial W}{\partial\varepsilon_{ij}} = \mathbb{C}_{ijkl}\varepsilon_{kl} = \mathbb{C}_{ijkl}\beta_{kl} \tag{A.8}$$

is the Cauchy stress tensor. Using the additive decomposition (A.1), the equilibrium equation in terms of displacement takes the form of the following inhomogeneous Navier equation:

$$L_{ik}u_k = \mathbb{C}_{ijkl}\beta^{\star}_{kl,j}\,, \tag{A.9}$$

where

$$L_{ik} = \mathbb{C}_{ijkl}\partial_j\partial_l \tag{A.10}$$

is the Navier differential operator. This operator admits the well-known Green's tensor G_{im} given in Eq. (D.22). Because the Green's tensor G_{im} is the fundamental solution of the Navier operator, the particular solution of Eq. (A.9) can be obtained from it by convolution with the source term. In particular, for an infinite medium the displacement field reads [Mura, 1987]:

$$\begin{aligned} u_i &= -\mathbb{C}_{mnpq}G_{im} * \beta^{\star}_{pq,n} \\ &= -\mathbb{C}_{mnpq}G_{im,n} * \beta^{\star}_{pq} \qquad \text{(generalized Volterra equation)}\,. \end{aligned} \tag{A.11}$$

In Eq. (A.11) we have introduced the symbol $*$ to indicate convolution over the infinite three-dimensional space. With dislocation theory in mind, Eq. (A.11) can be considered a *generalized Volterra solution* valid for an arbitrary source of eigendistortion.

An expression for the displacement gradient split into elastic and plastic contributions is obtained using the Mura-Willis procedure:

$$\begin{aligned} u_{i,j} &= -\mathbb{C}_{mnpq}G_{im,n} * \beta^{P}_{pq,j} = -\mathbb{C}_{mnpq}G_{im,n} * \left(\beta^{P}_{pj,q} + \epsilon_{qjr}\alpha_{pr}\right) \\ &= -\mathbb{C}_{mnpq}G_{im,nq} * \beta^{P}_{pj} - \mathbb{C}_{mnpq}\epsilon_{qjr}G_{im,n} * \alpha_{pr} \\ &= -L_{pm}G_{im} * \beta^{P}_{pj} - \mathbb{C}_{mnpq}\epsilon_{qjr}G_{im,n} * \alpha_{pr} = \delta_{ip}\delta * \beta^{P}_{pj} - \mathbb{C}_{mnpq}\epsilon_{qjr}G_{im,n} * \alpha_{pr} \\ &= \beta^{P}_{ij} + \mathbb{C}_{mnpq}\epsilon_{jqr}G_{im,n} * \alpha_{pr} \end{aligned} \tag{A.12}$$

From (A.12), the elastic distortion follows immediately as

$$\beta_{ij} = \mathbb{C}_{mnpq}\epsilon_{jqr}G_{im,n} * \alpha_{pr}\,. \qquad \text{(Mura-Willis equation)} \tag{A.13}$$

Clearly, the Cauchy stress field σ_{ij} can be obtained from (A.13) using the Hooke's law (A.8).

$$\sigma_{ij} = \mathbb{C}_{ijkl}\mathbb{C}_{mnpq}\epsilon_{lqr}G_{km,n} * \alpha_{pr} \qquad \text{(anisotropic Peach-Koehler stress equation)}\,. \tag{A.14}$$

Note that elastic distortion (A.13) and stress (A.14) depend on the eigendistortion $\boldsymbol{\beta}^\star$ only through its curl $\boldsymbol{\alpha}$, and therefore they vanish is $\boldsymbol{\beta}^\star$ is compatible.

We are now interested in obtaining an alternative expression of the displacement field (A.11) where the contribution of elastic and plastic distortions appear in separate additive terms. Following [Lazar and Kirchner, 2013], we first take the derivative of Eq. (A.1) with respect to x_j in order to obtain the Poisson equation:

$$\Delta u_i = u_{i,jj} = \beta_{ij,j} + \beta^\star_{ij,j} \ , \tag{A.15}$$

where $\Delta = \partial_j \partial_j$ is the Laplace operator. Second we "invert" Eq. (A.15) using the Green's function[1] of the Laplace operator G^Δ:

$$u_i = \left(\beta^\star_{ij,j} + \beta_{ij,j}\right) * G^\Delta = \beta^\star_{ij} * G^\Delta_{,j} + \beta_{ij,j} * G^\Delta \, . \tag{A.17}$$

It will be clearer in the following sections that Eq. (A.17) can be considered as a *generalized Burgers equation*. In fact, although Eq. (A.17) is valid for an arbitrary source of eigendistortion in an anisotropic medium, its name is well-justified in the case of dislocation loops because the term $\beta^\star_{ij} * G^\Delta_{,j}$ corresponds to the contribution of the solid angle subtended by a loop, which was first isolated by Burgers [Burgers, 1939a,b] in the classical isotropic case. Substituting Eq. (A.13) in (A.17), we find the generalized Burgers solution for the displacement field (see also Lazar and Kirchner [2013]):

$$u_i = G^\Delta_{,j} * \beta^\star_{ij} - \mathbb{C}_{mnpq}\epsilon_{jqr} F_{jnim} * \alpha_{pr} \qquad \text{(generalized Burgers equation)} \, . \tag{A.18}$$

Note that, following Kirchner [1984] and Lazar and Kirchner [2013], in Eq. (A.18) we have introduced the fourth-rank tensor F_{jnim} defined as:

$$F_{jnim} = -G_{im,jn} * G^\Delta \, . \tag{A.19}$$

Similar to the Green's tensor, also the "$\boldsymbol{F}$-tensor" has an explicit form, which is given in Eq. (D.25).

The $\boldsymbol{F}$-tensor is also useful in deriving a compact expression for the interaction energy between two sources of eigendistortion. In fact, using the identity

$$G_{im,j} = G_{im,jnn} * G^\Delta = -F_{jnim,n} \, , \tag{A.20}$$

the interaction energy W_{AB} between two between two sources of eigendistortion, labeled A and B respectively, can be obtained as follows [Po et al., 2018]:

$$\begin{aligned}
W_{AB} &= \int_{\mathbb{R}^3} \sigma^{(A)}_{il} \, \beta^{(B)}_{il} \, \mathrm{d}V = \int_{\mathbb{R}^3} \mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(G_{mr,s} * \alpha^{(A)}_{tq}\right) \beta^{(B)}_{il} \, \mathrm{d}V \\
&= -\int_{\mathbb{R}^3} \mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(F_{skmr,k} * \alpha^{(A)}_{tq}\right) \beta^{(B)}_{il} \, \mathrm{d}V = \int_{\mathbb{R}^3} \mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(F_{skmr} * \alpha^{(A)}_{tq}\right) \beta^{(B)}_{il,k} \, \mathrm{d}V \\
&= \int_{\mathbb{R}^3} \mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(F_{skmr} * \alpha^{(A)}_{tq}\right) \left(\beta^{(B)}_{ik,l} + \epsilon_{jkl}\alpha^{(B)}_{ij}\right) \mathrm{d}V \\
&= -\int_{\mathbb{R}^3} \underbrace{\mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(G_{mr,skl} * \alpha^{(A)}_{tq}\right)}_{\sigma_{il,lk}=0} * G^\Delta \, \beta^{(B)}_{ik} \, \mathrm{d}V \\
&+ \int_{\mathbb{R}^3} \epsilon_{jkl}\mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(F_{skmr} * \alpha^{(A)}_{tq}\right) \alpha^{(B)}_{ij} \, \mathrm{d}V \\
&= \int_{\mathbb{R}^3} \epsilon_{jkl}\mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp} \left(F_{skmr} * \alpha^{(A)}_{tq}\right) \alpha^{(B)}_{ij} \, \mathrm{d}V \quad \text{(generalized Blin's interaction energy equation)} \, .
\end{aligned} \tag{A.21}$$

This result shows that the interaction energy between two sources of eigendistortion depends only on their curls. Therefore it can be regarded as an anisotropic generalization of Blin's equation [Blin, 1955], valid for arbitrary sources of eigendistortion[2].

[1] The three-dimensional Green's function of the Laplace operator is [Vladimirov, 1971]

$$G^\Delta(\boldsymbol{R}) = -\frac{1}{4\pi R} \tag{A.16}$$

where $R = \sqrt{\boldsymbol{R}\cdot\boldsymbol{R}}$ is the Euclidean norm of $\boldsymbol{R}$.

[2] Surprisingly, such fundamental result has long been missing in anisotropic eigendistortion theory, and it was found only recently by Lazar and Kirchner [2013] using the method of stress functions.

while the configurational force exerted on the eigendistortion can be computed from the divergence of Eshelby's stress tensor. This gives the Peach-Koehler force on the eigendistortion:

$$\mathcal{F}_k = \int_{\mathbb{R}^3} \left(W\delta_{kj} - \sigma_{ij}\beta_{ik}\right)_{,j} \mathrm{d}V = \int_{\mathbb{R}^3} \epsilon_{kjm}\sigma_{ij}\alpha_{im}\,\mathrm{d}V\,. \tag{A.22}$$

A.1.2 Eigendistortion theory in infinite isotropic media

In the isotropic case: Starting from (A.18)

$$c_{mnpq} = \mu\left(\frac{2\nu}{1-2\nu}\delta_{mn}\delta_{pq} + \delta_{mp}\delta_{nq} + \delta_{mq}\delta_{np}\right) \tag{A.23}$$

and

$$G_{im,n}(\boldsymbol{x}) = \frac{1}{16\pi\mu(1-\nu)}\left[2(1-\nu)\delta_{im}\partial_n\Delta - \partial_n\partial_i\partial_m\right]R(\boldsymbol{x}) \tag{A.24}$$

where $R(\boldsymbol{x}) = \sqrt{\boldsymbol{x}\cdot\boldsymbol{x}}$ is the distance function. Therefore

$$c_{mnpq}G_{im,n} = \frac{1}{8\pi(1-\nu)}\left[\nu\delta_{pq}\partial_i\Delta + (1-\nu)\left(\delta_{ip}\partial_q\Delta + \delta_{iq}\partial_p\Delta\right) - \partial_i\partial_p\partial_q\right]R(\boldsymbol{x}) \tag{A.25}$$

and obtain

$$\begin{aligned} u_i &= -\frac{b_i\Omega}{4\pi} - \frac{1}{8\pi(1-\nu)}\left[(1-\nu)\Delta R * \epsilon_{kip}\alpha_{pk} - \partial_i\partial_q R * \epsilon_{kqp}\alpha_{pk}\right] \\ &= -\frac{b_i\Omega}{4\pi} - \frac{1}{8\pi}\epsilon_{kqp}\left[\delta_{iq}\Delta - \frac{1}{1-\nu}\partial_i\partial_q\right]R * \alpha_{pk} \\ &= -\frac{b_i\Omega}{4\pi} - \frac{1}{8\pi}\epsilon_{lmk}\left[\delta_{im}\Delta - \frac{1}{1-\nu}\partial_i\partial_m\right]R * \alpha_{kl} \\ &= -\frac{b_i\Omega}{4\pi} - U_{ikl}R * \alpha_{kl} \end{aligned} \tag{A.26}$$

The operator U_{ill} is:

$$U_{ikl} = \frac{1}{8\pi}\epsilon_{klm}\left[\delta_{im}\Delta - \frac{1}{1-\nu}\partial_i\partial_m\right] \tag{A.27}$$

In order to find a compact expression for the displacement gradient we define the function

$$B_{ijkl}(\boldsymbol{x}) = \mathbb{C}_{mnkq}\epsilon_{jql}G_{im,n}(\boldsymbol{x}) = \frac{1}{8\pi}\left[\left(-\delta_{ik}\epsilon_{jlm}\partial_m + \epsilon_{ilk}\partial_j\right)\Delta + \frac{1}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right]R(\boldsymbol{x}) \tag{A.28}$$

With this definition we obtain the displacement gradient as

$$u_{i,j} = \beta^P_{ij} + \mathbb{C}_{mnkq}\epsilon_{jql}G_{im,n} * \alpha_{kl} = \beta^P_{ij} + B_{ijkl} * \alpha_{kl} \tag{A.29}$$

which corresponds to the result of de Wit p 264.

Elastic distortion

$$\beta^E_{ij} = u_{i,j} - \beta^P_{ij} = B_{ijkl}R * \alpha_{kl} \tag{A.30}$$

Elastic strain

$$\varepsilon_{ij}^E = \frac{1}{2}\left(\beta_{ij}^E + \beta_{ji}^E\right) = E_{ijkl} R * \alpha_{kl} \tag{A.31}$$

$$\begin{aligned}
E_{ijkl} &= \frac{1}{2}\left(B_{ijkl} + B_{jikl}\right) \\
&= \frac{1}{16\pi}\left[\left(-\delta_{ik}\epsilon_{jlm}\partial_m - \delta_{jk}\epsilon_{ilm}\partial_m + \epsilon_{ilk}\partial_j + \epsilon_{jlk}\partial_i\right)\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\left(-\delta_{ik}\epsilon_{jlm}\delta_{mr} - \delta_{jk}\epsilon_{ilm}\delta_{mr} + \epsilon_{ilk}\delta_{jr} + \epsilon_{jlk}\delta_{ir}\right)\partial_r\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\left(\epsilon_{ilk}\delta_{jr} - \delta_{jk}\epsilon_{ilm}\delta_{mr} + \epsilon_{jlk}\delta_{ir} - \delta_{ik}\epsilon_{jlm}\delta_{mr}\right)\partial_r\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\left(\epsilon_{ilm}(\delta_{mk}\delta_{jr} - \delta_{jk}\delta_{mr}) + \epsilon_{jlm}(\delta_{mk}\delta_{ir} - \delta_{ik}\delta_{mr})\right)\partial_r\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\left(\epsilon_{ilm}\epsilon_{smj}\epsilon_{skr} + \epsilon_{jlm}\epsilon_{smi}\epsilon_{skr}\right)\partial_r\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\epsilon_{skr}\left(\epsilon_{mil}\epsilon_{mjs} + \epsilon_{mjl}\epsilon_{mis}\right)\partial_r\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\epsilon_{skr}\left(\delta_{ij}\delta_{ls} - \delta_{is}\delta_{lj} + \delta_{ij}\delta_{ls} - \delta_{js}\delta_{il}\right)\partial_r\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right] \\
&= \frac{1}{16\pi}\left[\left(2\epsilon_{lkm}\delta_{ij} - \epsilon_{ikm}\delta_{lj} - \epsilon_{jkm}\delta_{il}\right)\partial_m\Delta + \frac{2}{1-\nu}\epsilon_{klm}\partial_i\partial_j\partial_m\right]
\end{aligned} \tag{A.32}$$

same as de Wit p 264.

Stress

$$\sigma_{pq} = c_{pqij}\beta_{ij}^E = \mu\left(\frac{2\nu}{1-2\nu}\delta_{pq}\delta_{ij} + \delta_{pi}\delta_{qj} + \delta_{pj}\delta_{qi}\right)E_{ijkl}R * \alpha_{kl} = S_{pqkl} * \alpha_{kl} \tag{A.33}$$

$$S_{ijkl} = \frac{\mu}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk} + \delta_{jl}\epsilon_{imk}\right)\partial_m\Delta + \frac{2}{1-\nu}\epsilon_{klm}\left(\partial_i\partial_j\partial_m - \delta_{ij}\partial_m\Delta\right)\right] \tag{A.34}$$

A.1.3 Eigendistortion theory in finite media: the superposition principle

So far we have discussed the DDD method in the infinite elastic medium. However, many applications in micro-plasticity, such as micro indentation and micro-pillar compression, require that the dislocation network be embedded in a finite domain subject to prescribed boundary conditions. In order to extend the DDD method to finite domains, let us consider the eigenstrain problem in a finite domain $\mathcal{B}$. In this case, the displacement field u_k must satisfy the following boundary value problem (BVP) :

$$\begin{cases}
\sigma_{ij} = c_{ijkl}\left(u_{k,l} - \beta_{kl}^\star\right) & \text{in } \mathcal{B} \\
\sigma_{ij,j} = 0 & \text{in } \mathcal{B} \\
\sigma_{ij}\hat{n}_j = p_i & \text{on } \partial_N\mathcal{B} \\
u_i = \overline{u}_i & \text{on } \partial_D\mathcal{B}
\end{cases} \tag{A.35}$$

where p_i is the prescribed traction on the portion of the boundary $\partial_N\mathcal{B}$, and $\overline{u}_i$ is the prescribed displacement on the portion of the boundary $\partial_D\mathcal{B}$. Ideally, the BVP (A.35) could be solved at each DDD step by incrementally updating

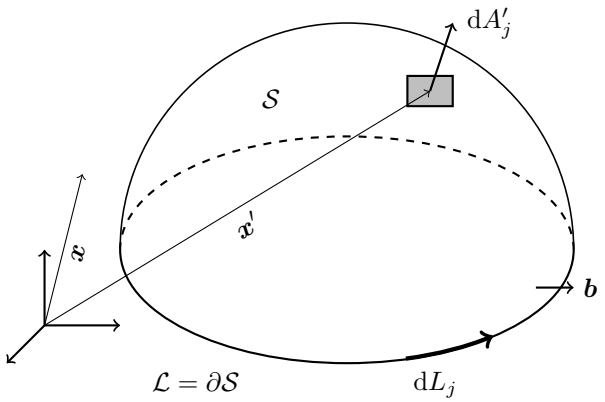


Figure A.1: The plastic distortion is concentrated on the surface $\mathcal{S}$, which is bounded by the dislocation line $\mathcal{L} = \partial\mathcal{S}$.

β^P_{ij} using its rate from eq. (A.41). In a finite element framework this would provide the nodal displacements, from which the total stress could be computed and used to drive the discrete dislocation system inside $\mathcal{B}$. Practically, however, this approach is unfeasible because stresses, obtained as derivatives of displacement shape functions, would result in smooth fields which fail to capture sharp gradients in proximity of dislocation lines. In order to overcome this issue, an alternative formulation has been sought (c.f. Eshelby [1979], Lubarda et al. [1993], Van der Giessen and Needleman [1995], Deng et al. [2008], Weygand et al. [2002]), where the problem is decomposed into two auxiliary problems. The first problem, with infinite domain $\mathbb{R}^3$, consists in finding u^∞_i such that:

$$\begin{cases} \sigma^\infty_{ij} = c_{ijkl}(u^\infty_{k,l} - \beta^P_{kl}) & \text{in } \mathbb{R}^3 \\ \sigma^\infty_{ij,j} = 0 & \text{in } \mathbb{R}^3 \end{cases} \tag{A.36}$$

The second is a purely elastic “correction" problem, with domain $\mathcal{B}$ consisting in finding u^c_i such that:

$$\begin{cases} \sigma^c_{ij} = c_{ijkl}u^c_{k,l} & \text{in } \mathcal{B} \\ \sigma^c_{ij,j} = 0 & \text{in } \mathcal{B} \\ \sigma^c_{ij}\hat{n}_j = p_i - \sigma^\infty_{ij}\hat{n}_j & \text{on } \partial_N\mathcal{B} \\ u^c_i = \overline{u}_i - u^\infty_i & \text{on } \partial_D\mathcal{B} \end{cases} \tag{A.37}$$

The rationale behind this decomposition is clearly that (A.36) is the standard dislocation eigenstrain problem in an infinite medium that we already solved in the previous sections. The total solution is clearly obtained as a sum: $u_i = u^\infty_i + u^c_i$ and the total stress field $\sigma_{ij} = \sigma^\infty_{ij} + \sigma^c_{ij}$ is used to drive dislocations in the finite-domain version of DDD.

A.2 Discrete dislocation loops

A.2.1 Discrete dislocation loops in infinite anisotropic media

Let us know assume that the eigendistortion $\boldsymbol{\beta}^\star$ is only due to plastic deformation cause by dislocations, that is

$$\boldsymbol{\beta}^\star = \boldsymbol{\beta}^P \tag{A.38}$$

The key equations of classical dislocation theory are now obtained by prescribing the form of the eigendistortion tensor $\boldsymbol{\beta}^P$. For a dislocation extending over a surface $\mathcal{S}$, the classical plastic eigendistortion tensor is taken in following form:

$$\beta^P_{kl}(\boldsymbol{x}) = -\int_{\mathcal{S}} \delta(\boldsymbol{x} - \boldsymbol{x}')b_k \,\mathrm{d}A'_l\,, \tag{A.39}$$

where δ is the Dirac delta function, and b_i is the displacement jump across $\mathcal{S}$, or Burgers vector. For a Volterra dislocation, as opposed to a Somigliana dislocation, the Burgers vector is constant. In this case, the tensor

$\alpha_{ij} = -\epsilon_{jkm}\beta^P_{im,k}$ turns out to be concentrated on the dislocation line, that is the closed line $\mathcal{L} = \partial\mathcal{S}$ bounding the surface $\mathcal{S}$, and it is known as the *dislocation density tensor*:

$$\alpha_{ij}(\boldsymbol{x}) = \oint_{\mathcal{L}} \delta(\boldsymbol{x} - \boldsymbol{x}')b_i \, \mathrm{d}L'_j \, . \tag{A.40}$$

The rate of plastic distortion can be written in terms of the local dislocation velocity w_k as

$$\dot{\beta}^P_{ij}(\boldsymbol{x}) = -\oint_{\mathcal{L}} \delta(\boldsymbol{x} - \boldsymbol{x}')b_i\epsilon_{jkm}w_k(\boldsymbol{x}') \, \mathrm{d}L'_m \, . \tag{A.41}$$

Using the special forms (A.39) and (A.40) in the results obtained in the previous section we find all the key equations of classical dislocation theory in the anisotropic case. Letting $\boldsymbol{R} = \boldsymbol{x} - \boldsymbol{x}'$, these are:

$$u_i(\boldsymbol{x}) = -\frac{b_i \, \Omega(\boldsymbol{x})}{4\pi} - \oint_{\mathcal{L}} \mathbb{C}_{mnpq}\epsilon_{jqr}b_p F_{jnim}(\boldsymbol{R}) \, \mathrm{d}L'_r \qquad \text{(anisotropic Burgers equation)} \tag{A.42a}$$

$$\beta_{ij}(\boldsymbol{x}) = \oint_{\mathcal{L}} \mathbb{C}_{mnpq}\epsilon_{jqr}G_{im,n}(\boldsymbol{R})b_p \, \mathrm{d}L'_r \qquad \text{(Mura-Willis equation)} \tag{A.42b}$$

$$\sigma_{ij}(\boldsymbol{x}) = \oint_{\mathcal{L}} \mathbb{C}_{ijkl}\mathbb{C}_{mnpq}\epsilon_{lqr}G_{km,n}(\boldsymbol{R})b_p \, \mathrm{d}L'_r \qquad \text{(anisotropic Peach-Koehler stress equation)} \tag{A.42c}$$

$$W_{AB} = \oint_{\mathcal{L}_A}\oint_{\mathcal{L}_B} \epsilon_{jkl}\mathbb{C}_{ilmn}\epsilon_{npq}\mathbb{C}_{rstp}F_{skmr}(\boldsymbol{R}) \, b^A_t b^B_i \, \mathrm{d}L^A_q \, \mathrm{d}L^B_j \qquad \text{(anisotropic Blin's formula)} \tag{A.42d}$$

$$\mathcal{F}_k = \oint_{\mathcal{L}} \epsilon_{kjm}\sigma_{ij}b_i \, \mathrm{d}L_m \qquad \text{(Peach-Koehler force)} \tag{A.42e}$$

Note that, in the Burgers equation, we have introduced the solid angle $\Omega(\boldsymbol{x})$ subtended by the loop from the relationship $G^{\Delta}_{,j} * \beta^P_{ij} = -b_i \, \Omega/4\pi$, which yields:

$$\Omega(\boldsymbol{x}) = 4\pi \int_{\mathcal{S}} G^{\Delta}_{,j} \, \mathrm{d}A'_j = \int_{\mathcal{S}} \frac{R_j}{R^3} \, \mathrm{d}A'_j \, . \tag{A.43}$$

Status of the inherited derivation

The source notes stop before discussing the kernel of the preceding loop-field expressions. That missing derivation has not been reconstructed here.

A.2.2 Discrete dislocation loops in infinite isotropic media

Status of the inherited derivation

The original isotropic specialization contains working equations but not a complete connecting derivation.

$$\boldsymbol{u} = -\frac{\boldsymbol{b}\Omega}{4\pi} - \frac{1}{4\pi}\oint \left[\frac{\boldsymbol{b}\times\boldsymbol{\xi}}{R} + \frac{\lambda+\mu}{\lambda+2\mu}\left(\frac{\boldsymbol{\xi}\times\boldsymbol{b}}{R} - \frac{(\boldsymbol{b}\times\boldsymbol{R})\cdot\boldsymbol{\xi}}{R^3}\boldsymbol{R}\right)\right] dl \tag{A.44}$$

where

$$\Omega = \int \frac{R_k\hat{n}_k}{R^3} dS = -\frac{1}{2}\int R_{,ppk}\hat{n}_k dS \tag{A.45}$$

When using the line integral expression of the solid angle (D.9), eq. (A.44) becomes:

$$\begin{aligned}\boldsymbol{u} &= \frac{1}{4\pi}\oint \left[\frac{(\boldsymbol{s}\times\boldsymbol{R})\cdot\boldsymbol{\xi}}{R(R+\boldsymbol{s}\cdot\boldsymbol{R})}\boldsymbol{b} + \frac{\boldsymbol{\xi}\times\boldsymbol{b}}{R} + \frac{1}{2(1-\nu)}\left(\frac{\boldsymbol{b}\times\boldsymbol{\xi}}{R} + \frac{(\boldsymbol{b}\times\boldsymbol{R})\cdot\boldsymbol{\xi}}{R^3}\boldsymbol{R}\right)\right] dl \\ &= \underbrace{\frac{1}{8\pi(1-\nu)}}_{c_4}\oint \frac{1}{R}\left[2\underbrace{(1-\nu)}_{c1}\frac{(\boldsymbol{s}\times\boldsymbol{R})\cdot\boldsymbol{\xi}}{R+\boldsymbol{s}\cdot\boldsymbol{R}}\boldsymbol{b} + \underbrace{(1-2\nu)}_{c_3}\boldsymbol{\xi}\times\boldsymbol{b} + \frac{(\boldsymbol{\xi}\times\boldsymbol{b})\cdot\boldsymbol{R}}{R^2}\boldsymbol{R}\right] dl\end{aligned} \tag{A.46}$$

Stress field

$$\sigma_{ij}(\boldsymbol{F}) = \frac{\mu b_n}{8\pi}\int_{\mathcal{S}}\left[R_{,mpp}\left(\in_{jmn}\xi_i + \in_{imn}\xi_j\right) + \frac{2}{1-\nu}\in_{kmn}\left(R_{,ijm} - \delta_{ij}R_{,ppm}\right)\xi_k\right]dl^s \tag{A.47}$$

where $\boldsymbol{R} = \boldsymbol{F} - \boldsymbol{S}$ is the vector connecting the source point to the field point and $\xi = \frac{d\boldsymbol{S}}{dl^s}$ s the unit tangent along the source dislocation.

Substituting

$$R_{,ijk} = -\frac{\delta_{ij}R_k + \delta_{jk}R_i + \delta_{ki}R_j}{R^3} + \frac{3R_iR_jR_k}{R^5} \qquad R_{,ipp} = -\frac{2R_i}{R^3} \tag{A.48}$$

$$\sigma_{ij}(\boldsymbol{F}) = \frac{\mu b_n}{4\pi}\int_{\mathcal{S}}\left[-\frac{R_m}{R^3}\left(\in_{jmn}\xi_i + \in_{imn}\xi_j\right) + \frac{1}{1-\nu}\in_{kmn}\left(\frac{\delta_{ij}R_m - \delta_{jm}R_i - \delta_{mi}R_j}{R^3} + \frac{3R_iR_jR_m}{R^5}\right)\xi_k\right]dl^s \tag{A.49}$$

or in dyadic form:

$$\begin{aligned}\frac{d\boldsymbol{\sigma}}{dl} &= \frac{\mu}{4\pi R^2}\Big\{\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\hat{\boldsymbol{R}}) + (\boldsymbol{b}\times\hat{\boldsymbol{R}})\otimes\hat{\boldsymbol{\xi}} \\ &+ \frac{1}{1-\nu}\Big[\hat{\boldsymbol{R}}\otimes(\hat{\boldsymbol{\xi}}\times\boldsymbol{b}) + (\hat{\boldsymbol{\xi}}\times\boldsymbol{b})\otimes\hat{\boldsymbol{R}} + \Big[(\hat{\boldsymbol{R}}\times\boldsymbol{b})\cdot\hat{\boldsymbol{\xi}}\Big]\Big(\boldsymbol{I} + 3\hat{\boldsymbol{R}}\otimes\hat{\boldsymbol{R}}\Big)\Big]\Big\}\end{aligned} \tag{A.50}$$

more efficient to compute

$$\begin{aligned}\frac{d\boldsymbol{\sigma}}{dl} &= \frac{\mu}{4\pi(1-\nu)R^2}\Big\{(1-\nu)\Big[\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\hat{\boldsymbol{R}}) + (\boldsymbol{b}\times\hat{\boldsymbol{R}})\otimes\hat{\boldsymbol{\xi}}\Big] \\ &+ \hat{\boldsymbol{R}}\otimes(\hat{\boldsymbol{\xi}}\times\boldsymbol{b}) + (\hat{\boldsymbol{\xi}}\times\boldsymbol{b})\otimes\hat{\boldsymbol{R}} \\ &+ \Big[(\hat{\boldsymbol{R}}\times\boldsymbol{b})\cdot\hat{\boldsymbol{\xi}}\Big]\Big(\boldsymbol{I} + 3\hat{\boldsymbol{R}}\otimes\hat{\boldsymbol{R}}\Big)\Big\}\end{aligned} \tag{A.51}$$

or factoring the symmetric part

$$\frac{d\boldsymbol{\sigma}}{dl} = \frac{\mu}{2\pi(1-\nu)R^2}\left\{(1-\nu)\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\hat{\boldsymbol{R}}) + \hat{\boldsymbol{R}}\otimes(\hat{\boldsymbol{\xi}}\times\boldsymbol{b}) + \frac{1}{2}\Big[(\hat{\boldsymbol{R}}\times\boldsymbol{b})\cdot\hat{\boldsymbol{\xi}}\Big]\Big(\boldsymbol{I} + 3\hat{\boldsymbol{R}}\otimes\hat{\boldsymbol{R}}\Big)\right\}_{sym} \tag{A.52}$$

where $\boldsymbol{x}_{\text{sym}} = \frac{1}{2}(\boldsymbol{x} + \boldsymbol{x}^T)$

A.2.3 Line-integral representation of the solid angle subtended by a loop

$$\boldsymbol{u}^0(\boldsymbol{x}) = -\frac{\boldsymbol{b}\Omega^0}{4\pi} - \frac{1}{8\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R}\Big\{\,(1-2\nu)\,\boldsymbol{b}\times\hat{\boldsymbol{\xi}}' + \Big[\hat{\boldsymbol{R}}\cdot\Big(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}'\Big)\Big]\hat{\boldsymbol{R}}\Big\}\,\mathrm{d}L' \tag{A.53}$$

$$\begin{aligned}\boldsymbol{\sigma}^0(\boldsymbol{x}) = \frac{\mu}{4\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R^2}\Big\{&(1-\nu)\Big[\hat{\boldsymbol{\xi}}'\otimes(\boldsymbol{b}\times\hat{\boldsymbol{R}}) + (\boldsymbol{b}\times\hat{\boldsymbol{R}})\otimes\hat{\boldsymbol{\xi}}'\Big] + \Big[(\hat{\boldsymbol{\xi}}'\times\boldsymbol{b})\otimes\hat{\boldsymbol{R}} + \hat{\boldsymbol{R}}\otimes(\hat{\boldsymbol{\xi}}'\times\boldsymbol{b})\Big] \\ &+\hat{\boldsymbol{R}}\cdot(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}')\Big[3\hat{\boldsymbol{R}}\otimes\hat{\boldsymbol{R}} + \boldsymbol{I}\Big]\Big\}\,\mathrm{d}L'\end{aligned} \tag{A.54}$$

Numerical implementation

$$\mathbf{u}(\boldsymbol{x}) = -\frac{\mathbf{b}\Omega(\mathbf{x})}{4\pi} - \frac{1}{8\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R}\Big\{\,(1-2\nu)\,\mathbf{b}\times\hat{\xi}' + \Big[\hat{\mathbf{R}}\cdot\Big(\mathbf{b}\times\hat{\xi}'\Big)\Big]\hat{\mathbf{R}}\Big\}\,\mathrm{d}L' \tag{A.55}$$

where $\mathbf{R}=\mathbf{x}-\mathbf{x}'$, and $\Omega(\mathbf{x})$ is the solid angle subtended by the surface $\mathcal{S}$:

$$\Omega(\mathbf{x})=\int_{\mathcal{S}}\left(-\frac{1}{2}\partial_k\Delta R\right)\,dA'_k=\int_{\mathcal{S}}v_k(\mathbf{R})\,dA'_k \qquad\qquad v_k(\mathbf{y})=\frac{y_k}{y^3} \tag{A.56}$$

In order to transform the solid angle into a line integral, we need to introduce a fictitious vector field $v^f_k(\mathbf{R})$ with the property $v^f_{k,k}=-v_{k,k}$. In this way we can find a vector potential $\Psi_m(\mathbf{R})$ for the divergence-less sum $v_k+v^f_k$. The vector potential satisfies both $v_k+v^f_k=\epsilon_{klm}\Psi_{m,l}$, and $\Psi_{k,k}=0$. Therefore, using Stokes theorem, we can write

$$\begin{aligned}\Omega(\mathbf{x})&=\int_{\mathcal{S}}\left(\epsilon_{klm}\Psi_{m,l}-v^f_k\right)\,dA'_k=\int_{\mathcal{S}}\left(-\epsilon_{klm}\Psi_{m,l'}-v^f_k\right)\,dA'_k\\&=-\oint_{\mathcal{L}}\Psi_m\,dL'_m-\int_{\mathcal{S}}v^f_k\,dA'_k=-\oint_{\mathcal{L}}\Psi_i\,dL'_i-\Omega^f\end{aligned} \tag{A.57}$$

Note that there is an error in Po-Lazar 2014, where the first term in the last equation has the wrong sign. We now need to find the vector potential Ψ_i and the fictitious vector field v^f_i. To do this we consider the auxiliary curve (Dirac string) $\mathcal{D}$, starting from $\mathbf{x}$ and ending at a point at infinity. Using the Dirac string, the fictitious vector can be found as

$$v^f_k(\mathbf{R})=\int_{\mathcal{D}}v_{m,m}(\mathbf{R}+\mathbf{s})ds_k=\int_{\mathcal{D}}\left(-\frac{1}{2}\Delta\Delta|\mathbf{x}+\mathbf{s}|\right)ds_k=4\pi\int_{\mathcal{D}}\delta(\mathbf{R}+\mathbf{s})ds_k \tag{A.58}$$

The aforementioned property of $\mathbf{v}^f$ can be verified as follows:

$$\begin{aligned}v^f_{k,k}(\mathbf{R})&=\frac{\partial}{\partial x_k}\int_{\mathcal{D}}v_{m,m}(\mathbf{R}+\mathbf{s})ds_k=\int_{\mathcal{D}}\frac{\partial}{\partial x_k}v_{m,m}(\mathbf{R}+\mathbf{s})ds_k\\&=\int_{\mathcal{D}}\frac{\partial}{\partial s_k}v_{m,m}(\mathbf{R}+\mathbf{s})ds_k=\left[v_{m,m}(\mathbf{R}+\mathbf{s})\right]^{\infty}_{\mathbf{0}}=-v_{m,m}(\mathbf{R})\end{aligned} \tag{A.59}$$

Knowing the fictitious vector, the vector potential can be found observing that

$$-\Delta\Psi_i=\epsilon_{ijk}(v_k+v^f_k)_{,j}=\epsilon_{ijk}v^f_{k,j}=-\epsilon_{ijk}\partial_j\int_{\mathcal{D}}\frac{1}{2}\Delta\Delta|\mathbf{R}+\mathbf{s}|ds_k \tag{A.60}$$

Therefore

$$\Psi_i(\mathbf{R})=\epsilon_{ijk}\int_{\mathcal{D}}\frac{1}{2}\partial_j\Delta|\mathbf{R}+\mathbf{s}|ds_k=-\epsilon_{ijk}\int_{\mathcal{D}}\frac{R_j+s_j}{|\mathbf{R}+\mathbf{s}|^3}ds_k \tag{A.61}$$

Choosing the Dirac string to be a straight line with unit direction $\hat{\mathbf{s}}$, the expression above becomes:

$$\begin{aligned}\Psi_i(\mathbf{R})&=-\epsilon_{ijk}\int_0^{\infty}\frac{R_j+\alpha\hat{s}_j}{|\mathbf{R}+\alpha\hat{\mathbf{s}}|^3}\hat{s}_k\,d\alpha=-\epsilon_{ijk}R_j\hat{s}_k\int_0^{\infty}\frac{1}{|\mathbf{R}+\alpha\hat{\mathbf{s}}|^3}\,d\alpha\\&=-\epsilon_{ijk}R_j\hat{s}_k\left[\frac{\alpha+\mathbf{R}\cdot\hat{\mathbf{s}}}{(R^2-(\mathbf{R}\cdot\hat{\mathbf{s}})^2)\,|\mathbf{R}+\alpha\hat{\mathbf{s}}|}\right]^{\infty}_0\\&=-\epsilon_{ijk}R_j\hat{s}_k\left[\frac{1}{R^2-(\mathbf{R}\cdot\hat{\mathbf{s}})^2}-\frac{\mathbf{R}\cdot\hat{\mathbf{s}}}{(R^2-(\mathbf{R}\cdot\hat{\mathbf{s}})^2)\,R}\right]\\&=-\epsilon_{ijk}R_j\hat{s}_k\frac{R-\mathbf{R}\cdot\hat{\mathbf{s}}}{(R^2-(\mathbf{R}\cdot\hat{\mathbf{s}})^2)\,R}=-\frac{\epsilon_{ijk}R_j\hat{s}_k}{R\,(R+\mathbf{R}\cdot\hat{\mathbf{s}})}\end{aligned} \tag{A.62}$$

In vector form this reads

$$\boldsymbol{\Psi}(\mathbf{x})=-\frac{\mathbf{R}\times\hat{\mathbf{s}}}{R\,(R+\mathbf{R}\cdot\hat{\mathbf{s}})} \tag{A.63}$$

Finally, the Burgers equation becomes

$$\begin{aligned}\mathbf{u}(\mathbf{x}) &= -\frac{\mathbf{b}}{4\pi}\left(-\oint_{\mathcal{L}}\Psi_i(\mathbf{R})\;dL'_i-\Omega^f\right)-\frac{1}{8\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R}\left\{\;(1-2\nu)\,\mathbf{b}\times\hat{\xi}'+\left[\hat{\mathbf{R}}\cdot\left(\mathbf{b}\times\hat{\xi}'\right)\right]\hat{\mathbf{R}}\right\}\mathrm{d}L'\\ &= \frac{\mathbf{b}\Omega^f}{4\pi}-\frac{\mathbf{b}}{4\pi}\oint_{\mathcal{L}}\frac{(\mathbf{R}\times\hat{\mathbf{s}})\cdot\hat{\xi}'}{R\,(R+\mathbf{R}\cdot\hat{\mathbf{s}})}\;dL'-\frac{1}{8\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R}\left\{\;(1-2\nu)\,\mathbf{b}\times\hat{\xi}'+\left[\hat{\mathbf{R}}\cdot\left(\mathbf{b}\times\hat{\xi}'\right)\right]\hat{\mathbf{R}}\right\}\mathrm{d}L'\\ &= \mathbf{b}^f(\mathbf{x})-\frac{1}{8\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R}\left\{\;\frac{2(1-\nu)(\hat{\mathbf{R}}\times\hat{\mathbf{s}})\cdot\hat{\xi}'}{1+\hat{\mathbf{R}}\cdot\hat{\mathbf{s}}}\mathbf{b}+(1-2\nu)\,\mathbf{b}\times\hat{\xi}'+\left[\hat{\mathbf{R}}\cdot\left(\mathbf{b}\times\hat{\xi}'\right)\right]\hat{\mathbf{R}}\right\}\mathrm{d}L'\\ &= \mathbf{b}^f(\mathbf{x})+\frac{1}{8\pi(1-\nu)}\oint_{\mathcal{L}}\frac{1}{R}\left\{\;\frac{2(1-\nu)(\hat{\mathbf{s}}\times\hat{\mathbf{R}})\cdot\hat{\xi}'}{1+\hat{\mathbf{R}}\cdot\hat{\mathbf{s}}}\mathbf{b}+(1-2\nu)\,\hat{\xi}'\times\mathbf{b}+\left[\hat{\mathbf{R}}\cdot\left(\hat{\xi}'\times\mathbf{b}\right)\right]\hat{\mathbf{R}}\right\}\mathrm{d}L'\end{aligned} \tag{A.64}$$

Notice that

$$\begin{aligned}\mathbf{b}^f(\mathbf{x}) &= \frac{\mathbf{b}\Omega^f}{4\pi}=\frac{\mathbf{b}}{4\pi}\int_{\mathcal{S}}v_k^f(\mathbf{R})\;dA'_k=\frac{\mathbf{b}}{4\pi}\int_{\mathcal{S}}4\pi\int_{\mathcal{D}}\delta(\mathbf{R}+\mathbf{s})ds_k\;dA'_k\\ &= \begin{cases}+\mathbf{b} & \text{if the Dirac string crosses the slip surface positively}\\ -\mathbf{b} & \text{if the Dirac string crosses the slip surface negatively}\\ \mathbf{0} & \text{if the Dirac string does not cross the slip surface}\end{cases}\end{aligned} \tag{A.65}$$

A.2.4 Multipole expansion

Stress field

$$\sigma_{ij}=\oint_{\mathcal{L}}S_{ijkl}R(\mathbf{x}-\mathbf{x}')b'_k\xi'_l\;dL' \tag{A.66}$$

Now let $\mathbf{x}'=\mathbf{x}^c-\tilde{\mathbf{x}}$, then

$$\sigma_{ij}=\oint_{\mathcal{L}}S_{ijkl}R(\mathbf{x}-\mathbf{x}^c+\tilde{\mathbf{x}})b'_k\xi'_l\;dL' \tag{A.67}$$

Now assume

$$|\tilde{\mathbf{x}}|\ll|\mathbf{x}-\mathbf{x}^c| \tag{A.68}$$

then, expanding R for $\tilde{\mathbf{x}}\to 0$ yields

$$R(\mathbf{x}-\mathbf{x}^c+\tilde{\mathbf{x}})\approx R(\mathbf{x}-\mathbf{x}^c)+\partial_pR(\mathbf{x}-\mathbf{x}^c)\tilde{x}_p+\dots \tag{A.69}$$

and

$$\sigma_{ij}=S_{ijkl}R(\mathbf{x}-\mathbf{x}^c)\oint_{\mathcal{L}}b'_k\xi'_l\;dL'+S_{ijkl}\partial_pR(\mathbf{x}-\mathbf{x}^c)\oint_{\mathcal{L}}\tilde{x}_pb'_k\xi'_l+\dots \tag{A.70}$$

First order expansion:

$$\begin{aligned}\boldsymbol{\sigma}^0(\boldsymbol{x}) = \frac{\mu}{4\pi(1-\nu)R^2}&\left\{(1-\nu)\left[\hat{\boldsymbol{\xi}}'\otimes(\boldsymbol{b}\times\hat{\boldsymbol{R}})+(\boldsymbol{b}\times\hat{\boldsymbol{R}})\otimes\hat{\boldsymbol{\xi}}'\right]+\left[(\hat{\boldsymbol{\xi}}'\times\boldsymbol{b})\otimes\hat{\boldsymbol{R}}+\hat{\boldsymbol{R}}\otimes(\hat{\boldsymbol{\xi}}'\times\boldsymbol{b})\right]\right.\\ &\left.+\hat{\boldsymbol{R}}\cdot(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}')\left[3\hat{\boldsymbol{R}}\otimes\hat{\boldsymbol{R}}+\boldsymbol{I}\right]\right\}\end{aligned} \tag{A.71}$$

Letting

$$(\boldsymbol{b}\times\hat{\boldsymbol{R}})\otimes\hat{\boldsymbol{\xi}})_{ij}=\epsilon_{ikm}b_kR_m\xi_j=\epsilon_{ikm}R_m\alpha_{kj}=\mathbf{S}\cdot\alpha \tag{A.72}$$

$$\mathbf{S} = \begin{bmatrix} 0 & R_3 & -R_2 \\ -R_3 & 0 & R_1 \\ R_2 & -R_1 & 0 \end{bmatrix} \tag{A.73}$$

$$\boldsymbol{b} \times \hat{\boldsymbol{\xi}}' = \epsilon_{ikm} b_k \xi_m = a_i \tag{A.74}$$

$$\mathbf{a} = \begin{bmatrix} \alpha_{23} - \alpha_{32} \\ \alpha_{31} - \alpha_{13} \\ \alpha_{12} - \alpha_{21} \end{bmatrix} \tag{A.75}$$

we obtain

$$\boldsymbol{\sigma}^0(\boldsymbol{x}) = \frac{\mu}{4\pi(1-\nu)R^2} \Big\{ (1-\nu) \left[(\mathbf{S}\cdot\alpha)^T + \mathbf{S}\cdot\alpha \right] - \left[\mathbf{a} \otimes \hat{\boldsymbol{R}} + \hat{\boldsymbol{R}} \otimes \mathbf{a} \right] + (\hat{\boldsymbol{R}} \cdot \boldsymbol{a}) \left[3\hat{\boldsymbol{R}} \otimes \hat{\boldsymbol{R}} + \boldsymbol{I} \right] \Big\} \tag{A.76}$$

Displacement field

$$\mathbf{u}(\mathbf{x}) = \mathbf{b}^f(\mathbf{x}) + \frac{1}{8\pi(1-\nu)} \frac{1}{R} \left\{ \boldsymbol{\alpha} \cdot \frac{2(1-\nu)(\hat{\mathbf{s}} \times \hat{\mathbf{R}})}{1 + \hat{\mathbf{R}} \cdot \hat{\mathbf{s}}} - (1-2\nu)\, \boldsymbol{a} - \left(\hat{\mathbf{R}} \cdot \boldsymbol{a} \right) \hat{\mathbf{R}} \right\} \tag{A.77}$$

Interaction Energy

$$E_I = -\frac{\mu}{4\pi(1-\nu)} \oint_{\mathcal{L}_2} \oint_{\mathcal{L}_1} \frac{1}{R} \Big\{ \Big[(1-\nu) \left(\boldsymbol{b}_1 \cdot \hat{\boldsymbol{\xi}}_1 \right) \left(\boldsymbol{b}_2 \cdot \hat{\boldsymbol{\xi}}_2 \right) + 2\nu \left(\boldsymbol{b}_1 \cdot \hat{\boldsymbol{\xi}}_2 \right) \left(\boldsymbol{b}_2 \cdot \hat{\boldsymbol{\xi}}_1 \right) - 2 \left(\boldsymbol{b}_1 \cdot \boldsymbol{b}_2 \right) \left(\hat{\boldsymbol{\xi}}_1 \cdot \hat{\boldsymbol{\xi}}_2 \right) \Big] + \left(\hat{\boldsymbol{\xi}}_1 \cdot \hat{\boldsymbol{\xi}}_2 \right) \Big[\left(\boldsymbol{b}_1 \cdot \boldsymbol{b}_2 \right) - \left(\boldsymbol{b}_1 \cdot \hat{\boldsymbol{R}} \right) \left(\boldsymbol{b}_2 \cdot \hat{\boldsymbol{R}} \right) \Big] \Big\} dL_1 dL_2 \tag{A.78}$$

$$E_I = -\frac{\mu}{4\pi(1-\nu)} \oint_{\mathcal{L}_2} \oint_{\mathcal{L}_1} \frac{1}{R} \Big\{ (1-\nu) \left(\boldsymbol{b}_1 \cdot \hat{\boldsymbol{\xi}}_1 \right) \left(\boldsymbol{b}_2 \cdot \hat{\boldsymbol{\xi}}_2 \right) + 2\nu \left(\boldsymbol{b}_1 \cdot \hat{\boldsymbol{\xi}}_2 \right) \left(\boldsymbol{b}_2 \cdot \hat{\boldsymbol{\xi}}_1 \right) - \left(\boldsymbol{b}_1 \cdot \boldsymbol{b}_2 \right) \left(\hat{\boldsymbol{\xi}}_1 \cdot \hat{\boldsymbol{\xi}}_2 \right) - \left(\hat{\boldsymbol{\xi}}_1 \cdot \hat{\boldsymbol{\xi}}_2 \right) \left(\boldsymbol{b}_1 \cdot \hat{\boldsymbol{R}} \right) \left(\boldsymbol{b}_2 \cdot \hat{\boldsymbol{R}} \right) \Big\} dL_1 dL_2 \tag{A.79}$$

A.2.5 Plastic strain rate

For conservative motion (glide) $\boldsymbol{v} \cdot \hat{\boldsymbol{n}} = 0$, therefore, from (D.4):

$$\dot{\beta}^p_{ij}(\boldsymbol{x}) = -b_i \int_{\boldsymbol{A}}^{\boldsymbol{B}} \delta(\boldsymbol{x} - \boldsymbol{x}') \epsilon_{jkm} v_k dl'_m \tag{A.80}$$

A.2.6 Discrete dislocations in finite media

From a numerical viewpoint, it is interesting to ask; what is the distribution of β^P_{ij} outside $\mathcal{B}$ in (A.36)? We shall call this quantity β^{P*}_{ij}. In fact, although the solution to the problems (A.36) and (A.37) depend individually on β^{P*}_{ij}, it is easy to see that the total solution is independent of β^{P*}_{ij}. Therefore β^{P*}_{ij} can be chosen arbitrarily[3]. The arbitrariness of β^{P*}_{ij} can be exploited in the numerical implementation. To see how, we can cast (A.37) in its corresponding Galerkin weak form

$$\int_{\mathcal{B}} c_{ijkl} u^c_{k,l} \tilde{u}_{i,j} d\mathcal{V} = \int_{\partial_N \mathcal{B}} \left(p_i - \sigma^\infty_{ij} \hat{n}_j \right) \tilde{u}_i d\mathcal{A} \tag{A.81}$$

[3]We observe that the arbitrariness of β^{P*}_{ij} implies that also its bounding curve (or virtual dislocation line) is arbitrary, in agreement with the "mirror image construction" of Weygand et al. [2002] and the "independence of the total stress on the choice of virtual segments" mentioned in Weinberger et al. [2009].

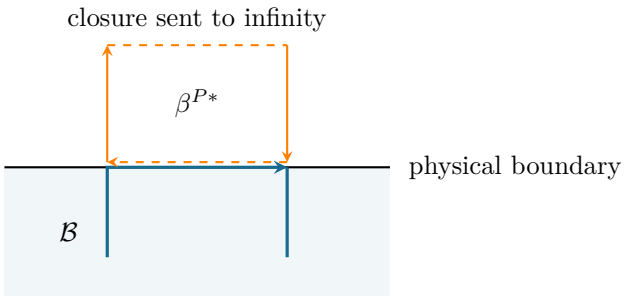


Figure A.2: Schematic external-loop construction. Oppositely oriented overlapping boundary segments cancel; the external closure is taken to infinity. The drawing shows connectivity, not a finite computational truncation prescription.

and notice that the RHS involves a surface integral of the stress field σ_{ij}^{∞} induced by the dislocation network. When a dislocation segment reaches the domain boundary, the numerical evaluation of this integral becomes challenging. This numerical difficulty can be overcome using the external-loop construction in Fig. A.2. In fact, because β_{ij}^{P*} is arbitrary, it is possible to add an external loop with the same Burgers vector of the boundary segment and opposite line direction, constructed so that one side of the loop overlaps with the boundary segment, and the opposite side is pushed to infinity along the direction of the boundary normal vector. In this way, because the contributions of the overlapping segments cancel out, the line integral involved in the calculation of σ_{ij}^{∞} can be limited to the portion of the dislocation configuration strictly inside the domain, and the virtual straight dislocation lines extending from the domain boundary to infinity. Notice that this construction unambiguously defines β_{ij}^{P*} and the surface over which it extends, a fundamental requirement for the calculation of u_i^{∞} needed to satisfy displacement boundary conditions in the BVP (A.37).

A.2.7 Force vector of a triangular element due to a straight segment

We would like to compute the force vector of a triangular boundary element due to a straight segment. The weak form involved in the FEM problem is

$$\int_{\mathcal{T}} \tilde{u}_i t_i \mathrm{d}A = \sum_n \tilde{u}_i^n \int_{\mathcal{T}} N^n \sigma_{ij} \hat{n}_j \mathrm{d}A \tag{A.82}$$

where N^n is the shape function of the n-th node of the triangle, and $\tilde{u}_i^n$ its test displacement. The stress field generated by the dislocaiton loops is

$$\sigma_{ij}(\boldsymbol{x}) = \tilde{S}_{ijl}\left[q_l(\boldsymbol{x})\right] \tag{A.83}$$

where

$$q_l(\boldsymbol{x}) = \oint_{\mathcal{L}} R_a(\boldsymbol{x} - \boldsymbol{x}')\, d\ell_l' \tag{A.84}$$

and $\tilde{S}$ is the the third-order dislocation stress differential operator, which is defined as

$$\tilde{S}_{ijl}[\cdot] = \frac{\mu b_k}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk} + \delta_{jl}\epsilon_{imk}\right)\partial_m \Delta + \frac{2}{1-\nu}\epsilon_{klm}\left(\partial_i\partial_j\partial_m - \delta_{ij}\partial_m\Delta\right)\right] \tag{A.85}$$

Status of the inherited derivation

The following operator manipulation was highlighted as working material in the original notes. It is retained for review, not promoted to a verified implementation formula.

Note that this operator can be rewritten as

$$\tilde{S}_{ijl}[\cdot] = \partial_m \hat{S}_{ijlm}[\cdot] \tag{A.86}$$

where the auxiliary second-order differential operator $\hat{S}_{ijlm}[\cdot]$ is

$$\tilde{S}_{ijlm}[\cdot] = \frac{\mu b_k}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk}+\delta_{jl}\epsilon_{imk}\right)\Delta + \frac{2}{1-\nu}\epsilon_{klm}\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \tag{A.87}$$

Let us focus on the integral

$$\begin{aligned} F_i^{(n)} &= \int_{\mathcal{T}} N^{(n)}\sigma_{ij}\mathrm{d}A_j = \int_{\mathcal{T}} N^n\partial_m\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}A_j \\ &= \int_{\mathcal{T}}\partial_m\left\{N^n\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\right\}\mathrm{d}A_j - \int_{\mathcal{T}} N^n_{,m}\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}A_j \end{aligned} \tag{A.88}$$

We now apply Stokes theorem (D.2) to the first term of (A.88) to find

$$\begin{aligned} F_i^{(n)} &= \oint_{\partial\mathcal{T}}\epsilon_{kjm}N^{(n)}\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}\ell_k + \int_{\mathcal{T}}\partial_j\left\{N^n\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\right\}\mathrm{d}A_m - \int_{\mathcal{T}} N^n_{,m}\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}A_j \\ &= \oint_{\partial\mathcal{T}}\epsilon_{kjm}N^{(n)}\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}\ell_k + \int_{\mathcal{T}} N^n\partial_j\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}A_m \\ &+ \int_{\mathcal{T}} N^n_{,j}\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}A_m - \int_{\mathcal{T}} N^n_{,m}\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right]\,\mathrm{d}A_j \end{aligned} \tag{A.89}$$

Note that in the expression above, the only third-order differential operator is $\partial_j\hat{S}_{ijlm}$. However a closer inspection reveals that

$$\begin{aligned} \partial_j\hat{S}_{ijlm}\left[q_l(\boldsymbol{x})\right] &= \frac{\mu b_k}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk}+\delta_{jl}\epsilon_{imk}\right)\partial_j\Delta + \frac{2}{1-\nu}\epsilon_{klm}\cancel{\left(\partial_i\partial_j\partial_j-\delta_{ij}\partial_j\Delta\right)}\right]\left[q_l(\boldsymbol{x})\right] \\ &= \frac{\mu b_k}{8\pi}\left(\epsilon_{jmk}\Delta\partial_j\left[q_i(\boldsymbol{x})\right] + \epsilon_{imk}\Delta\left[\oint_{\mathcal{L}}\frac{\partial}{\partial x_l}R_a(\boldsymbol{x}-\boldsymbol{x}')\,d\ell'_l\right]\right) \\ &= \frac{\mu b_k}{8\pi}\left(\epsilon_{jmk}\Delta\partial_j\left[q_i(\boldsymbol{x})\right] - \epsilon_{imk}\Delta\left[\cancel{\oint_{\mathcal{L}}\frac{\partial}{\partial x'_l}R_a(\boldsymbol{x}-\boldsymbol{x}')\,d\ell'_l}\right]\right) \end{aligned}$$

The first term is zero if the Burgers vector is aligned with the surface normal (not very useful). Moreover

$$\begin{aligned} \epsilon_{kjm}\tilde{S}_{ijlm} &= \frac{\mu b_p}{8\pi}\left[\left(\delta_{il}\epsilon_{kjm}\epsilon_{jmp}+\delta_{jl}\epsilon_{mkj}\epsilon_{mpi}\right)\Delta + \frac{2}{1-\nu}\epsilon_{mkj}\epsilon_{mpl}\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \\ &= \frac{\mu b_p}{8\pi}\left[\left(\delta_{il}2\delta_{kp}+\delta_{jl}\left(\delta_{kp}\delta_{ij}-\delta_{ik}\delta_{jp}\right)\right)\Delta + \frac{2}{1-\nu}\left(\delta_{kp}\delta_{jl}-\delta_{kl}\delta_{jp}\right)\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \\ &= \frac{\mu b_p}{8\pi}\left[\left(2\delta_{il}\delta_{kp}+\delta_{kp}\delta_{il}-\delta_{ik}\delta_{lp}\right)\Delta + \frac{2}{1-\nu}\left(\delta_{kp}\delta_{jl}-\delta_{kl}\delta_{jp}\right)\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \end{aligned} \tag{A.90}$$

Moreover

$$\begin{aligned} \epsilon_{kjm}\tilde{S}_{ijlm}[\cdot] &= \frac{\mu b_p}{8\pi}\left[\left(\delta_{il}\epsilon_{kjm}\epsilon_{jmp}+\delta_{jl}\epsilon_{kjm}\epsilon_{imp}\right)\Delta + \frac{2}{1-\nu}\epsilon_{kjm}\epsilon_{plm}\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \\ &= \frac{\mu b_p}{8\pi}\left[\left(2\delta_{il}\delta_{pk}+\delta_{jl}(\delta_{kp}\delta_{ji}-\delta_{ki}\delta_{jp})\right)\Delta + \frac{2}{1-\nu}(\delta_{kp}\delta_{jl}-\delta_{kl}\delta_{jp})\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \\ &= \frac{\mu b_p}{8\pi}\left[\left(2\delta_{il}\delta_{pk}+\delta_{kp}\delta_{li}-\delta_{ki}\delta_{lp}\right)\Delta + \frac{2}{1-\nu}(\delta_{kp}\delta_{jl}-\delta_{kl}\delta_{jp})\left(\partial_i\partial_j-\delta_{ij}\Delta\right)\right] \end{aligned} \tag{A.91}$$

A.2.8 Discrete dislocations in periodic media

The translation lattice is described in Section 1.8; finite-image controls and periodic glide-plane geometry are described in Section 8.3. The original notes supplied only this heading; a complete periodic elastic-sum derivation remains to be written.

Appendix B

DDD: regularization of singular fields

Status of the inherited derivation

These finite-core expressions were previously outside the compiled manual. They are retained as working theory; the index notation and equivalence of the component and dyadic forms still require a dedicated mathematical review. The inherited Levi–Civita notation is standardized to ϵ_{ijk}.

The regularized distance is $R_a = \sqrt{\boldsymbol{R}\cdot\boldsymbol{R} + a^2}$. The plot below illustrates the smoothing of this distance.

B.1 Stress kernel

$$\sigma_{ij}^{\mathrm{ns}}(\boldsymbol{F}) = \frac{\mu b_n}{8\pi}\int_{\mathcal{S}}\left[R_{a,mpp}\left(\epsilon_{jmn}\xi_i + \epsilon_{imn}\xi_j\right) + \frac{2}{1-\nu}\epsilon_{kmn}\left(R_{a,ijm} - \delta_{ij}R_{a,ppm}\right)\xi_k\right]dl^s \tag{B.1}$$

where $\boldsymbol{R} = \boldsymbol{F} - \boldsymbol{S}$ is the vector connecting the source point to the field point and $\xi = \frac{d\boldsymbol{S}}{dl^s}$ s the unit tangent along the source dislocation.

Substituting

$$R_{,ijk} = -\frac{\delta_{ij}R_k + \delta_{jk}R_i + \delta_{ki}R_j}{R_a^3} + \frac{3R_iR_jR_k}{R_a^5} \qquad R_{a,ipp} = -\left(2 + \frac{3a^2}{R_a^2}\right)\frac{R_i}{R_a^3} \tag{B.2}$$

we find

$$\begin{aligned}\sigma_{ij}(\boldsymbol{F}) = \frac{\mu b_n}{4\pi}\int_{\mathcal{S}}\Bigg[&-\left(1+\frac{3a^2}{2R_a^2}\right)\frac{R_m}{R_a^3}\left(\epsilon_{jmn}\xi_i + \epsilon_{imn}\xi_j\right)\\ &+\frac{1}{1-\nu}\epsilon_{kmn}\left(\frac{\delta_{ij}\left(1+\frac{3a^2}{R_a^2}\right)R_m - \delta_{jm}R_i - \delta_{mi}R_j}{R_a^3} + \frac{3R_iR_jR_m}{R_a^5}\right)\xi_k\Bigg]dl^s\end{aligned} \tag{B.3}$$

In dyadic form

$$\begin{aligned}\frac{d\boldsymbol{\sigma}}{dL} = \frac{\mu}{4\pi(1-\nu)R_a^2}\Bigg\{&(1-\nu)\left(1+\frac{3a^2}{2R_a^2}\right)\left(\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\tilde{\boldsymbol{R}}) + (\boldsymbol{b}\times\tilde{\boldsymbol{R}})\otimes\hat{\boldsymbol{\xi}}\right)\\ &+\left(\tilde{\boldsymbol{R}}\otimes(\hat{\boldsymbol{\xi}}\times\boldsymbol{b}) + (\hat{\boldsymbol{\xi}}\times\boldsymbol{b})\otimes\tilde{\boldsymbol{R}}\right)\Bigg\}\\ &+[(\tilde{\boldsymbol{R}}\times\boldsymbol{b})\cdot\hat{\boldsymbol{\xi}}]\left[\left(1+\frac{3a^2}{R_a^2}\right)\boldsymbol{I} + 3\tilde{\boldsymbol{R}}\otimes\tilde{\boldsymbol{R}}\right]\Bigg\}\end{aligned} \tag{B.4}$$

where $\tilde{\boldsymbol{R}} = \boldsymbol{R}/R_a$

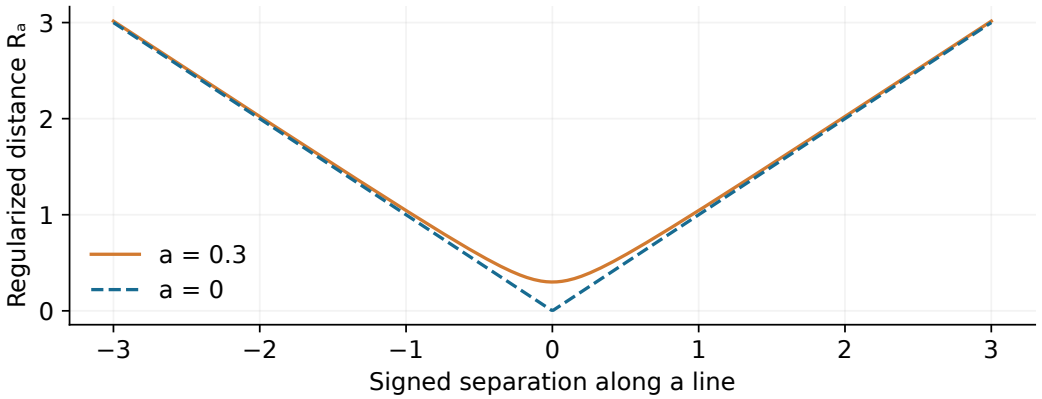


Figure B.1: Analytical distance kernel $R_a = \sqrt{x^2 + a^2}$ for two core radii. This illustrates the smoothing operation, not the full dislocation stress or a simulated core structure.

Appendix C

DDD: fields of piecewise-straight loops

C.1 Straight segments

For a straight segment starting at $\boldsymbol{A}$ and ending at $\boldsymbol{B}$, it is useful to preliminary compute the function

$$q(\boldsymbol{x}) = \int_{\boldsymbol{A}}^{\boldsymbol{B}} R_a \, d\ell' \tag{C.1}$$

where $R_a = \sqrt{\boldsymbol{R}\cdot\boldsymbol{R} + a^2}$, $\boldsymbol{R} = \boldsymbol{x} - \boldsymbol{x}'$. Integration is performed on the primed variable along the segment $\boldsymbol{AB}$. This can be parametrized as a function of a scalar variable u as

$$\boldsymbol{x}'(u) = \boldsymbol{A}(1-u) + \boldsymbol{B}u = \boldsymbol{A} + (\boldsymbol{B}-\boldsymbol{A})u \qquad u \in [0,1] \tag{C.2}$$

Therefore

$$\begin{aligned} q(\boldsymbol{x}) &= L\int_0^1 \sqrt{[(\boldsymbol{A}-\boldsymbol{x}) + (\boldsymbol{B}-\boldsymbol{A})u]^2 + a^2}\, du \\ &= L\int_0^1 \sqrt{(\boldsymbol{B}-\boldsymbol{A})^2u^2 + 2(\boldsymbol{A}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A})u + (\boldsymbol{A}-\boldsymbol{x})^2 + a^2}\, du \end{aligned} \tag{C.3}$$

Now

$$\int \sqrt{du^2+bu+c}\, du = \left(\frac{b}{4d} + \frac{u}{2}\right)\sqrt{du^2+bu+c} + \frac{4dc-b^2}{8d^{\frac{3}{2}}}\ln\left(\frac{2du+b}{\sqrt{d}} + 2\sqrt{du^2+bu+c}\right) \tag{C.4}$$

So

$$\int_0^1 \sqrt{du^2+bu+c}\, du = \frac{2d+b}{4d}\sqrt{d+b+c} - \frac{b}{4d}\sqrt{c} + \frac{4dc-b^2}{8d^{\frac{3}{2}}}\ln\left(\frac{\frac{2d+b}{\sqrt{d}} + 2\sqrt{d+b+c}}{\frac{b}{\sqrt{d}} + 2\sqrt{c}}\right) \tag{C.5}$$

Note that in our case

$$d = (\boldsymbol{B}-\boldsymbol{A})^2 = L^2 \tag{C.6}$$
$$b = 2(\boldsymbol{A}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A}) = 2L(\boldsymbol{A}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}} \tag{C.7}$$
$$c = (\boldsymbol{A}-\boldsymbol{x})^2 + a^2 \tag{C.8}$$
$$d+b+c = (\boldsymbol{B}-\boldsymbol{x})^2 + a^2 \tag{C.9}$$
$$2d+b = 2(\boldsymbol{B}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A}) = 2L(\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}} \tag{C.10}$$
$$\begin{aligned}
4dc-b^2 &= 4(\boldsymbol{B}-\boldsymbol{A})^2(\boldsymbol{A}-\boldsymbol{x})^2 - [2(\boldsymbol{A}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A})]^2 + 4da^2\\
&= 4L^2[(\boldsymbol{A}-\boldsymbol{x})^2 - ((\boldsymbol{A}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2]\\
&= 4L^2[(\boldsymbol{A}-\boldsymbol{B}+\boldsymbol{B}-\boldsymbol{x})^2 - ((\boldsymbol{A}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2]\\
&= 4L^2[(\boldsymbol{A}-\boldsymbol{B})^2 + 2(\boldsymbol{A}-\boldsymbol{B})\cdot(\boldsymbol{B}-\boldsymbol{x}) + (\boldsymbol{B}-\boldsymbol{x})^2 - ((\boldsymbol{A}-\boldsymbol{B})\cdot\hat{\boldsymbol{\xi}} + (\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2]\\
&= 4L^2[L^2 - 2L\hat{\boldsymbol{\xi}}\cdot(\boldsymbol{B}-\boldsymbol{x}) + (\boldsymbol{B}-\boldsymbol{x})^2 - (-L + (\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2]\\
&= 4L^2[L^2 - 2L\hat{\boldsymbol{\xi}}\cdot(\boldsymbol{B}-\boldsymbol{x}) + (\boldsymbol{B}-\boldsymbol{x})^2 - L^2 + 2L(\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}} - ((\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2]\\
&= 4L^2[(\boldsymbol{B}-\boldsymbol{x})^2 - ((\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2]\\
&= 4L^2\rho_a^2
\end{aligned} \tag{C.11}$$

where

$$\rho_a^2 = (\boldsymbol{B}-\boldsymbol{x})^2 - ((\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2 = (\boldsymbol{A}-\boldsymbol{x})^2 - ((\boldsymbol{A}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}})^2 + a^2 \tag{C.12}$$

ρ^2 is the regularized distance of $\boldsymbol{x}$ from the line passing through $\boldsymbol{A}$ and $\boldsymbol{B}$.
Therefore

$$\begin{aligned}
\int_0^1 \sqrt{du^2+bu+c}\,du =& \frac{(\boldsymbol{B}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A})}{2(\boldsymbol{B}-\boldsymbol{A})^2}\sqrt{(\boldsymbol{B}-\boldsymbol{x})^2+a^2} - \frac{(\boldsymbol{A}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A})}{2(\boldsymbol{B}-\boldsymbol{A})^2}\sqrt{(\boldsymbol{A}-\boldsymbol{x})^2+a^2}\\
&+ \frac{4dc-b^2}{8L^3}\ln\left(\frac{\frac{2(\boldsymbol{B}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A})}{\sqrt{d}} + 2\sqrt{(\boldsymbol{B}-\boldsymbol{x})^2+a^2}}{\frac{2(\boldsymbol{A}-\boldsymbol{x})\cdot(\boldsymbol{B}-\boldsymbol{A})}{\sqrt{d}} + 2\sqrt{(\boldsymbol{A}-\boldsymbol{x})^2+a^2}}\right)\\
=& \frac{(\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}}}{2L}\sqrt{(\boldsymbol{B}-\boldsymbol{x})^2+a^2} - \frac{(\boldsymbol{A}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}}}{2L}\sqrt{(\boldsymbol{A}-\boldsymbol{x})^2+a^2}\\
&+ \frac{\rho_a^2}{2L}\ln\left(\frac{(\boldsymbol{B}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}} + \sqrt{(\boldsymbol{B}-\boldsymbol{x})^2+a^2}}{(\boldsymbol{A}-\boldsymbol{x})\cdot\hat{\boldsymbol{\xi}} + \sqrt{(\boldsymbol{A}-\boldsymbol{x})^2+a^2}}\right)
\end{aligned} \tag{C.13}$$

If we define

$$\begin{aligned}
f(\boldsymbol{z}) &= \frac{\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}}}{2}z_a + \frac{\rho_a^2}{2}\ln\left(\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}} + z_a\right)\\
&= \frac{\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}}}{2}z_a + \frac{\rho_a^2}{2}\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right)
\end{aligned} \tag{C.14}$$

and

$$\boldsymbol{Y}_a = \boldsymbol{z} + z_a\hat{\boldsymbol{\xi}} \tag{C.15}$$
$$\rho_a^2 = \boldsymbol{\rho}_a\cdot\boldsymbol{\rho}_a = z^2 - (\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}})^2 + a^2 \tag{C.16}$$
$$\boldsymbol{\rho}_a = \boldsymbol{z} - (\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}} \pm a)\hat{\boldsymbol{\xi}} \tag{C.17}$$

then we have

$$q(\boldsymbol{x}) = f(\boldsymbol{B}-\boldsymbol{x}) - f(\boldsymbol{A}-\boldsymbol{x}) \tag{C.18}$$

Now consider the following identities:

$$\boldsymbol{\rho}_a \cdot \hat{\boldsymbol{\xi}} = \mp a \tag{C.19}$$

$$\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}} = \boldsymbol{z} \cdot \hat{\boldsymbol{\xi}} + z_a \tag{C.20}$$

$$Y_a^2 = 2z_a \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}} - a^2 \tag{C.21}$$

$$\frac{\partial \rho_{ai}}{\partial z_j} = \delta_{ij} - \hat{\xi}_i \hat{\xi}_j \tag{C.22}$$

$$\frac{\partial \rho_a^2}{\partial z_i} = 2\rho_{ak}\frac{\partial \rho_{ak}}{\partial z_i} = 2\rho_{ak}\left(\delta_{ki} - \hat{\xi}_i\hat{\xi}_k\right) = 2(\underbrace{z_i - (\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}})\hat{\xi}_i}_{\rho_i}) = 2\rho_i \tag{C.23}$$

$$\frac{\partial \rho_i}{\partial z_j} = \delta_{ij} - \hat{\xi}_i \hat{\xi}_j \tag{C.24}$$

$$\frac{\partial Y_{ai}}{\partial z_j} = \delta_{ij} + \frac{z_j \hat{\xi}_i}{z_a} \tag{C.25}$$

$$\frac{\partial (\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})}{\partial z_i} = \frac{Y_{ai}}{z_a} \tag{C.26}$$

$$\frac{\partial}{\partial z_i}\ln(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}) = \frac{Y_{ai}}{z_a \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} \tag{C.27}$$

First derivative

$$\begin{aligned}
\frac{\partial f}{\partial z_i} &= \frac{z_a\hat{\xi}_i}{2} + \frac{z_p\hat{\xi}_p z_i}{2z_a} + \rho_i \ln(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}) + \frac{\rho_a^2}{2}\frac{Y_{ai}}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}} \\
&= \frac{z_a\hat{\xi}_i}{2} + \frac{z_p\hat{\xi}_p z_i}{2z_a} + \rho_i \ln(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}) + \frac{(z^2 - (z_q\hat{\xi}_q)^2 + a^2)}{2}\frac{Y_{ai}}{z_a(z_a + z_p\hat{\xi}_p)} \\
&= \frac{z_a\hat{\xi}_i}{2} + \frac{z_p\hat{\xi}_p z_i}{2z_a} + \rho_i \ln(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}) + \frac{(z_a - z_q\hat{\xi}_q)}{2}\frac{Y_{ai}}{z_a} \\
&= \frac{z_a^2\hat{\xi}_i + z_p\hat{\xi}_p z_i + (z_a - z_q\hat{\xi}_q)(z_i + z_a\hat{\xi}_i)}{2z_a} + \rho_i \ln(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}) \\
&= \frac{\rho_i}{2} + z_a\hat{\xi}_i + \rho_i \ln(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})
\end{aligned} \tag{C.28}$$

Second derivative

$$\begin{aligned}
\frac{\partial^2 f}{\partial z_i \partial z_j} &= \frac{\rho_{i,j}}{2} + \frac{z_j}{z_a}\hat{\xi}_i + \rho_{i,j}\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{\rho_i Y_{aj}}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}} \\
&= \frac{\delta_{ij} - \hat{\xi}_i\hat{\xi}_j}{2} + \frac{z_j}{z_a}\hat{\xi}_i + (\delta_{ij} - \hat{\xi}_i\hat{\xi}_j)\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{\rho_i Y_{aj}}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}} \\
&= \frac{\delta_{ij} - \hat{\xi}_i\hat{\xi}_j}{2} + (\delta_{ij} - \hat{\xi}_i\hat{\xi}_j)\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{\rho_i Y_{aj} + z_j\hat{\xi}_i\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}} \\
&= \frac{\delta_{ij} - \hat{\xi}_i\hat{\xi}_j}{2} + (\delta_{ij} - \hat{\xi}_i\hat{\xi}_j)\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{1}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\left(\frac{z_i z_j}{z_a} + z_i\hat{\xi}_j + z_j\hat{\xi}_i - z_q\hat{\xi}_q\hat{\xi}_i\hat{\xi}_j\right)
\end{aligned} \tag{C.29}$$

$$\begin{aligned}
\frac{\partial^2 f}{\partial z_p \partial z_p} &= 1 + 2\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{1}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\left(\frac{z^2}{z_a} + \boldsymbol{z}\cdot\hat{\boldsymbol{\xi}}\right) \\
&= 1 + 2\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{1}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\left(z_a + \boldsymbol{z}\cdot\hat{\boldsymbol{\xi}} - \frac{a^2}{z_a}\right) \\
&= 2 + 2\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) - \frac{a^2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}
\end{aligned} \tag{C.30}$$

Third derivative

$$\frac{\partial^3 f}{\partial z_i \partial z_j z_m} = \frac{\delta_{jm} Y_{ai} + \delta_{im} Y_{aj} + \delta_{ij} Y_{am}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})} \tag{C.31}$$

$$+ \frac{1}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} \left(\frac{\boldsymbol{z} \cdot \hat{\boldsymbol{\xi}}}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} - 2 \right) \hat{\xi}_i \hat{\xi}_j \hat{\xi}_m \tag{C.32}$$

$$- \frac{1}{z_a^2 \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} \left(\frac{1}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} + \frac{1}{z_a} \right) z_i z_j z_m \tag{C.33}$$

$$- \frac{\hat{\xi}_i z_m Y_{aj} + \hat{\xi}_j z_i Y_{am} + \hat{\xi}_m z_j Y_{ai}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} \tag{C.34}$$

$$\tag{C.35}$$

Letting $i = j$

$$\frac{\partial^3 f}{\partial z_p \partial z_p z_m} = \frac{5 Y_{am}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})} \tag{C.36}$$

$$+ \frac{1}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} \left(\frac{\boldsymbol{z} \cdot \hat{\boldsymbol{\xi}}}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} + \frac{a^2}{z_a \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} - 2 \right) \hat{\xi}_m \tag{C.37}$$

$$- \frac{1}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} \left(\frac{1}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} + \frac{1}{z_a} \right) \left(1 - \frac{a^2}{z_a^2} \right) z_m \tag{C.38}$$

$$- \frac{z_a + 2(\boldsymbol{z} \cdot \hat{\boldsymbol{\xi}})}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} Y_{am}$$

$$= \frac{5 Y_{am}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})} \tag{C.39}$$

$$- \frac{z^2 + z_a \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} \hat{\xi}_m \tag{C.40}$$

$$- \frac{z_a + \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} \left(1 - \frac{a^2}{z_a^2} \right) z_m \tag{C.41}$$

$$- \frac{z_a + 2(\boldsymbol{z} \cdot \hat{\boldsymbol{\xi}})}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} Y_{am} \tag{C.42}$$

$$= \frac{5 Y_{am}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})} \tag{C.43}$$

$$- \frac{z^2 + z_a \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}}{z_a^2(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} Y_{am} + \frac{a^2}{z_a^3 \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} z_m$$

$$- \frac{z_a + 2(\boldsymbol{z} \cdot \hat{\boldsymbol{\xi}})}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} Y_{am}$$

$$= \left(\frac{2}{z_a \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} + \frac{a^2}{z_a^2(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} \right) Y_{am} + \frac{a^2}{z_a^3 \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} z_m \tag{C.44}$$

Since $\epsilon_{klm} \hat{\xi}_l \hat{\xi}_m = 0$, then $\epsilon_{klm} \hat{\xi}_l Y_m = \epsilon_{klm} \hat{\xi}_l Y_{am} = \epsilon_{klm} \hat{\xi}_l \rho_m = \epsilon_{klm} \hat{\xi}_l z_m$. Therefore

$$\epsilon_{klm} \hat{\xi}_l \frac{\partial^3 f}{\partial z_i \partial z_j z_m} = \frac{\epsilon_{klj} \hat{\xi}_l Y_{ai} + \epsilon_{kli} \hat{\xi}_l Y_{aj} + \epsilon_{klm} \hat{\xi}_l \delta_{ij} Y_{am}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})} \tag{C.45}$$

$$- \epsilon_{klm} \hat{\xi}_l \frac{1}{z_a^2 \boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} \left(\frac{1}{\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}}} + \frac{1}{z_a} \right) z_i z_j Y_{am} \tag{C.46}$$

$$- \frac{\epsilon_{klm} \hat{\xi}_l \hat{\xi}_i Y_{am} Y_{aj} + \epsilon_{klm} \hat{\xi}_l \hat{\xi}_j z_i Y_{am}}{z_a(\boldsymbol{Y}_a \cdot \hat{\boldsymbol{\xi}})^2} \tag{C.47}$$

and

$$\epsilon_{klm}\hat{\xi}_l\frac{\partial^3 f}{\partial z_p\partial z_p z_m}=\epsilon_{klm}\hat{\xi}_l\left(\frac{2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}+\frac{a^2}{z_a^2(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})^2}+\frac{a^2}{z_a^3\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right)Y_{am} \tag{C.48}$$

Simplifying:

$$\begin{aligned}\epsilon_{klm}\hat{\xi}_l\frac{\partial^3 f}{\partial z_i\partial z_j z_m}&=\frac{\epsilon_{klj}\hat{\xi}_l Y_{ai}+\epsilon_{kli}\hat{\xi}_l Y_{aj}}{z_a(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})}\\&+\frac{\epsilon_{klm}\hat{\xi}_l Y_{am}}{z_a(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})}\left[\delta_{ij}-\frac{z_a+\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}{z_a^2\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}z_iz_j-\frac{\hat{\xi}_iz_j+\hat{\xi}_jz_i+z_a\hat{\xi}_i\hat{\xi}_j}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right]\end{aligned} \tag{C.49}$$

$$\tag{C.50}$$

C.2 Stress field of a straight segment

The stress field of the segment $\boldsymbol{A}\to\boldsymbol{B}$ is

$$\sigma_{ij}(\boldsymbol{x})=b_k\int_{\boldsymbol{A}}^{\boldsymbol{B}}S_{ijkl}R_a\ d\ell'_l \tag{C.51}$$

$$S_{ijkl}=\frac{\mu}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk}+\delta_{jl}\epsilon_{imk}\right)\partial_m\Delta+\frac{2}{1-\nu}\epsilon_{klm}\left(\partial_i\partial_j\partial_m-\delta_{ij}\partial_m\Delta\right)\right] \tag{C.52}$$

Now notice that differentiation is in $\boldsymbol{x}$, while integration is in $\boldsymbol{x}'$. Therefore they commute.

$$\begin{aligned}\sigma_{ij}(\boldsymbol{x})&=b_k\hat{\xi}_lS_{ijkl}q(\boldsymbol{x})\\&=\hat{\xi}_lb_k\frac{\mu}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk}+\delta_{jl}\epsilon_{imk}\right)\partial_m\Delta+\frac{2}{1-\nu}\epsilon_{klm}\left(\partial_i\partial_j\partial_m-\delta_{ij}\partial_m\Delta\right)\right]\left[f(\boldsymbol{B}-\boldsymbol{x})-f(\boldsymbol{A}-\boldsymbol{x})\right]\\&=\hat{\xi}_lb_k\frac{\mu}{8\pi}\left[\left(\delta_{il}\epsilon_{jmk}+\delta_{jl}\epsilon_{imk}\right)\partial_m\Delta+\frac{2}{1-\nu}\epsilon_{klm}\left(\partial_i\partial_j\partial_m-\delta_{ij}\partial_m\Delta\right)\right]\left[f(\boldsymbol{z})\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}\\&=\frac{\mu}{4\pi(1-\nu)}\left[\frac{1-\nu}{2}\left(\hat{\xi}_ib_k\epsilon_{jmk}+\hat{\xi}_jb_k\epsilon_{imk}\right)\frac{\partial^3 f}{\partial z_p\partial z_p\partial z_m}+\epsilon_{klm}\hat{\xi}_lb_k\left(\frac{\partial^3 f}{\partial z_i\partial z_j\partial z_m}-\delta_{ij}\frac{\partial^3 f}{\partial z_p\partial z_p\partial z_m}\right)\right]_{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}\end{aligned} \tag{C.53}$$

Plugging back in (C.53) we obtain

$$\begin{aligned}\sigma_{ij}(\boldsymbol{x})=\frac{\mu}{4\pi(1-\nu)}\Bigg[&\frac{1-\nu}{2}\left(\hat{\xi}_ib_k\epsilon_{jmk}+\hat{\xi}_jb_k\epsilon_{imk}\right)\left[\left(\frac{2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}+\frac{a^2}{z_a^2(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})^2}\right)Y_{am}+\frac{a^2}{z_a^3\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}z_m\right]\\&+\frac{\epsilon_{klj}b_k\hat{\xi}_lY_{ai}+\epsilon_{kli}b_k\hat{\xi}_lY_{aj}}{z_a(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})}\\&+\frac{\epsilon_{klm}b_k\hat{\xi}_lY_{am}}{z_a(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})}\left(\delta_{ij}-\frac{z_a+\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}{z_a^2\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}z_iz_j-\frac{\hat{\xi}_iz_j+\hat{\xi}_jz_i+z_a\hat{\xi}_i\hat{\xi}_j}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right)\\&-\epsilon_{klm}b_k\hat{\xi}_lY_{am}\delta_{ij}\left(\frac{2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}+\frac{a^2}{z_a^2(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}})^2}+\frac{a^2}{z_a^3\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right)\Bigg]_{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}\end{aligned} \tag{C.54}$$

Factoring the term $z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}=(Y_a^2+a^2)/2$ we obtain

$$\begin{aligned}\sigma_{ij}(\boldsymbol{x})=\frac{\mu}{4\pi(1-\nu)}\Bigg[&\frac{2}{Y_a^2+a^2}\Bigg\{(1-\nu)\left(\hat{\xi}_ib_k\epsilon_{jmk}+\hat{\xi}_jb_k\epsilon_{imk}\right)\left[\left(1+\frac{a^2}{Y_a^2+a^2}\right)Y_{am}+\frac{a^2}{2z_a^2}z_m\right]\\&+\epsilon_{klj}b_k\hat{\xi}_lY_{ai}+\epsilon_{kli}b_k\hat{\xi}_lY_{aj}\\&-\epsilon_{klm}b_k\hat{\xi}_lY_{am}\left(\left(1+2\frac{a^2}{Y_a^2+a^2}+\frac{a^2}{z_a^2}\right)\delta_{ij}+2\frac{z_a+\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}{z_a(Y_a^2+a^2)}z_iz_j+\frac{\hat{\xi}_iz_j+\hat{\xi}_jz_i+z_a\hat{\xi}_i\hat{\xi}_j}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right)\Bigg\}\Bigg]_{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}\end{aligned} \tag{C.55}$$

Inverting the order of evaluation, and changing the sign within the square bracket we have

$$\sigma_{ij}(\boldsymbol{x}) = \frac{\mu}{4\pi(1-\nu)}\left[\frac{2}{Y_a^2+a^2}\left\{(1-\nu)\left(\hat{\xi}_i b_k\epsilon_{jkm}+\hat{\xi}_j b_k\epsilon_{ikm}\right)\left[\left(1+\frac{a^2}{Y_a^2+a^2}\right)Y_{am}+\frac{a^2}{2z_a^2}z_m\right]\right.\right.$$
$$-\left(Y_{ai}\epsilon_{jkl}b_k\hat{\xi}_l+\epsilon_{ikl}b_k\hat{\xi}_l Y_{aj}\right)$$
$$\left.\left.-\epsilon_{klm}b_k Y_{al}\hat{\xi}_m\left(\left(1+2\frac{a^2}{Y_a^2+a^2}+\frac{a^2}{z_a^2}\right)\delta_{ij}+2\frac{z_a+\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}{z_a(Y_a^2+a^2)}z_i z_j+\frac{\hat{\xi}_i z_j+\hat{\xi}_j z_i+z_a\hat{\xi}_i\hat{\xi}_j}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right)\right\}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}} \tag{C.56}$$

It is more efficient to compute the stress field as

$$\boldsymbol{\sigma}(\boldsymbol{x}) = \frac{\mu}{4\pi(1-\nu)}\left(\boldsymbol{s}(\boldsymbol{x})+\boldsymbol{s}^T(\boldsymbol{x})\right) \tag{C.57}$$

where

$$\boldsymbol{s}(\boldsymbol{x}) = \tilde{\boldsymbol{s}}(\boldsymbol{B}-\boldsymbol{x})-\tilde{\boldsymbol{s}}(\boldsymbol{A}-\boldsymbol{x}) \tag{C.58}$$

and

$$\tilde{\boldsymbol{s}}(\boldsymbol{z}) = \frac{2}{Y_a^2+a^2}\left\{(1-\nu)\left(1+\frac{a^2}{Y_a^2+a^2}\right)\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\boldsymbol{Y}_a)\right.$$
$$+(1-\nu)\frac{a^2}{2z_a^2}\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\boldsymbol{z})$$
$$-\boldsymbol{Y}_a\otimes\left(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}\right)$$
$$-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y}_a)\cdot\hat{\boldsymbol{\xi}}\right]}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\boldsymbol{z}$$
$$\left.-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y}_a)\cdot\hat{\boldsymbol{\xi}}\right]}{2}\left(\left(1+2\frac{a^2}{Y_a^2+a^2}+\frac{a^2}{z_a^2}\right)\boldsymbol{I}+2\frac{z_a+\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}{z_a(Y_a^2+a^2)}\boldsymbol{z}\otimes\boldsymbol{z}+\frac{z_a}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\hat{\boldsymbol{\xi}}\right)\right\} \tag{C.59}$$

with

$$\boldsymbol{Y}_a(\boldsymbol{z}) = \boldsymbol{z}+z_a\hat{\boldsymbol{\xi}} \tag{C.60}$$
$$Y_a^2(\boldsymbol{z}) = 2z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}-a^2 \tag{C.61}$$

Note that

$$\left(Y_a^2+a^2\right)^2 = 4z_a^2\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right)^2 = 4z_a^2\left(\boldsymbol{z}\cdot\hat{\boldsymbol{\xi}}+z_a\right)^2 \tag{C.62}$$

Stress of a straight segment on a distant point

Suppose that we want to compute the stress field of a segment $\boldsymbol{AB}$ at a point $\boldsymbol{x}$, which is located at distance d from the center $\boldsymbol{C}$ of the segment. Let the length of the segment be $AB=2L$, then

$$\boldsymbol{B}-\boldsymbol{x} = \boldsymbol{C}-\boldsymbol{x}+\boldsymbol{C}-\boldsymbol{B} = \boldsymbol{C}-\boldsymbol{x}-L\hat{\boldsymbol{\xi}} = d\left(\hat{\boldsymbol{r}}_c-\frac{L}{d}\hat{\boldsymbol{\xi}}\right) \tag{C.63}$$

$$\boldsymbol{A}-\boldsymbol{x} = \boldsymbol{C}-\boldsymbol{x}+\boldsymbol{C}-\boldsymbol{A} = \boldsymbol{C}-\boldsymbol{x}+L\hat{\boldsymbol{\xi}} = d\left(\hat{\boldsymbol{r}}_c+\frac{L}{d}\hat{\boldsymbol{\xi}}\right) \tag{C.64}$$

where $\hat{\boldsymbol{r}}_c = (\boldsymbol{C}-\boldsymbol{x})/\|\boldsymbol{C}-\boldsymbol{x}\| = (\boldsymbol{C}-\boldsymbol{x})/d$. Therefore we have

$$\boldsymbol{s}(\boldsymbol{x}) = \tilde{\boldsymbol{s}}\left(d\left(\hat{\boldsymbol{r}}_c-\frac{L}{d}\hat{\boldsymbol{\xi}}\right)\right)-\tilde{\boldsymbol{s}}\left(d\left(\hat{\boldsymbol{r}}_c+\frac{L}{d}\hat{\boldsymbol{\xi}}\right)\right) \tag{C.65}$$

If d is large compared to the segment length, that is

$$\frac{L}{d} \ll 1 \tag{C.66}$$

then the difference above can be approximated as

$$\boldsymbol{s}(\boldsymbol{x}) \approx \left.\frac{\partial \tilde{\boldsymbol{s}}(\boldsymbol{z})}{\partial \boldsymbol{z}}\right|_{\boldsymbol{z}=\boldsymbol{C}-\boldsymbol{x}} \cdot (-2L\hat{\boldsymbol{\xi}}) \tag{C.67}$$

Note that

$$\frac{\partial Y_i}{\partial z_j}\hat{\xi}_j = \frac{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}{z}\hat{\xi}_i \tag{C.68}$$

then

$$\begin{aligned}
\frac{\partial \tilde{\boldsymbol{s}}(\boldsymbol{z})}{\partial \boldsymbol{z}} = \frac{\partial}{\partial \boldsymbol{z}}\Bigg[\frac{2}{Y^2}\Bigg\{&(1-\nu)\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\boldsymbol{Y})\\
&-\boldsymbol{Y}\otimes\left(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}\right)\\
&-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\boldsymbol{z}\\
&-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{2}\left(\boldsymbol{I}+2\frac{z+\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}{zY^2}\boldsymbol{z}\otimes\boldsymbol{z}+\frac{z}{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\hat{\boldsymbol{\xi}}\right)\Bigg\}\Bigg]\cdot(-2L\hat{\boldsymbol{\xi}})\\
=&\left[-\frac{2}{Y}\frac{\partial Y}{\partial \boldsymbol{z}}\cdot\left(-2L\hat{\boldsymbol{\xi}}\right)\right]\tilde{\boldsymbol{s}}(\boldsymbol{z})\\
&+\frac{2}{Y^2}\Bigg[\frac{\partial}{\partial \boldsymbol{z}}\Bigg\{(1-\nu)\hat{\boldsymbol{\xi}}\otimes(\boldsymbol{b}\times\boldsymbol{Y})\\
&-\boldsymbol{Y}\otimes\left(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}\right)\\
&-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\boldsymbol{z}\\
&-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{2}\left(\boldsymbol{I}+2\frac{z+\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}{zY^2}\boldsymbol{z}\otimes\boldsymbol{z}+\frac{z}{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\hat{\boldsymbol{\xi}}\right)\Bigg\}\Bigg]\cdot(-2L\hat{\boldsymbol{\xi}})\\
=&\frac{4L}{zY^2}\left(\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}\right)^2\tilde{\boldsymbol{s}}(\boldsymbol{z})\\
&-\frac{4L}{Y^2}\Bigg[\Bigg\{(1-\nu)\frac{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}{z}\hat{\boldsymbol{\xi}}\otimes\left(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}\right)\\
&-\frac{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}{z}\hat{\boldsymbol{\xi}}\otimes\left(\boldsymbol{b}\times\hat{\boldsymbol{\xi}}\right)\\
&+\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{z\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\boldsymbol{z}-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\hat{\boldsymbol{\xi}}\\
&-\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]\left(\frac{z+\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}{zY^2}\boldsymbol{z}\otimes\boldsymbol{z}\right)\cdot\hat{\boldsymbol{\xi}}\Bigg\}\Bigg]\\
&-\frac{\left[(\boldsymbol{b}\times\boldsymbol{Y})\cdot\hat{\boldsymbol{\xi}}\right]}{2}\frac{\partial}{\partial \boldsymbol{z}}\left(\frac{z}{\boldsymbol{Y}\cdot\hat{\boldsymbol{\xi}}}\hat{\boldsymbol{\xi}}\otimes\hat{\boldsymbol{\xi}}\right)\cdot\hat{\boldsymbol{\xi}}\Bigg\}\Bigg]
\end{aligned} \tag{C.69}$$

Now let C be the center of the segment $\boldsymbol{AB}$, so that

Stress of a straight segment in the neighborhood of a far point

Suppose that we want to compute the stress field of segment $\boldsymbol{AB}$ at a point $\boldsymbol{x}$ which is in the neighborhood of a point $\boldsymbol{x}_0$. Let the distance from $\boldsymbol{x}_0$ to the segment $\boldsymbol{AB}$ be d. Then we work under the assumption $AB \ll d$ and $\|\boldsymbol{x}-\boldsymbol{x}_0\| \ll d$.

C.3 Straight Displacement

$$\begin{aligned}
u_i &= -\frac{b_i\Omega}{4\pi} - b_k\hat{\xi}_l U_{ikl}\oint_{\mathcal{L}} R_a dL' \\
&= -\frac{b_i\Omega}{4\pi} - b_k\hat{\xi}_l\frac{1}{8\pi}\epsilon_{klm}\left[\delta_{im}\Delta - \frac{1}{1-\nu}\partial_i\partial_m\right][f(\boldsymbol{B}-\boldsymbol{x}) - f(\boldsymbol{A}-\boldsymbol{x}))\\
&= -\frac{b_i\Omega}{4\pi} - b_k\hat{\xi}_l\frac{1}{8\pi}\epsilon_{klm}\left[\delta_{im}\frac{\partial^2 f}{\partial z_p\partial z_p} - \frac{1}{1-\nu}\frac{\partial^2 f}{\partial z_i\partial z_m}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}\\
&= -\frac{b_i\Omega}{4\pi} + \epsilon_{ilk}\hat{\xi}_l b_k\frac{1}{8\pi}\left[\frac{\partial^2 f}{\partial z_p\partial z_p}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}} + \frac{1}{8\pi}\frac{1}{1-\nu}\epsilon_{klm}b_k\hat{\xi}_l\left[\frac{\partial^2 f}{\partial z_i\partial z_m}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}
\end{aligned} \tag{C.70}$$

From Eq. (C.30)

$$\begin{aligned}
u_i = &-\frac{b_i\Omega}{4\pi}\\
&+\epsilon_{ilk}\hat{\xi}_l b_k\frac{1}{8\pi}\left[2+2\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) - \frac{a^2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}\\
&+\frac{1}{8\pi}\frac{1}{1-\nu}\epsilon_{klm}b_k\hat{\xi}_l\left[\frac{\delta_{im}-\hat{\xi}_i\hat{\xi}_m}{2} + (\delta_{im}-\hat{\xi}_i\hat{\xi}_m)\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{1}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\left(\frac{z_iz_m}{z_a} + z_i\hat{\xi}_m + z_m\hat{\xi}_i - z_q\hat{\xi}_q\hat{\xi}_i\hat{\xi}_m\right)\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}
\end{aligned} \tag{C.71}$$

Dropping vanishing terms of containing $\epsilon_{klm}b_k\hat{\xi}_l\hat{\xi}_m$

$$\begin{aligned}
u_i = &-\frac{b_i\Omega}{4\pi}\\
&+\epsilon_{ilk}\hat{\xi}_l b_k\frac{1}{8\pi}\left[2+2\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) - \frac{a^2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}\\
&+\frac{1}{8\pi}\frac{1}{1-\nu}\epsilon_{klm}b_k\hat{\xi}_l\left[\frac{\delta_{im}}{2} + \delta_{im}\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) + \frac{1}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\left(\frac{z_iz_m}{z_a} + z_m\hat{\xi}_i\right)\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}
\end{aligned} \tag{C.72}$$

Factoring common terms

$$\begin{aligned}
u_i = &-\frac{b_i\Omega}{4\pi}\\
&-\epsilon_{ikl}b_k\hat{\xi}_l\frac{1}{8\pi}\left[2-\frac{1}{2}\frac{1}{1-\nu} + \left(2-\frac{1}{1-\nu}\right)\ln\left(\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}\right) - \frac{a^2}{z_a\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}\\
&+\frac{1}{8\pi}\frac{1}{1-\nu}\epsilon_{klm}b_k\hat{\xi}_l\left[\frac{z_m}{\boldsymbol{Y}_a\cdot\hat{\boldsymbol{\xi}}}\frac{Y_{ai}}{z_a}\right]_{\boldsymbol{z}=\boldsymbol{A}-\boldsymbol{x}}^{\boldsymbol{z}=\boldsymbol{B}-\boldsymbol{x}}
\end{aligned} \tag{C.73}$$

Straight Solid Angle

$$\Omega = -\oint\frac{\epsilon_{ijk}\hat{s}_j R_k}{R(R+\hat{s}_p R_p)}dL'_k \tag{C.74}$$

where $\boldsymbol{R}=\boldsymbol{x}-\boldsymbol{x}'$. We'll follow the integration method proposed by Asvestas and England [1994]. To do this we work with $\boldsymbol{Y}=-\boldsymbol{R}$ and $\hat{\boldsymbol{Y}}=\boldsymbol{Y}/Y$ to obtain

$$\Omega=\oint\frac{\epsilon_{ijk}\hat{s}_jY_k}{Y(Y-\hat{s}_pY_p)}dL'_k=\oint\frac{\epsilon_{ijk}\hat{s}_j\hat{Y}_k}{Y(1-\hat{s}_p\hat{Y}_p)}dL'_k \tag{C.75}$$

This integral can be evaluated by summing over straight segments. Each segment can be evaluated on unit sphere where $Y=1$, therefore

$$\Omega_n=\int_0^{\alpha_n}\frac{\epsilon_{ijk}\hat{s}_j\hat{Y}_k}{(1-\hat{s}_p\hat{Y}_p)}\hat{t}'_kd\alpha \tag{C.76}$$

where $\cos\alpha_n=\boldsymbol{Y}_n\cdot\boldsymbol{Y}_{n+1}$. If we let

$$\hat{\boldsymbol{e}}_1=\hat{\boldsymbol{Y}}_n\qquad\hat{\boldsymbol{e}}_3=\frac{\hat{\boldsymbol{Y}}_n\times\hat{\boldsymbol{Y}}_{n+1}}{|\hat{\boldsymbol{Y}}_n\times\hat{\boldsymbol{Y}}_{n+1}|}\qquad\hat{\boldsymbol{e}}_2=\hat{\boldsymbol{e}}_3\times\hat{\boldsymbol{e}}_1 \tag{C.77}$$

then

$$\hat{\boldsymbol{Y}}(\alpha)=\cos\alpha\,\hat{\boldsymbol{e}}_1+\sin\alpha\,\hat{\boldsymbol{e}}_2 \tag{C.78}$$

$$\hat{\boldsymbol{t}}(\alpha)=-\sin\alpha\,\hat{\boldsymbol{e}}_1+\cos\alpha\,\hat{\boldsymbol{e}}_2 \tag{C.79}$$

$$\hat{\boldsymbol{s}}=s_1\hat{\boldsymbol{e}}_1+s_2\hat{\boldsymbol{e}}_2+s_3\hat{\boldsymbol{e}}_3 \tag{C.80}$$

Substituting

$$\Omega_n=\int_0^{\alpha_n}\frac{s_3}{1-s_1\cos\alpha-s_2\sin\alpha}d\alpha \tag{C.81}$$

This integral is evaluated using the transformation

$$\alpha=2\tan^{-1}w\qquad\cos\alpha=\frac{1-w^2}{1+w^2}\qquad\sin\alpha=\frac{2w}{1+w^2}\qquad d\alpha=\frac{2dw}{1+w^2} \tag{C.82}$$

which gives

$$\begin{aligned}
\Omega_n&=2s_3\int_0^{w^*}\frac{1}{1+w^2-s_1(1-w^2)-s_2 2w}d\alpha=2s_3\int_0^{w^*}\frac{1}{(1+s_1)w^2-2s_2w+1-s_1}d\alpha\\
&=2s_3\left[\frac{2}{\sqrt{4-4s_1^2-4s_2^2}}\tan^{-1}\left(\frac{2(1+s_1)w-2s_2}{\sqrt{4-4s_1^2-4s_2^2}}\right)\right]_0^{w^*}\\
&=2\frac{s_3}{|s_3|}\left[\tan^{-1}\left(\frac{(1+s_1)w-s_2}{|s_3|}\right)\right]_0^{w^*}\\
&=2\frac{s_3}{|s_3|}\tan^{-1}\left(\frac{\frac{(1+s_1)w^*-s_2}{|s_3|}+\frac{s_2}{|s_3|}}{1+\frac{(1+s_1)w^*-s_2}{|s_3|}\frac{-s_2}{|s_3|}}\right)\\
&=2\frac{s_3}{|s_3|}\tan^{-1}\left(\frac{(1+s_1)w^*}{|s_3|-s_2\frac{(1+s_1)w^*-s_2}{|s_3|}}\right)\\
&=2\frac{s_3}{|s_3|}\tan^{-1}\left(\frac{|s_3|(1+s_1)w^*}{s_3^2-s_2((1+s_1)w^*-s_2)}\right)\\
&=2\frac{s_3}{|s_3|}\tan^{-1}\left(\frac{|s_3|w^*}{1-s_1-s_2w^*}\right)\\
&=2\tan^{-1}\left(\frac{s_3w^*}{1-s_1-s_2w^*}\right)
\end{aligned} \tag{C.83}$$

where

$$w^*=\sqrt{\frac{1-\boldsymbol{Y}_n\cdot\boldsymbol{Y}_{n+1}}{1+\boldsymbol{Y}_n\cdot\boldsymbol{Y}_{n+1}}} \tag{C.84}$$

Non-singular Straight Solid Angle

$$\Omega = -\oint \frac{\epsilon_{ijk}\hat{s}_j R_k}{R_a(R_a+\hat{s}_p R_p)} dL'_k = \oint \frac{\epsilon_{ijk}\hat{s}_j Y_k}{Y_a(Y_a-\hat{s}_p Y_p)} dL'_k \tag{C.85}$$

If we now evaluate this expression on the unit sphere where $Y = 1$ and $Y_a = \sqrt{1+a^2}$ we have

$$\Omega_n = \int_0^{\alpha_n} \frac{\epsilon_{ijk}\hat{s}_j \hat{Y}_k}{\sqrt{1+a^2}(\sqrt{1+a^2}-\hat{s}_p\hat{Y}_p)} \hat{t}'_k d\alpha \tag{C.86}$$

We can then proceed as before and finally find

$$\Omega_n = \frac{2}{\sqrt{1+a^2}\sqrt{s_3^2+a^2}} \tan^{-1}\left(\frac{\sqrt{s_3^2+a^2}w^*}{\sqrt{1+a^2}-s_1-s_2w^*}\right) \tag{C.87}$$

Appendix D

DDD: mathematical identities

D.1 Mathematical preliminaries

D.1.1 Kelvin-Stokes Theorem

Theorem 1. *(Kelvin-Stokes Theorem)*

$$\int_{\mathcal{S}} \epsilon_{ikl} f_{,l} \hat{n}_k ds = \oint_{\partial \mathcal{S}} f dl_i \tag{D.1}$$

Theorem 2. *(Corollary)*

$$\int_{\mathcal{S}} \left(f_{,q} \hat{n}_p - f_{,p} \hat{n}_q \right) ds = \oint_{\partial \mathcal{S}} \epsilon_{ipq} f dl_i \tag{D.2}$$

D.1.2 Surface Divergence Theorem

In this section we recall some useful theorems in surface integration. Slattery et al. [2006], Scovazzi and Hughes [2007].

See Slattery et al. [2006] p. 669.

Status of the inherited derivation

The original notes did not supply the surface-divergence theorem here. The cited reference is retained for completion.

D.1.3 Surface Transport Theorem

Theorem 3. *(Surface Transport Theorem)*

$$\frac{d}{dt} \int_{\mathcal{S}_x = \boldsymbol{\varphi}(\mathcal{S}_X)} \alpha \hat{n}_i ds = \int_{\mathcal{S}_x} \left(\frac{\partial \alpha}{\partial t} + (\alpha \dot{\varphi}_k)_{,k} \right) \hat{n}_i ds - \int_{\mathcal{S}_x} \alpha \dot{\varphi}_{k,i} \hat{n}_k ds \tag{D.3}$$

Proof. From Scovazzi and Hughes [2007]. □

Theorem 4. *(Surface Transport Theorem (alternative form))*

$$\frac{d}{dt} \int_{\mathcal{S}_x} \alpha ds = \int_{\mathcal{S}_x} \frac{\partial \alpha}{\partial t} \hat{n}_i ds + \oint \epsilon_{ikm} \alpha v_k dl_m + \int_{\mathcal{S}_x} \alpha_{,i} v_k \hat{n}_k ds \tag{D.4}$$

Proof. Consider the second term on the r.h.s. of D.3 and use (D.2):

$$\int_{\mathcal{S}_x} \left(\alpha\dot{\varphi}_k\right)_{,k} \hat{n}_i ds = \int_{\mathcal{S}_x} \left(\alpha\dot{\varphi}_k\right)_{,i} \hat{n}_k ds + \oint \epsilon_{mik}\alpha v_k dl_m \tag{D.5}$$

Now consider the third term on the r.h.s. of D.3

$$\int_{\mathcal{S}_x} \alpha\dot{\varphi}_{k,i}\hat{n}_k ds = \int_{\mathcal{S}_x} \left(\alpha v_k\right)_{,i} \hat{n}_k ds - \int_{\mathcal{S}_x} \alpha_{,i} v_k \hat{n}_k ds \tag{D.6}$$

Subtract to obtain result □

Theorem 5. *(Surface Transport Theorem (alternative form))*

$$\frac{d}{dt}\int_{\mathcal{S}_x=\boldsymbol{\varphi}(\mathcal{S}_X)} \alpha ds = \tag{D.7}$$

Proof. See Slattery et al. [2006] p 61. □

D.1.4 Solid angles and line integration

Theorem 6. *(Surface to Line integration) Let $\mathcal{S}$ be a surface bounded by the closed contour Γ. Then let $\boldsymbol{R} = \boldsymbol{R}_s - \boldsymbol{R}_f$ where $\boldsymbol{R}_f$ is the field point and $\boldsymbol{R}_s$ is the source point on $\mathcal{S}$. If $\boldsymbol{s}$ is a unit vector applied at $\boldsymbol{R}_f$ such that the direction containing it never intersects $\mathcal{S}$ nor Γ, then:*

$$-\int \frac{\boldsymbol{R}}{R^3}\cdot\hat{n}dS = \oint \frac{\boldsymbol{s}\times\boldsymbol{R}}{R(R-\boldsymbol{s}\cdot\boldsymbol{R})}\cdot d\boldsymbol{l} \tag{D.8}$$

Obviously if $\boldsymbol{R}$ has opposite definition, i.e. $\boldsymbol{R} = \boldsymbol{R}_f - \boldsymbol{R}_s$ then

$$\int \frac{\boldsymbol{R}}{R^3}\cdot\hat{n}dS = -\oint \frac{\boldsymbol{s}\times\boldsymbol{R}}{R(R+\boldsymbol{s}\cdot\boldsymbol{R})}\cdot d\boldsymbol{l} \tag{D.9}$$

Note that for unit vector $\boldsymbol{s}$ we have $(\boldsymbol{R}-R\boldsymbol{s})^2 = R^2+R^2-2R\boldsymbol{R}\cdot\boldsymbol{s} = 2R(R-\boldsymbol{R}\cdot\boldsymbol{s})$. This means that $2R(R-\boldsymbol{R}\cdot\boldsymbol{s}) = 0$ only when $\boldsymbol{R}$ is aligned with $\boldsymbol{s}$.

Proof. From Stokes theorem:

$$\oint \frac{\boldsymbol{s}\times\boldsymbol{R}}{R(R-\boldsymbol{s}\cdot\boldsymbol{R})}\cdot d\boldsymbol{l} = \int \nabla\times\left[\frac{\boldsymbol{s}\times\boldsymbol{R}}{R(R-\boldsymbol{s}\cdot\boldsymbol{R})}\right] ds \tag{D.10}$$

Now exapand the curl

$$\begin{aligned}
\nabla\times\left[\frac{\boldsymbol{s}\times\boldsymbol{R}}{R(R-\boldsymbol{s}\cdot\boldsymbol{R})}\right] &= \epsilon_{ijk}\partial_j\left[\frac{\epsilon_{kmn}s_mR_n}{R(R-s_pR_p)}\right] = (\delta_{im}\delta_{jn}-\delta_{in}\delta_{jm})\,\partial_j\left[\frac{s_mR_n}{R(R-s_pR_p)}\right]\\
&= (\delta_{im}\delta_{jn}-\delta_{in}\delta_{jm})\left[\frac{s_mR_{n,j}}{R(R-s_pR_p)}-\frac{s_mR_n\left[R_{,j}(R-s_pR_p)+R(R_{,j}-s_pR_{p,j})\right]}{R^2(R-s_pR_p)^2}\right]\\
&= (\delta_{im}\delta_{jn}-\delta_{in}\delta_{jm})\left[\frac{s_m\delta_{nj}}{R(R-s_pR_p)}-\frac{s_mR_n\left[\frac{R_j}{R}(R-s_pR_p)+R(\frac{R_j}{R}-s_p\delta_{pj})\right]}{R^2(R-s_pR_p)^2}\right]\\
&= (\delta_{im}\delta_{jn}-\delta_{in}\delta_{jm})\left[\frac{s_m\delta_{nj}}{R(R-s_pR_p)}-\frac{s_mR_n\left[\frac{R_j}{R}(R-s_pR_p)+(R_j-Rs_j)\right]}{R^2(R-s_pR_p)^2}\right]\\
&= \left[\frac{\delta_{im}\delta_{jn}s_m\delta_{nj}}{R(R-s_pR_p)}-\frac{\delta_{im}\delta_{jn}s_mR_n\left[\frac{R_j}{R}(R-s_pR_p)+(R_j-Rs_j)\right]}{R^2(R-s_pR_p)^2}\right]\\
&- \left[\frac{\delta_{in}\delta_{jm}s_m\delta_{nj}}{R(R-s_pR_p)}-\frac{\delta_{in}\delta_{jm}s_mR_n\left[\frac{R_j}{R}(R-s_pR_p)+(R_j-Rs_j)\right]}{R^2(R-s_pR_p)^2}\right]\\
&= \left[\frac{3s_i}{R(R-s_pR_p)}-\frac{s_i\left[R(R-s_pR_p)+R(R-R_js_j)\right]}{R^2(R-s_pR_p)^2}\right]\\
&- \left[\frac{s_i}{R(R-s_pR_p)}-\frac{R_i\left[\frac{R_js_j}{R}(R-s_pR_p)+(R_js_j-R)\right]}{R^2(R-s_pR_p)^2}\right]\\
&= \frac{3s_i}{R(R-s_pR_p)}-\frac{2s_i}{R(R-s_pR_p)}\\
&- \frac{s_i}{R(R-s_pR_p)}+R_i\frac{\left[-R-\frac{R_js_js_pR_p}{R}+2R_js_j\right]}{R^2(R^2+s_pR_ps_jR_j-2Rs_pR_p)}\\
&= -\frac{R_i}{R^3}
\end{aligned}\tag{D.11}$$

□

D.2 Elastic energy of a dislocation loop: working derivation

D.2.1 Volume to Surface

(Eshelby) Let's consider a body delimited by a closed surface S_0 having a source of eigenstress[1] I enclosed within the surface S and a source of stress II outside S.

$$\begin{aligned}
E &= \frac{1}{2}\int_V\sigma_{ij}\epsilon_{ij}dV = \frac{1}{2}\int_V\left(\sigma^I_{ij}+\sigma^{II}_{ij}\right)\left(\epsilon^I_{ij}+\epsilon^{II}_{ij}\right)dV\\
&= \frac{1}{2}\int_V\sigma^I_{ij}\epsilon^I_{ij}dV+\frac{1}{2}\int_V\sigma^{II}_{ij}\epsilon^{II}_{ij}dV+\underbrace{\frac{1}{2}\int_V\sigma^I_{ij}\epsilon^{II}_{ij}dV+\frac{1}{2}\int_V\sigma^{II}_{ij}\epsilon^I_{ij}dV}_{\text{equal for reciprocity theorem}}\\
&= \underbrace{\frac{1}{2}\int_V\sigma^I_{ij}\epsilon^I_{ij}dV}_{E_I}+\underbrace{\frac{1}{2}\int_V\sigma^{II}_{ij}\epsilon^{II}_{ij}dV}_{E_{II}}+\underbrace{\int_V\sigma^I_{ij}\epsilon^{II}_{ij}dV}_{E_{int}}
\end{aligned}\tag{D.12}$$

[1] no external surface traction

Away from their respective sources strains are gradients of displacement fields therefore $\epsilon_{ij}^I = \frac{1}{2}\left(u_{i,j}^I + u_{j,i}^I\right)$ in V_{II} and $\epsilon_{ij}^{II} = \frac{1}{2}\left(u_{i,j}^{II} + u_{j,i}^{II}\right)$ in V_I, but not conversely.

$$\begin{aligned} E_{int} &= \int_{V_I} \sigma_{ij}^I u_{i,j}^{II} dV_I + \int_{V_{II}} \sigma_{ij}^{II} u_{i,j}^I dV_{II} \\ &= \oint_S \sigma_{ij}^I u_i^{II} \hat{n}_j dS - \underbrace{\int_{V_I} \sigma_{i,j}^I u_i^{II} dV_I}_{\text{equilibrium}} \\ &+ \oint_{S_0} \sigma_{ij}^{II} u_i^I \hat{n}_j dS - \oint_S \sigma_{ij}^{II} u_i^I \hat{n}_j dS + \underbrace{\int_{V_{II}} \sigma_{i,j}^{II} u_i^I dV_{II}}_{\text{equilibrium}} \\ &= \oint_S \left(\sigma_{ij}^I u_i^{II} - \sigma_{ij}^{II} u_i^I\right) \hat{n}_j dS + \underbrace{\oint_{S_0} t_i^{II} u_i^I dS}_{\text{no surface tractions}} \end{aligned} \tag{D.13}$$

the end result is

$$E_{int} = b_i \int_{\mathcal{S}} \sigma_{ij}^{II} \hat{n}_j dS \tag{D.14}$$

D.3 Convolution integrals

Throughout this document, the symbol $*$ stands for the convolution operator over the infinite three-dimensional space $\mathbb{R}^3$:

$$f * g = \int_{\mathbb{R}^3} f(\boldsymbol{x} - \boldsymbol{x}') g(\boldsymbol{x}') \, \mathrm{d}V' \tag{D.15}$$

Convolution enjoys the following properties

$$f * g = \int_{\mathbb{R}^3} f(\boldsymbol{x} - \boldsymbol{x}') g(\boldsymbol{x}') \, \mathrm{d}V' = -\int_{-\mathbb{R}^3} f(\boldsymbol{x}'') g(\boldsymbol{x} - \boldsymbol{x}'') \, \mathrm{d}V'' = g * f \tag{D.16}$$

$$(f * g)_{,i} = \int_{\mathbb{R}^3} f_{,i}(\boldsymbol{x} - \boldsymbol{x}') g(\boldsymbol{x}') \, \mathrm{d}V' = f_{,i} * g = f * g_{,i} \tag{D.17}$$

$$(f * g) * h = f * (g * h) \tag{D.18}$$

D.4 The nabla operator

We use ∇ for the spatial gradient, $\Delta = \partial_i \partial_i$ for the Laplacian, and a comma for component differentiation. Repeated Cartesian indices are summed.

D.5 The Green's tensor and the F-tensor in classical anisotropic elasticity

In classical elasticity, the Green's tensor of the anisotropic Navier operator G_{ij}^0 satisfies the following inhomogeneous PDE:

$$L_{ik} G_{kj}^0 + \delta_{ij} \delta = 0 \, . \tag{D.19}$$

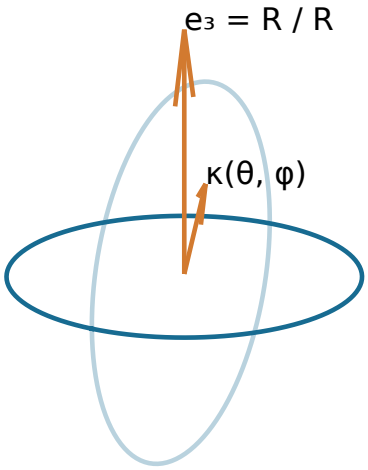


Figure D.1: The unit sphere in Fourier space. The unit vector $\boldsymbol{\kappa}(\theta,\phi)$ is defined by the azimuth angle ϕ, and the zenith angle θ measured from the axis $\hat{\boldsymbol{e}}_3 = \boldsymbol{R}/R$.

In Fourier space[2] this reads:

$$\hat{G}^0_{ik}(\boldsymbol{k}) = \frac{1}{k^2}\,\hat{L}^{-1}_{ik}(\boldsymbol{\kappa})\,. \tag{D.21}$$

where $\hat{L}_{ik}(\boldsymbol{\kappa}) = \mathbb{C}_{ijkl}\kappa_j\kappa_l$, $\boldsymbol{\kappa} = \boldsymbol{k}/k$, and $k = \sqrt{\boldsymbol{k}\cdot\boldsymbol{k}}$. The Green's tensor in real space is obtained by inverse Fourier transform. Expressing the elementary volume element in Fourier space as $\mathrm{d}\hat{V} = k^2\,\mathrm{d}k\,\mathrm{d}\omega$, where $\mathrm{d}\omega$ is an elementary surface element of the unit sphere $\mathcal{S}$, we obtain:

$$G^0_{ik}(\boldsymbol{R}) = \frac{1}{(2\pi)^3}\int_{\mathbb{R}^3}\frac{\hat{L}^{-1}_{ik}(\boldsymbol{\kappa})}{k^2}\,\mathrm{e}^{\mathrm{i}\boldsymbol{k}\cdot\boldsymbol{x}}\,\mathrm{d}\hat{V} = \frac{1}{(2\pi)^3}\int_{\mathcal{S}}\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\int_0^\infty\cos(k\boldsymbol{\kappa}\cdot\boldsymbol{R})\,\mathrm{d}k\,\mathrm{d}\omega = \frac{1}{8\pi^2 R}\int_{\mathcal{S}}\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\,\delta(\boldsymbol{\kappa}\cdot\boldsymbol{R})\,\mathrm{d}\omega\,.$$

Choosing a reference system with $\hat{\boldsymbol{e}}_3$ aligned with $\boldsymbol{R}$, as shown in Fig. D.1, and using the sifting property of the Dirac δ-function, we finally obtain the expression for the Green tensor as:

$$G^0_{ik}(\boldsymbol{R}) = \frac{1}{8\pi^2 R}\int_0^{2\pi}\hat{L}^{-1}_{ik}(\boldsymbol{n})\,\mathrm{d}\phi\,. \tag{D.22}$$

Here, $\boldsymbol{n}$ indicates a unit vector on the equatorial plane of the unit sphere in Fourier space. This result was first obtained by Lifshitz and Rosenzweig [1947] and Synge [1957].

The classical $\boldsymbol{F}$-tensor, introduced by Kirchner [1984], is defined by Eq. (A.19), which in Fourier space reads:

$$\hat{F}^0_{ijkl} = -\hat{G}^0_{kl}\,k_i k_j\,\hat{G}^\Delta = -\frac{1}{k^2}\,\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\,k_i k_j\,\frac{1}{k^2} = -\frac{1}{k^2}\,\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\,\kappa_i\kappa_j\,. \tag{D.23}$$

[2]The Fourier transform and its inverse are defined as, respectively [Vladimirov, 1971]:

$$\hat{f}(\boldsymbol{k}) = \int_{\mathbb{R}^3} f(\boldsymbol{x})\,\mathrm{e}^{-\mathrm{i}\boldsymbol{k}\cdot\boldsymbol{x}}\,\mathrm{d}V\,, \qquad f(\boldsymbol{x}) = \frac{1}{(2\pi)^3}\int_{\mathbb{R}^3}\hat{f}(\boldsymbol{k})\,\mathrm{e}^{\mathrm{i}\boldsymbol{k}\cdot\boldsymbol{x}}\,\mathrm{d}\hat{V}\,. \tag{D.20}$$

The classical $\boldsymbol{F}$-tensor in real space is obtained by inverse Fourier transform:

$$F^0_{ijkl}(\boldsymbol{R}) = -\frac{1}{(2\pi)^3}\int_{\mathbb{R}^3}\frac{1}{k^2}\,\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\,\kappa_i\kappa_j\,\mathrm{e}^{\mathrm{i}\boldsymbol{k}\cdot\boldsymbol{R}}\,\mathrm{d}\hat{V} = -\frac{1}{(2\pi)^3}\int_{\mathcal{S}}\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\,\kappa_i\kappa_j\int_0^\infty\cos(k\boldsymbol{\kappa}\cdot\boldsymbol{R})\,\mathrm{d}k\,\mathrm{d}\omega$$
$$= -\frac{1}{8\pi^2 R}\int_{\mathcal{S}}\hat{L}^{-1}_{kl}(\boldsymbol{\kappa})\,\kappa_i\kappa_j\,\delta(\boldsymbol{\kappa}\cdot\boldsymbol{R})\,\mathrm{d}\omega\,. \tag{D.24}$$

In the reference system of Fig. D.1, we finally obtain:

$$F^0_{ijkl}(\boldsymbol{R}) = -\frac{1}{8\pi^2 R}\int_0^{2\pi}\hat{L}^{-1}_{kl}(\boldsymbol{n})\,n_i n_j\;\mathrm{d}\phi\,. \tag{D.25}$$

Appendix E

DDD: supplementary derivations

Status of the inherited derivation

This chapter groups the earlier disconnected notes with the DDD theory. The distributional manipulations are retained for review. The uniform-controller algebra is useful for DDD loading, although its current implementation belongs to ElasticDeformation (Chapter 6).

E.1 Kunin identity

see Berdichevsky, Victor: Variational Principles of Continuum Mechanics: I. Fundamentals, 2009. p.239.

$$\begin{aligned}\int \varphi(\boldsymbol{x}) e^{jkl} \frac{\partial}{\partial x_k} \int_{\mathcal{S}} \delta(\boldsymbol{x}-\boldsymbol{x}') \hat{n}_l dS' dV &= \int_{\mathcal{S}} \int \varphi(\boldsymbol{x}) e^{jkl} \frac{\partial}{\partial x_k} \delta(\boldsymbol{x}-\boldsymbol{x}') \hat{n}_l dV dS' \\ &= -\int_{\mathcal{S}} \int \frac{\partial \varphi}{\partial x_k}(\boldsymbol{x}) e^{jkl} \delta(\boldsymbol{x}-\boldsymbol{x}') \hat{n}_l dV dS' \\ &= -\int_{\mathcal{S}} \frac{\partial \varphi}{\partial x_k}(\boldsymbol{x}') e^{jkl} \hat{n}_l dS'\end{aligned} \tag{E.1}$$

E.2 Distributional identities for loop fields

Now, since

$$\delta(R) = -\frac{1}{4\pi}\nabla^2\left(\frac{1}{R}\right) = -\frac{1}{8\pi}\nabla^2(R_{,pp}) = -\frac{1}{8\pi}R_{,ppqq} \tag{E.2}$$

then:

$$\beta^p_{kl}(\boldsymbol{x}') = \frac{b_k}{8\pi}\int_{\mathcal{S}} R_{,ppqq}\hat{n}_l dS'' \tag{E.3}$$

Consider the derivative of the solid angle using eq. (D.2)

$$\Omega_{,j} = -\frac{1}{2}\int R_{,ppkj}\hat{n}_k dS = -\frac{1}{2}\int R_{,ppkk}\hat{n}_j dS - \frac{1}{2}\oint \epsilon_{mkj} R_{,ppk}\xi_m dl \tag{E.4}$$

then, from eq. (E.3)

$$\begin{aligned}-\frac{b_i}{4\pi}\Omega_{,j} &= \frac{b_i}{8\pi}\int R_{,ppkk}\hat{n}_j dS + \frac{b_i}{4\pi}\oint \epsilon_{kjm} R_{,ppk}\xi_m dl \\ &= \beta^p_{ij} + \frac{b_i}{4\pi}\oint \epsilon_{kjm} R_{,ppk}\xi_m dl\end{aligned} \tag{E.5}$$

E.3 Uniform loading: retained derivation

The original loading notes describe a uniform controller used to apply a uniform external stress state to the dislocations in the RVE. The RVE is assumed to be subjected to two loads, an applied stress $\boldsymbol{\sigma}_0$, and a stress $\boldsymbol{\sigma}_s$ exerted by a "multiaxial spring". Assuming Voigt notation throughout, the spring has *diagonal* stiffness $\boldsymbol{D}$, and it is under an applied strain $\boldsymbol{\varepsilon}_0$, so that $\boldsymbol{\sigma}_s = \boldsymbol{D}\left(\boldsymbol{\varepsilon}_0 - \boldsymbol{\varepsilon}\right)$, where $\boldsymbol{\varepsilon}$ is the strain in the RVE. Therefore the stress in the RVE is

$$\boldsymbol{\sigma} = \boldsymbol{\sigma}_0 + \boldsymbol{\sigma}_s = \boldsymbol{\sigma}_0 + \boldsymbol{D}\left(\boldsymbol{\varepsilon}_0 - \boldsymbol{\varepsilon}\right) \tag{E.6}$$

On the other hand, the RVE stress $\boldsymbol{\sigma}$ is related to the RVE strain $\boldsymbol{\varepsilon}$ by the relation

$$\boldsymbol{\sigma} = \boldsymbol{C}\left(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_p\right) \tag{E.7}$$

where $\boldsymbol{C}$ is the material stiffness, and ε_p is the plastic strain in the RVE. From (E.6) and (E.7), we see that the RVE strain is

$$\boldsymbol{\varepsilon} = \left(\boldsymbol{C} + \boldsymbol{D}\right)^{-1}\left[\boldsymbol{\sigma}_0 + \boldsymbol{D}\boldsymbol{\varepsilon}_0 + \boldsymbol{C}\boldsymbol{\varepsilon}_p\right] \tag{E.8}$$

while the RVE stress is

$$\boldsymbol{\sigma} = \boldsymbol{C}\left(\boldsymbol{C} + \boldsymbol{D}\right)^{-1}\left[\boldsymbol{\sigma}_0 + \boldsymbol{D}\left(\boldsymbol{\varepsilon}_0 - \boldsymbol{\varepsilon}_p\right)\right] \tag{E.9}$$

We take the diagonal values of $\boldsymbol{D}$ to be $D_i = \alpha_i C_{ii}$ (no summation over i), where α_i is an array called "*machine stiffness ratio*". To understand the effects of the machine stiffness ratio, consider the two limits of small and large $\{\alpha_i\}$'s.

First, suppose that $\alpha_i = 0$ for some i. Let us rewrite stress as

$$\boldsymbol{\sigma} = \underbrace{\left(\boldsymbol{I} + \boldsymbol{D}\boldsymbol{C}^{-1}\right)^{-1}}_{\boldsymbol{A}}\boldsymbol{\sigma}_0 + \underbrace{\left(\boldsymbol{I} + \boldsymbol{D}\boldsymbol{C}^{-1}\right)^{-1}\boldsymbol{D}}_{\boldsymbol{B}}\left(\boldsymbol{\varepsilon}_0 - \boldsymbol{\varepsilon}_p\right) \tag{E.10}$$

Note that the i-th row of the matrix $\boldsymbol{I} + \boldsymbol{D}\boldsymbol{C}^{-1}$ has a 1 in position i and zeros everywhere else. This implies that also the i-th row of the matrix $\boldsymbol{A}$ has a 1 in position i and zeros everywhere else, and in turn that the i-th row of $\boldsymbol{B}$ is zero. This means $\sigma_i = \sigma_{0i}$. In other words, setting the i-th component of the machine stiffness ratio to zero implies that the corresponding stress component σ_i matches the applied stress component σ_{0i}.

Next, consider $\alpha_i \to \infty$ for some i. Let us rewrite strain as

$$\boldsymbol{\varepsilon} = \underbrace{\left(\boldsymbol{I} + \boldsymbol{D}^{-1}\boldsymbol{C}\right)^{-1}\boldsymbol{D}^{-1}}_{\boldsymbol{E}}\left[\boldsymbol{\sigma}_0 + \boldsymbol{C}\boldsymbol{\varepsilon}_p\right] + \underbrace{\left(\boldsymbol{I} + \boldsymbol{D}^{-1}\boldsymbol{C}\right)^{-1}}_{\boldsymbol{F}}\boldsymbol{\varepsilon}_0 \tag{E.11}$$

Here we simply note that the matrices $\boldsymbol{E}$ and $\boldsymbol{F}$ behave similar to the matrices $\boldsymbol{B}$ and $\boldsymbol{A}$ above, respectively. Hence the $\varepsilon_i = \varepsilon_{0i}$. In other words, setting the i-th component of the machine stiffness ratio to ∞ implies that the corresponding strain component ε_i matches the applied strain component ε_{0i}.

Appendix F

Cluster dynamics: current interface and revision boundary

Cluster dynamics is kept last because its physical model is expected to change. This chapter records the current architectural connection and user entry points; it does not freeze a new constitutive specification.

F.1 Connection to the common architecture

`ClusterDynamics` implements `MicrostructureBase`. Its base context and parameter object define species data and numerical resources, and its field queries expose mobile concentrations and immobile-cluster quantities to the other microstructures. DDD climb can consume these concentrations. The container also accepts the CD contribution to average plastic distortion and inelastic displacement rate.

The current spatial model uses the shared FEM infrastructure. Species counts are compile-time parameters `MODELIB_CD_MSIZE` and `MODELIB_CD_ISIZE`; their values must match the material arrays. The default build uses four mobile components and eight immobile fields. These are implementation choices of this snapshot, not a proposed permanent interface for the revised model.

Implementation: `include/ClusterDynamics/ClusterDynamics.h`.

Implementation: `include/ClusterDynamics/ClusterDynamicsParameters.h`.

Implementation: `src/ClusterDynamics/ClusterDynamics.cpp`.

F.2 Current user entry points

Select `ClusterDynamics` in `physics`, enable the shared FEM, and use a material and `ClusterDynamics.txt` compatible with the build. `tutorials/spatialCDtest/` is the entry point for the present spatial model. Coupled climb examples require a compatible DDD selection and species configuration as well.

F.3 What to document after the revision

The final CD chapter should define the new state variables and units first, then the conservation equations, diffusion and reaction operators, boundary conditions, time integration, and coupling to DDD and elastic deformation. Each field's contribution to the common interface must be stated. A completed revision should include mass-balance checks, positivity or admissibility criteria, mesh/time-step convergence, and plots from a reproducible example. Those model-specific derivations and plots are intentionally deferred until the implementation settles.

Bibliography

JS Asvestas and DC England. Computing the solid angle subtended by a planar figure,. *Optical Engineeering*, 33 (12):4055–4059, 1994.

J Blin. Energie Mutuelle De Deux Dislocations. *Acta Metall Mater*, 3(2):199–200, 1955.

Johannes Martinus Burgers. Some considerations on the fields of stress connected with dislocations in a regular crystal lattice I. *P K Ned Akad Wetensc*, 42(1/5):293–325, 1939a.

Johannes Martinus Burgers. Some considerations on the fields of stress connected with dislocations in a regular crystal lattice II . *P K Ned Akad Wetensc*, 42(1/5):378–399, 1939b.

J Deng, A El-Azab, and B C Larson. On the elastic boundary value problem of dislocations in bounded crystals. *Philosophical Magazine*, 88:3527–3548, 2008.

JD Eshelby. Boundary problems. In Frank Reginald Nunes Nabarro, editor, *Dislocations in Solids V1*, pages 167–221. North-Holland, Amsterdam, 1979.

H.O.K. Kirchner. The concept of the line tension: theory and experiments. In P. P. Veyssière, L. Kubin, and J. Castaing, editors, *Dislocations 1984*, pages 53–71. Éditions du CNRS, Paris, 1984, 1984.

Markus Lazar and Helmut O K Kirchner. Dislocation loops in anisotropic elasticity: displacement field, stress function tensor and interaction energy. *Philosophical Magazine*, 93(1-3):174–185, 2013.

I.M. Lifshitz and L.N. Rosenzweig. On the construction of the Green tensor for the basic equation of the theory of elasticity of an anisotropic medium. *Zh. Eksper. Teor. Fiz*, 17:783–791, 1947.

V A Lubarda, J A Blume, and A Needleman. An analysis of equilibrium dislocation distributions. *Acta Metall Mater*, 41(2):625–642, 1993.

Toshio Mura. *Micromechanics of Defects in Solids*. Kluwer Academic Publishers, Dordrecht, The Netherlands, 2nd edition, 1987.

Giacomo Po, Markus Lazar, Nikhil Chandra Admal, and Nasr Ghoniem. A non-singular theory of dislocations in anisotropic crystals. *International Journal of Plasticity*, 103:1–22, 2018.

G Scovazzi and T. J. R. Hughes. Lecture Notes on Continuum Mechanics on Arbitrary Moving Domains, 2007.

John C. Slattery, Leonard Sagis, and Eun-Suok Oh. *Interfacial Transport Phenomena*. Springer, 2 edition, December 2006.

J.L. Synge. *The Hypercircle in Mathematical Physics*. Cambridge University Press, Cambridge, 1957.

Erik Van der Giessen and A Needleman. Discrete Dislocation Plasticity - a Simple Planar Model. *Model Simul Mater Sc*, 3(5):689–735, 1995.

V.S. Vladimirov. *Equations of Mathematical Physics*. Marcel Dekker, Inc., New York, 1971.

Christopher R Weinberger, Sylvie Aubry, Seok-Woo Lee, William D. Nix, and Wei Cai. Modelling dislocations in a free-standing thin film. *Model Simul Mater Sc*, 17(7):–, 2009.

Daniel Weygand, LH Friedman, Erik Van der Giessen, and A Needleman. Aspects of boundary-value problem solutions with three-dimensional dislocation dynamics. *Model Simul Mater Sc*, 10(4):437–468, 2002.